ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ
ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

Πτυχιακή Εργασία

Property Co-Pilot

Σύστημα υποστήριξης αποφάσεων για ακίνητα
με μηχανική μάθηση και εξηγήσιμες εκτιμήσεις

Property Co-Pilot
A decision-support system for real estate
with machine learning and explainable estimates

Ονοματεπώνυμο φοιτητή: Αντώνιος Λειβαδίτης
Αριθμός μητρώου: π22085
Πατρώνυμο: Γεώργιος
Επιβλέπων: Ιωάννης Δαυγιώτης

Πειραιάς, Οκτώβριος 2026

Πνευματικά δικαιώματα

Copyright © 2026. Αντώνιος Λειβαδίτης.

Επιτρέπεται η χρήση της εργασίας για μη εμπορικούς, εκπαιδευτικούς και ερευνητικούς σκοπούς με αναφορά της πηγής. Η εμπορική αναπαραγωγή, αποθήκευση ή διανομή του συνόλου ή μέρους της απαιτεί άδεια του δικαιούχου.

Οι απόψεις και τα συμπεράσματα της εργασίας αποδίδονται στον συγγραφέα και δεν αποτελούν επίσημες θέσεις του Πανεπιστημίου Πειραιώς.

Περίληψη / Abstract

Η εργασία παρουσιάζει το Property Co-Pilot, ένα διαδικτυακό σύστημα υποστήριξης αποφάσεων για κατοικίες πώλησης και μίσθωσης. Κίνητρο είναι η μείωση της πληροφοριακής ασυμμετρίας: η δυνατότητα σύγκρισης αγγελιών, εντοπισμού πιθανής υπερτιμολόγησης και αποφυγής δυσμενών συμφωνιών με περισσότερα τεκμηριωμένα στοιχεία. Η εφαρμογή δεν πιστοποιεί αξίες ή προθέσεις πωλητών και δεν αποδεικνύει συνολική διόρθωση της αγοράς.

Το σύστημα συνδυάζει React/TypeScript, Go, PostgreSQL/PostGIS και Python/FastAPI. Δύο φάσεις XGBoost συνδέουν χαρακτηριστικά κατοικίας με χωρικά κατάλοιπα. ONNX και UBJ εξυπηρετούν αντίστοιχα πρόβλεψη και native SHAP, ενώ manifests και hashes συντονίζουν τη φόρτωση. Η μελέτη αναλύει την αρχικοποίηση, τα συμβόλαια, την αποθήκευση, την ευρετική κατάταξη και τα προσωρινά μοντέλα περιοχής. Οι διαθέσιμες μετρικές δείχνουν ασθενή επικύρωση μισθώσεων και η target-derived χωρική πληροφορία απαιτεί καθαρότερο πειραματικό διαχωρισμό. Η συμβολή είναι η λειτουργική ολοκλήρωση εξηγήσιμης ανάλυσης με ρητή καταγραφή των ορίων της.

Επιστημονική περιοχή: Εφαρμοσμένη μηχανική μάθηση και μηχανική λογισμικού.

Λέξεις-κλειδιά: ακίνητα, XGBoost, SHAP, γεωχωρική ανάλυση, υποστήριξη αποφάσεων.

Abstract

This thesis presents Property Co-Pilot, a web-based decision-support system for residential sale and rental listings. Its motivation is to reduce information asymmetry by enabling comparable listing views, identifying potentially excessive asking prices and supporting better-informed negotiation. It does not certify market value, infer seller misconduct or demonstrate a market-wide price correction.

The system combines React/TypeScript, Go, PostgreSQL/PostGIS and Python/FastAPI. A two-stage XGBoost pipeline enriches property attributes with spatial residuals. ONNX serves inference, native UBJ models support SHAP explanations, and manifests with hashes coordinate loading. The study examines initialization, data contracts, persistence, heuristic ranking and temporary area models. Stored metrics reveal weak rental validation, while target-derived spatial features require a cleaner evaluation protocol. The contribution is an integrated explainable analysis workflow with explicit numerical, operational and scientific limitations.

Scientific field: Applied machine learning and software engineering.

Keywords: real estate, XGBoost, SHAP, geospatial analysis, decision support.

Πίνακας περιεχομένων

Πνευματικά δικαιώματα i
Περίληψη / Abstract ii
Abstract ii
1. Εισαγωγή 1
1.1 Το πρόβλημα της πληροφόρησης στην αγορά ακινήτων 1
1.2 Σκοπός και επιμέρους στόχοι 1
1.3 Επιστημονική περιοχή και ερευνητικά ερωτήματα 2
1.4 Συνοπτική παρουσίαση της εφαρμογής 2
1.5 Όρια του αντικειμένου και οργάνωση του κειμένου 3
2. Επιστημονικό υπόβαθρο και σχετικές εργασίες 4
2.1 Από τα χαρακτηριστικά στην τιμή 4
2.2 Αυτοματοποιημένη αποτίμηση και τιμές αγγελιών 4
2.3 Gradient boosting και XGBoost 4
2.4 Χωρική πληροφορία και κίνδυνος διαρροής 5
2.5 Εξηγησιμότητα και συνεισφορές SHAP 5
2.6 Ομαδοποίηση και σύγκριση ομοειδών κατοικιών 6
2.7 Σύνθεση βιβλιογραφίας και θέση της εργασίας 6
3. Απαιτήσεις, δεδομένα και μεθοδολογία 7
3.1 Μεθοδολογική προσέγγιση 7
3.2 Λειτουργικές απαιτήσεις και αποδοχή 7
3.3 Ποιότητα και μη λειτουργικές απαιτήσεις 8
3.4 Προέλευση, πληθυσμός και περιορισμοί δεδομένων 8
3.5 Μονάδες και σημασία των βασικών μεγεθών 9
4. Αρχιτεκτονική, σκελετός και αρχικοποίηση 10
4.1 Διάσπαση σε υποσυστήματα 10
4.2 Σκελετός αποθετηρίου 10
4.3 Η application ως σημείο σύνθεσης 11
4.4 Εκκίνηση Go και διαθέσιμα μοντέλα 13
4.5 Κατασκευή ModelPredictor και native πόροι 14
4.6 Mutex, παράλληλα αιτήματα και αλλαγή γενιάς 15
4.7 Εκκίνηση Python και ml_shared 16
5. Σχεδιασμός δεδομένων και συμβόλαιο χαρακτηριστικών 17
5.1 Οντότητες και διαχωρισμός ευθυνών 17
5.2 Σχήμα μεγαλύτερο από το ενεργό προϊόν 18
5.3 Αποθήκευση εξηγήσεων 18
5.4 Τι αποθηκεύει τελικά το global_stats 19
5.5 Είσοδος χρήστη και τελικό διάνυσμα 20
5.6 Κατηγορικές μεταβλητές και χρονικές συμβάσεις 23
6. Η καθολική αλυσίδα εκπαίδευσης 24
6.1 Συντονισμός στη Go 24
6.2 Προετοιμασία δεδομένων στο Tree.py 25
6.3 Δύο μοντέλα για δύο διαφορετικές μετρικές 27
6.4 GeoPremium: κατασκευή και ερμηνεία 27

6.5 Phase2 και παραγωγή των artifacts 28
6.6 Μαθηματική κατάταξη Elo 29
7. Εξηγησιμότητα, ροές και δημοσίευση μοντέλων 31
7.1 Γιατί υπάρχει ξεχωριστό πακέτο explainability 31
7.2 Τοπικές συνεισφορές και καθολική σημασία 31
7.3 Αλληλεπιδράσεις χαρακτηριστικών και κόστος 32
7.4 Αριθμητική προσθετικότητα και ανοχές 33
7.5 Πρωτόκολλο stdout και προοδευτική επεξεργασία 34
7.6 Reader, pipe, Scanner και context 35
7.7 Staging και δημοσίευση στη βάση 37
7.8 UBJ, manifest και SHA-256 38
7.9 Latest-only ενεργοποίηση αρχείων 40
7.10 Reload: ποιος ειδοποιεί ποιον 41
8. Backend, αιτήματα χρήστη και ανάλυση περιοχής 44
8.1 Authentication και όρια πρόσβασης 44
8.2 Αποκωδικοποίηση εισόδου και προαιρετικές τιμές 44
8.3 Διεύθυνση, επιλογή provider και αποθήκευση 45
8.4 Online εξήγηση της νέας κατοικίας 46
8.5 Browse: φίλτρα, ορθογώνιο και pagination 47
8.6 Portfolio, εικόνες και αναγνωστικό snapshot 48
8.7 Περιοχή, autocomplete και Nominatim 50
8.8 Ανάλυση περιοχής και προσωρινά μοντέλα 51
9. Διεπαφή React και διαδρομές χρήσης 53
9.1 Κατάσταση εφαρμογής και πλοήγηση 53
9.2 Ένα συμβόλαιο δεδομένων για κάρτες και λεπτομέρειες 53
9.3 Autocomplete με αποφυγή παλιών απαντήσεων 55
9.4 Browse και διατήρηση φίλτρων 56
9.5 Προσθήκη και λεπτομέρειες χαρτοφυλακίου 57
9.6 Analysis και καταστάσεις αποτυχίας 57
10. Αξιολόγηση, αποτελέσματα και απειλές εγκυρότητας 58
10.1 Τρεις διαφορετικές έννοιες επιτυχίας 58
10.2 Αναπαραγώγιμοι έλεγχοι λογισμικού 58
10.3 Διαθέσιμα αποτελέσματα μοντέλων 59
10.4 Αριθμητικό παράδειγμα ερμηνείας, όχι εμπειρικό εύρημα 59
10.5 Προτεινόμενο πρωτόκολλο ανεξάρτητης αξιολόγησης 60
10.6 Απειλές εγκυρότητας και υπεύθυνη χρήση 61
11. Εκκίνηση, λειτουργία και συντήρηση 62
11.1 Προϋποθέσεις περιβάλλοντος 62
11.2 Πραγματική σειρά εκκίνησης 62
11.3 Έλεγχος βασικών διαδρομών χωρίς εκπαίδευση 62
11.4 Διάγνωση ανά σύμπτωμα 63
11.5 Ασφάλεια και προτεραιότητες πριν από δημόσια διάθεση 64
12. Συμπεράσματα και μελλοντική εργασία 65
12.1 Αποτίμηση των στόχων 65

12.2 Περιορισμοί που καθορίζουν την επόμενη έκδοση 65
12.3 Τελική συμβολή 65
Πίνακας ορολογίας 66
Πίνακας συντμήσεων και ακρωνυμίων 68
Βιβλιογραφία 69
Παράρτημα Α. Συμβόλαια HTTP και διαδρομές 71
Α.1 Routes που χρησιμοποιεί η διεπαφή 71
Α.2 Εσωτερικό συμβόλαιο Python 72
Α.3 Παράδειγμα μορφής επιτυχίας προσθήκης 72
Παράρτημα Β. Ολοκληρωμένος κώδικας κρίσιμης ροής 73
Β.1 Εκτέλεση trainer και δημοσίευση SHAP 73
Β.2 Πλήρης μαθηματικός ορισμός του score στον κοινό helper 75
Β.3 Αναπαραγώγιμη επιλογή δείγματος αλληλεπιδράσεων 75
Παράρτημα Γ. Χάρτης ανάγνωσης και αναπαραγωγής 76
Γ.1 Από το σύμπτωμα στο αρχείο 76
Γ.2 Εντολές επαλήθευσης 76
Γ.3 Τι απαιτείται για πλήρη επιστημονική αναπαραγωγή 77

Κατάλογοι σχημάτων και πινάκων

Κατάλογος διαγραμμάτων

Διάγραμμα 1.1. Επισκόπηση λειτουργικών διαδρομών 3
Διάγραμμα 4.1. Δομή εκτέλεσης και επικοινωνίας 10
Διάγραμμα 5.1. Λογικός πυρήνας δεδομένων, όχι πλήρες ER όλων των πινάκων 17
Διάγραμμα 6.1. Ροή εκπαίδευσης δύο φάσεων 24
Διάγραμμα 8.1. Προσθήκη κατοικίας και σημεία αποθήκευσης 46
Διάγραμμα 10.1. Προτεινόμενη αξιολόγηση χωρίς προϋπολογισμό στο σύνολο 61

Κατάλογος πινάκων

Πίνακας 2.1. Σύνδεση επιστημονικών προσεγγίσεων με το έργο 6
Πίνακας 3.1. Ιχνηλασιμότητα βασικών απαιτήσεων 8
Πίνακας 3.2. Λεξικό οικονομικών και στατιστικών μεγεθών 9
Πίνακας 4.1. Κύριες δομές και διάρκεια ζωής 12
Πίνακας 5.1. Σημασιολογία εγγραφών global_stats 19
Πίνακας 5.2. Ακριβής σειρά χαρακτηριστικών Phase2 από το manifest 22
Πίνακας 10.1. Αποτελέσματα επαλήθευσης της 19/10/2026 58
Πίνακας 10.2. Μετρικές των αποθηκευμένων Phase2 μοντέλων 59
Πίνακας 10.3. Παράδειγμα επίδρασης asking με σταθερή πρόβλεψη 60
Πίνακας 11.1. Λειτουργική διάγνωση χωρίς αυθαίρετη επανεκπαίδευση 63
Πίνακας Ο.1. Βασικοί ελληνικοί και αγγλικοί όροι 66
Πίνακας Σ.1. Ανάπτυξη ακρωνυμίων που χρησιμοποιούνται στο κείμενο 68
Πίνακας Α.1. Διαδρομές και αναγνωστικά ή μεταβλητικά συμβόλαια 71
Πίνακας Α.2. Αιτήματα προς τη μικροϋπηρεσία 72
Πίνακας Γ.1. Αφετηρίες ανάγνωσης πηγαίου κώδικα 76

Κατάλογος εικόνων

Δεν χρησιμοποιούνται φωτογραφίες ή στιγμιότυπα οθόνης. Όλα τα σχήματα είναι πρωτότυπα διανυσματικά διαγράμματα της δομής και των ροών του έργου.

1. Εισαγωγή

1.1 Το πρόβλημα της πληροφόρησης στην αγορά ακινήτων

Η επιλογή κατοικίας συνδυάζει οικονομικούς περιορισμούς, προσωπικές ανάγκες και αβεβαιότητα. Μια αγγελία παρουσιάζει μια ζητούμενη τιμή, αλλά ο ενδιαφερόμενος δεν γνωρίζει εύκολα αν αυτή δικαιολογείται από τα χαρακτηριστικά του ακινήτου και τις συνθήκες της περιοχής. Δύο κατοικίες με ίδιο εμβαδόν μπορεί να διαφέρουν ουσιαστικά ως προς ηλικία, κατάσταση, όροφο, θέρμανση, θέα και θέση. Η απλή ταξινόμηση κατά συνολικό κόστος δεν λαμβάνει υπόψη αυτές τις διαφορές. Ακόμη και η σύγκριση σε ευρώ ανά τετραγωνικό μέτρο αποτελεί μόνο αφετηρία: εξομοιώνει το μέγεθος, όχι την ποιότητα ή το περιβάλλον του ακινήτου.

Το πρόβλημα που εξετάζει η εργασία είναι η μετατροπή διάσπαρτων χαρακτηριστικών και αγγελιών σε συγκρίσιμη, ερμηνεύσιμη πληροφορία. Ο αγοραστής ή ενοικιαστής χρειάζεται όχι μόνο έναν αριθμό εκτίμησης, αλλά και τη δυνατότητα να εξετάσει γιατί προκύπτει. Ο ιδιοκτήτης, αντίστοιχα, χρειάζεται ένα σημείο αναφοράς για το χαρτοφυλάκιό του. Η εφαρμογή Property Co-Pilot συνδυάζει τις δύο οπτικές: περιήγηση στην αγορά και εξέταση συγκεκριμένων ακινήτων με κοινές μονάδες μέτρησης και κοινή λογική προβλέψεων.

Βασικό κοινωνικοοικονομικό κίνητρο είναι η υποστήριξη μιας περισσότερο διαφανούς αγοράς. Η συστηματική ανάδειξη πιθανώς υπερτιμημένων αγγελιών μπορεί να προσφέρει στον χρήστη επιχειρήματα διαπραγμάτευσης και να τον βοηθήσει να αποφύγει δυσμενείς συμφωνίες. Με αυτή την έννοια, το έργο επιδιώκει να συμβάλει στη διόρθωση στρεβλών προσδοκιών τιμολόγησης. Δεν επιχειρεί να επιβάλει τιμές, να χαρακτηρίσει πωλητές ως κακόπιστους ή να πιστοποιήσει την πραγματική αξία ενός ακινήτου. Η απόκλιση μιας αγγελίας από ένα μοντέλο είναι ένδειξη προς διερεύνηση, όχι απόδειξη παραπλάνησης.

Η διάκριση αυτή είναι ιδιαίτερα σημαντική επειδή το μοντέλο μαθαίνει από ζητούμενες τιμές. Αν οι αγγελίες μιας περιοχής είναι συστηματικά ακριβές, η ίδια η εκτίμηση μπορεί να αναπαράγει αυτό το επίπεδο. Ένα σύστημα που εκπαιδεύεται σε τιμές προσφοράς δεν μπορεί, χωρίς πρόσθετα στοιχεία, να αποδείξει ότι ολόκληρη η αγορά είναι υπερτιμημένη. Η ασφαλής χρησιμότητά του βρίσκεται στη συγκριτική πληροφόρηση, στην αποκάλυψη διαφορών μεταξύ παρόμοιων καταχωρίσεων και στην εξήγηση της δικής του αριθμητικής συμπεριφοράς.

1.2 Σκοπός και επιμέρους στόχοι

Σκοπός της πτυχιακής είναι ο σχεδιασμός, η υλοποίηση και η κριτική αποτίμηση ενός διαδικτυακού συστήματος υποστήριξης αποφάσεων για οικιστικά ακίνητα. Η εφαρμογή πρέπει να συνδέει τη διαχείριση δεδομένων με γεωχωρική αναζήτηση, μηχανική μάθηση και εξηγήσιμα αποτελέσματα. Δεν αρκεί η απομονωμένη εκπαίδευση ενός δέντρου: το αποτέλεσμα πρέπει να φθάνει στον χρήστη μέσα από λειτουργικές διαδρομές αναζήτησης, χαρτοφυλακίου, προσθήκης και λεπτομερούς προβολής.

Ο πρώτος τεχνικός στόχος είναι η συμβατότητα εκπαίδευσης και πρόβλεψης. Τα δεδομένα διέρχονται από SQL, δομές Go, CSV, pandas, XGBoost και ONNX. Μια αλλαγή στη σειρά των στηλών θα μπορούσε να παράγει αριθμητικά έγκυρη αλλά σημασιολογικά λανθασμένη πρόβλεψη. Γι' αυτό η εργασία αντιμετωπίζει τη σειρά, τους τύπους και τις μονάδες των χαρακτηριστικών ως ρητό συμβόλαιο και όχι ως λεπτομέρεια υλοποίησης.

Ο δεύτερος στόχος είναι η σύνδεση της συνολικής εκτίμησης με ερμηνεύσιμες συνεισφορές χαρακτηριστικών. Η χρήση SHAP επιτρέπει την παρουσίαση θετικών και αρνητικών επιδράσεων σε ευρώ ανά τετραγωνικό μέτρο. Ο τρίτος στόχος αφορά την ενσωμάτωση της θέσης μέσω χωρικών καταλοίπων και την παραγωγή μιας ευρετικής κατάταξης. Ο τέταρτος αφορά τη λειτουργική ολοκλήρωση: αποθήκευση, ελεγχόμενη δημοσίευση μοντέλων, επαναφόρτωση, διεπαφή και δοκιμές.

Ο τελικός στόχος είναι επιστημονικά ειλικρινής αξιολόγηση. Η επιτυχής εκτέλεση ενός endpoint δεν ισοδυναμεί με καλή πρόβλεψη, ενώ μια υψηλή προσαρμογή στα δεδομένα εκπαίδευσης δεν αποδεικνύει γενίκευση. Η εργασία καταγράφει ξεχωριστά τα αποτελέσματα των

ελέγχων λογισμικού, τις διαθέσιμες μετρικές των αποθηκευμένων μοντέλων και τις μελλοντικές μετρήσεις που απαιτούνται για τεκμηρίωση πραγματικού οφέλους στην αγορά.

1.3 Επιστημονική περιοχή και ερευνητικά ερωτήματα

Το έργο εντάσσεται στην εφαρμοσμένη μηχανική μάθηση και στη μηχανική λογισμικού, με συναφή πεδία τα γεωγραφικά πληροφοριακά συστήματα, την εξηγήσιμη τεχνητή νοημοσύνη και τα συστήματα υποστήριξης αποφάσεων. Η πρόβλεψη είναι πρόβλημα επιβλεπόμενης παλινδρόμησης. Η ομαδοποίηση περιοχής είναι πρόβλημα μη επιβλεπόμενης μάθησης. Η μεταφορά ενός μοντέλου ανάμεσα σε διαφορετικές γλώσσες και διεργασίες είναι πρόβλημα ολοκλήρωσης και διαχείρισης κύκλου ζωής λογισμικού.

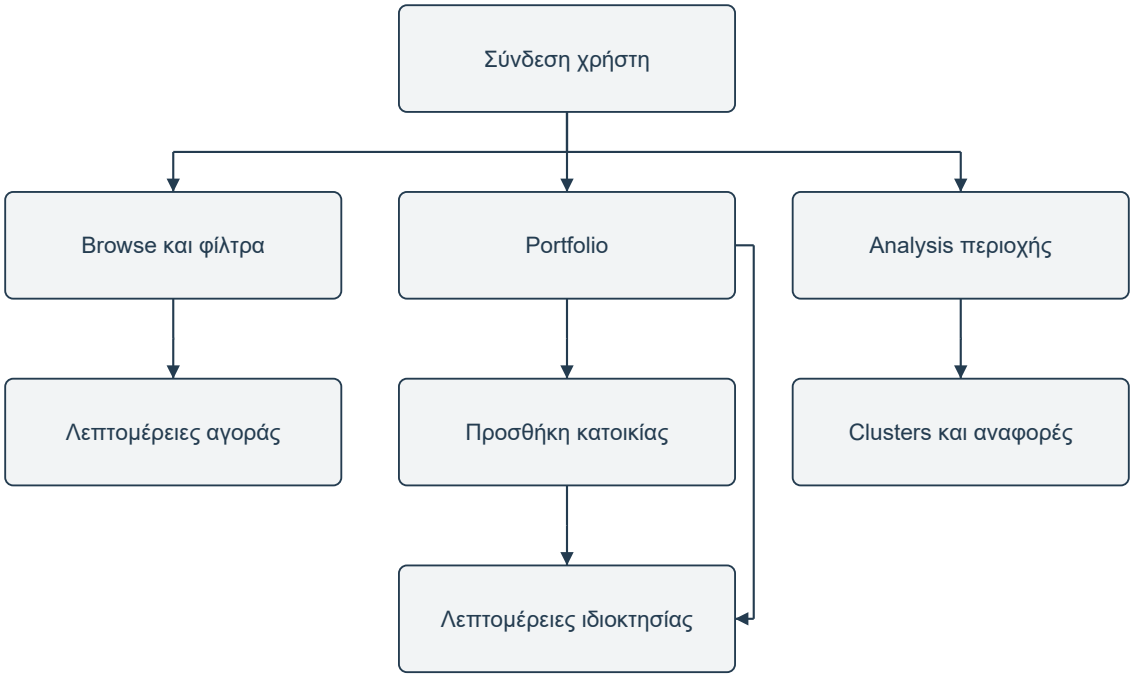
Τα ερευνητικά ερωτήματα οργανώνονται ως εξής. ΕΕ1: μπορεί να διατηρηθεί ένα ενιαίο συμβόλαιο μεταξύ μαζικής εκπαίδευσης, online πρόβλεψης και online εξήγησης; ΕΕ2: με ποιον τρόπο μπορεί να ενσωματωθεί η τοπική χωρική πληροφορία, χωρίς να συγχέεται με αποδεδειγμένη αιτιακή επίδραση; ΕΕ3: πώς πρέπει να παρουσιαστούν εκτίμηση, κατάλοιπο, κατάταξη και SHAP ώστε να μην παράγουν παραπλανητικά συμπεράσματα; ΕΕ4: ποια στοιχεία των διαθέσιμων ελέγχων τεκμηριώνουν την ορθότητα της υλοποίησης και ποια αφήνουν ανοιχτή την ποιότητα γενίκευσης;

Τα ερωτήματα δεν προϋποθέτουν ότι η εφαρμογή εισάγει νέο αλγόριθμο boosting ή νέα θεωρία Shapley. Η συνεισφορά είναι εφαρμοσμένη σύνθεση: ένα λειτουργικό σύστημα με σαφή διαχωρισμό ροών, ελέγχους συμβολαίων και καταγεγραμμένους περιορισμούς. Η μελέτη είναι περιπτωσιολογική. Δεν περιλαμβάνει τυχαιοποιημένο πείραμα με αγοραστές, οικονομική αποτίμηση της εξοικονόμησης ή σύγκριση με πιστοποιημένους εκτιμητές.

1.4 Συνοπτική παρουσίαση της εφαρμογής

Ο χρήστης συνδέεται και εισέρχεται στη σελίδα Browse. Εκεί μπορεί να επιλέξει διοικητική περιοχή, πώληση ή μίσθωση, εύρος ζητούμενης τιμής ανά τ.μ. και εύρος εμβαδού. Τα αποτελέσματα εμφανίζονται σε σελίδες είκοσι ακινήτων. Κάθε σύνοψη περιλαμβάνει βασικά οικονομικά και περιγραφικά στοιχεία. Όταν δεν υπάρχει εικόνα, η διεπαφή εμφανίζει εικονογράφηση SVG, χωρίς να κατασκευάζει φωτογραφία του ακινήτου.

Η σελίδα Portfolio συγκεντρώνει τα ακίνητα που συνδέονται με τον λογαριασμό. Παρέχει πλήθος και συνολική εκτιμώμενη αξία, ενώ η φόρμα Add property επιτρέπει την εισαγωγή νέας κατοικίας με επαληθευμένη επιλογή διεύθυνσης. Η ζητούμενη τιμή είναι προαιρετική. Η σελίδα λεπτομερειών εμφανίζει τα αποθηκευμένα χαρακτηριστικά, την πρόβλεψη, τη βαθμολογία Elo, το GeoPremium και τις διαθέσιμες εξηγήσεις. Η Analysis δημιουργεί περιγραφικές ομάδες και προσωρινά μοντέλα για επιλεγμένη περιοχή.



Διάγραμμα 1.1. Επισκόπηση λειτουργικών διαδρομών

1.5 Όρια του αντικειμένου και οργάνωση του κειμένου

Το αντικείμενο περιορίζεται στην υλοποίηση του διαθέσιμου bundle κατά την 19η Οκτωβρίου 2026. Κύρια πηγή τεκμηρίωσης είναι ο πρώτος πηγαίος κώδικας Go, Python και React/TypeScript, το κοινό πακέτο ml_shared και το σχήμα PostgreSQL. Οι εξαρτήσεις και τα παραγόμενα αρχεία δεν θεωρούνται πρωτότυπη υλοποίηση. Δεν αποδίδονται στο έργο λειτουργίες επειδή απλώς υπάρχουν αντίστοιχα ονόματα πινάκων στη βάση.

Η οργάνωση ακολουθεί τη λογική του παρεχόμενου κανονισμού: εισαγωγή, επιστημονικό υπόβαθρο, απαιτήσεις και μέθοδος, σχεδιασμός, υλοποίηση, αξιολόγηση και συμπεράσματα. Τα παραδείγματα πτυχιακών των Φρονίστα (2025) και Πάκια (2025), διαθέσιμα στη ΔΙΩΝΗ, εξετάστηκαν ως προς τη διάκριση εισαγωγής, υποβάθρου, κύριων κεφαλαίων και συμπερασμάτων. Δεν αποτελούν πηγές ισχυρισμών για την αγορά ακινήτων ούτε πρότυπα προς αντιγραφή περιεχομένου.

Τα κεφάλαια 2 και 3 θεμελιώνουν τις έννοιες και τη μεθοδολογία. Τα κεφάλαια 4 έως 9 αναλύουν την πραγματική υλοποίηση, από τις δομές εκκίνησης μέχρι τη διεπαφή. Τα κεφάλαια 10 και 11 παρουσιάζουν επαλήθευση, κριτική αποτίμηση και λειτουργία περιβάλλοντος. Ακολουθούν συμπεράσματα, ορολογία, βιβλιογραφία και παράρτημα επιλεγμένου κώδικα. Τα αποσπάσματα προέρχονται από τα αρχεία που κατονομάζονται και συνοδεύονται από γραμμές πηγής, ώστε η περιγραφή να παραμένει ελέγξιμη.

2. Επιστημονικό υπόβαθρο και σχετικές εργασίες

2.1 Από τα χαρακτηριστικά στην τιμή

Η ηδονική προσέγγιση τιμών (hedonic pricing) αντιμετωπίζει ένα διαφοροποιημένο αγαθό ως σύνολο χαρακτηριστικών. Ο Rosen (1974) θεμελιώνει τη σύνδεση παρατηρούμενων τιμών και χαρακτηριστικών σε ένα οικονομικό πλαίσιο. Για την παρούσα εφαρμογή, η χρήσιμη ιδέα είναι ότι η τιμή δεν συνδέεται μόνο με ένα φυσικό μέγεθος, αλλά με συνδυασμό γνωρισμάτων. Η χρήση αυτής της ιδέας δεν σημαίνει ότι η υλοποίηση εκτιμά το πλήρες οικονομικό υπόδειγμα του Rosen ή ότι τα δέντρα αποδίδουν αιτιακές «τιμές» σε κάθε γνώρισμα.

Μια γραμμική αφετηρία θα μπορούσε να συνδέει την τιμή ανά τ.μ. με εμβαδόν, ηλικία και δυαδικούς δείκτες παροχών. Το πλεονέκτημα είναι η απλή ερμηνεία παραμέτρων υπό τις κατάλληλες υποθέσεις. Το μειονέκτημα είναι ότι απαιτείται να επιλεγούν ρητά μη γραμμικοί όροι και αλληλεπιδράσεις. Για παράδειγμα, η διαφορά ανάμεσα στον πρώτο και στον δεύτερο όροφο μπορεί να μην έχει την ίδια οικονομική σημασία με τη διαφορά ανάμεσα στον έκτο και στον έβδομο, ιδίως όταν δεν υπάρχει ανελκυστήρας.

Η συγκεκριμένη εργασία επιλέγει δέντρα ενίσχυσης για να αναπαραστήσει τέτοιες εξαρτήσεις χωρίς να προδιαγράψει όλες τις μορφές τους. Η επιλογή είναι αρχιτεκτονική και εμπειρική, όχι απόδειξη υπεροχής. Στο διαθέσιμο έργο δεν υπάρχει ολοκληρωμένο πείραμα σύγκρισης με γραμμική παλινδρόμηση, Random Forest ή απλή διάμεσο περιοχής. Συνεπώς, μια σωστή ανάγνωση είναι «το σύστημα χρησιμοποιεί XGBoost», όχι «το XGBoost αποδείχθηκε το καλύτερο μοντέλο για τα δεδομένα αυτά».

2.2 Αυτοματοποιημένη αποτίμηση και τιμές αγγελιών

Τα συστήματα αυτοματοποιημένης αποτίμησης (automated valuation models, AVM) παράγουν αριθμητικές εκτιμήσεις από περιγραφικά και χωρικά δεδομένα. Η εργασία των Antipov και Pokryshevskaya (2012) εξετάζει Random Forest για μαζική αποτίμηση διαμερισμάτων και διαγνωστική αξιολόγηση με δέντρα. Το στοιχείο που συνδέεται με το παρόν έργο είναι η δυνατότητα χρήσης συνόλων δέντρων σε ετερογενή χαρακτηριστικά κατοικιών. Δεν μεταφέρονται οι επιδόσεις εκείνης της μελέτης στο ελληνικό σύνολο αγγελιών.

Στο Property Co-Pilot ο στόχος ονομάζεται AskingPricePerSqm. Η ονομασία είναι ουσιαστική: η ετικέτα αναφέρεται στη ζητούμενη τιμή της καταχώρισης. Το ποσό μπορεί να διαφέρει από την τελική συμφωνία, να περιέχει προσδοκίες διαπραγμάτευσης ή να έχει παραμείνει αμετάβλητο ενώ οι συνθήκες έχουν αλλάξει. Επομένως, η εκτίμηση πρέπει να περιγράφεται ως μοντελοποιημένη ζητούμενη τιμή και όχι ως πιστοποιημένη εμπορική αξία.

Η διάκριση επηρεάζει και την αξιολόγηση υπερτίμησης. Αν ζητούνται 3.000 €/τ.μ. και το μοντέλο προβλέπει 2.500 €/τ.μ., το σύστημα παρατηρεί απόκλιση από το μαθημένο πρότυπο. Δεν γνωρίζει αν υπάρχει μη καταγεγραμμένη πρόσφατη ανακαίνιση, ειδικό νομικό καθεστώς, ιδιαίτερη ποιότητα κατασκευής ή σφάλμα στη βάση. Ο χρήστης πρέπει να χρησιμοποιήσει την ένδειξη ως αφορμή για έλεγχο. Η τελική απόφαση απαιτεί αυτοψία και επαλήθευση των στοιχείων.

2.3 Gradient boosting και XGBoost

Η ενίσχυση κλίσης (gradient boosting) κατασκευάζει ένα αθροιστικό μοντέλο. Σε κάθε βήμα προστίθεται ένα δέντρο που επιδιώκει να μειώσει το υπόλοιπο σφάλμα, σύμφωνα με την επιλεγμένη συνάρτηση απώλειας. Οι Chen και Guestrin (2016) παρουσιάζουν το XGBoost ως αποδοτική υλοποίηση ενισχυμένων δέντρων με μηχανισμούς κανονικοποίησης και υπολογιστικής κλιμάκωσης. Η βιβλιογραφική αναφορά θεμελιώνει τη μέθοδο, ενώ οι συγκεκριμένες υπερπαράμετροι της εφαρμογής προκύπτουν από τον trainer.

$$F(x) = b + \Sigma[t=1..T] \eta \cdot h_t(x) \quad (2.1)$$

Στην εξίσωση, b είναι αρχική σταθερά, h_t ένα δέντρο και η ο ρυθμός μάθησης. Η αθροιστική δομή εξηγεί γιατί η τελική πρόβλεψη δεν αντιστοιχεί σε μία απλή διαδρομή ενός μόνο δέντρου. Διαφορετικά δέντρα μπορούν να αξιοποιούν διαφορετικά υποσύνολα χαρακτηριστικών. Το γεγονός αυτό αυξάνει την ευελιξία αλλά δυσκολεύει τη χειροκίνητη ερμηνεία, δημιουργώντας ανάγκη για χωριστό μηχανισμό εξήγησης.

Στον καθολικό trainer χρησιμοποιούνται 500 δέντρα, μέγιστο βάθος 8, learning_rate 0,05, min_child_weight 30 και δειγματοληψίες γραμμών και στηλών 0,8. Ο αλγόριθμος δέντρων είναι hist. Οι τιμές αυτές δεν πρέπει να αντιμετωπιστούν ως βέλτιστες επειδή υπάρχουν στον κώδικα. Δεν τεκμηριώνεται αναζήτηση υπερπαραμέτρων με ανεξάρτητο σύνολο ελέγχου. Αποτελούν σταθερή διαμόρφωση της εξεταζόμενης υλοποίησης.

2.4 Χωρική πληροφορία και κίνδυνος διαρροής

Η θέση είναι ιδιαίτερο χαρακτηριστικό επειδή γειτονικά ακίνητα μοιράζονται υπηρεσίες, προσβασιμότητα και τοπικές συνθήκες. Η εφαρμογή δεν εισάγει απλώς ένα όνομα περιοχής. Κατασκευάζει ένα πρόσθετο αριθμητικό χαρακτηριστικό από κατάλοιπα γειτόνων του πρώτου μοντέλου. Με αυτό τον τρόπο επιχειρεί να αποτυπώσει συστηματικές τοπικές αποκλίσεις που δεν εξηγούνται από τα υπόλοιπα γνωρίσματα.

Η χρήση γειτονικών ετικετών δημιουργεί όμως ειδικές απαιτήσεις αξιολόγησης. Οι Roberts και συν. (2017) αναλύουν τη σημασία κατάλληλων διαχωρισμών επικύρωσης όταν οι παρατηρήσεις έχουν χωρική, χρονική ή άλλη εξαρτημένη δομή. Η εφαρμογή στο παρόν έργο είναι αναλογική: ένας τυχαίος διαχωρισμός γραμμών μπορεί να αφήσει πολύ κοντινές ή σχετιζόμενες παρατηρήσεις και στα δύο σύνολα, κάνοντας το πρόβλημα ευκολότερο από την πρόβλεψη σε πραγματικά νέα περιοχή.

Επιπλέον, το GeoPremium παράγεται πριν από το validation split της Phase2. Έτσι, η τελική είσοδος επικύρωσης έχει κατασκευαστεί από μια προγενέστερη διαδικασία που αξιοποίησε το σύνολο των ετικετών. Αυτό δεν ακυρώνει τη λειτουργία της εφαρμογής, αλλά περιορίζει την ερμηνεία της μετρικής. Το σωστό πειραματικό πρωτόκολλο πρέπει να επανακατασκευάζει το χωρικό χαρακτηριστικό μέσα στο εκάστοτε σύνολο εκπαίδευσης και να χρησιμοποιεί μόνο διαθέσιμη πληροφορία για τις παρατηρήσεις ελέγχου.

2.5 Εξηγησιμότητα και συνεισφορές SHAP

Οι Lundberg και Lee (2017) εισάγουν τη SHAP ως ενιαία προσθετική προσέγγιση απόδοσης σημασίας χαρακτηριστικών σε συγκεκριμένες προβλέψεις. Η βασική ιδέα που αξιοποιείται εδώ είναι ότι η πρόβλεψη αποσυντίθεται σε τιμή αναφοράς και επιμέρους συνεισφορές. Η προσέγγιση επιτρέπει να εξηγείται διαφορετικά το ίδιο γνώρισμα σε δύο ακίνητα, αντί να αποδίδεται σε κάθε χαρακτηριστικό ένας σταθερός γενικός συντελεστής.

$$f(x) = \varphi_0 + \Sigma[j=1..m]\ \varphi_j(x) \quad (2.2)$$

Η φ_0 είναι η baseline του μοντέλου και οι φ_j οι προσημασμένες συνεισφορές. Για την παρούσα παλινδρόμηση η μονάδα είναι €/τ.μ. Η baseline δεν είναι μία τιμή ανά χαρακτηριστικό και δεν είναι απαραίτητα η απλή μέση ζητούμενη τιμή που θα υπολόγιζε ο χρήστης από τις αγγελίες. Προκύπτει από τον συγκεκριμένο μηχανισμό εξήγησης του εκπαιδευμένου μοντέλου. Η εφαρμογή τη λαμβάνει από την τελευταία στήλη του native αποτελέσματος συνεισφορών.

Οι Lundberg και συν. (2020) συνδέουν τις τοπικές εξηγήσεις δέντρων με τη συνολική κατανόηση του μοντέλου και με αλληλεπιδράσεις χαρακτηριστικών. Στο Property Co-Pilot η διάκριση υλοποιείται σε τρεις αποθηκευμένες οντότητες: εξήγηση συγκεκριμένου ακινήτου, καθολική σημασία χαρακτηριστικού και καθολική ισχύς ζεύγους χαρακτηριστικών. Πρόκειται για διαφορετικά μεγέθη, παρότι παράγονται από το ίδιο τελικό δέντρο.

Η εξήγηση αφορά το μοντέλο, όχι τον κόσμο ανεξάρτητα από αυτό. Μια θετική συνεισφορά του Parking δεν αποδεικνύει ότι η κατασκευή θέσης στάθμευσης θα αυξήσει την πραγματική τιμή κατά ακριβώς αυτό το ποσό. Το μοντέλο μπορεί να αξιοποιεί συσχετίσεις με άλλα

χαρακτηριστικά. Η πρακτική χρήση της SHAP είναι η εξέταση των λόγων της πρόβλεψης, η ανακάλυψη ύποπτων συμπεριφορών και η υποστήριξη της κριτικής ανάγνωσης.

2.6 Ομαδοποίηση και σύγκριση ομοειδών κατοικιών

Η ομαδοποίηση k-means χωρίζει παρατηρήσεις σε ομάδες γύρω από κέντρα, ελαχιστοποιώντας το άθροισμα τετραγωνικών αποστάσεων. Στο έργο δεν ομαδοποιούνται γεωγραφικές συντεταγμένες αλλά τυποποιημένα χαρακτηριστικά κατοικίας: εμβαδόν, ηλικία, ηλικία ανακαίνισης και όροφος. Η περιοχή επιλέγει πρώτα τον πληθυσμό. Έπειτα οι ομάδες εκφράζουν ομοιότητα χαρακτηριστικών μέσα σε αυτόν τον πληθυσμό.

$$J = \Sigma[k=1..K] \Sigma[x_i \in C_k] ||z_i - \mu_k||^2 \quad (2.3)$$

Η τυποποίηση είναι απαραίτητη επειδή τα γνωρίσματα έχουν διαφορετικές κλίμακες. Διαφορετικά, το εμβαδόν θα μπορούσε να κυριαρχεί στην απόσταση μόνο λόγω αριθμητικού μεγέθους. Η επιλογή αριθμού ομάδων με elbow είναι ευρετική. Μια ομάδα δεν αποτελεί αποδεδειγμένη υποαγορά και δεν έχει σταθερή ταυτότητα ανάμεσα σε δύο αναλύσεις με διαφορετικό πληθυσμό. Η παρουσίαση πρέπει επομένως να είναι περιγραφική και να συνδέει κάθε cluster με τα πραγματικά στατιστικά του.

2.7 Σύνθεση βιβλιογραφίας και θέση της εργασίας

Πίνακας 2.1. Σύνδεση επιστημονικών προσεγγίσεων με το έργο

Προσέγγιση	Τι προσφέρει	Τι υλοποιείται εδώ	Τι δεν τεκμηριώνεται
Ηδονική θεώρηση	Τιμή ως συνάρτηση γνωρισμάτων	Επιλογή περιγραφικών εισόδων	Οικονομική ή αιτιακή ταυτοποίηση
Δασικά/ενισχυμένα δέντρα	Μη γραμμικές εξαρτήσεις	XGBoost δύο φάσεων	Υπεροχή έναντι όλων των baselines
Χωρική μοντελοποίηση	Τοπικές εξαρτήσεις	Μέσο κατάλοιπο γειτόνων	Αμερόληπτη χωρική γενίκευση
SHAP	Εξήγηση πρόβλεψης	Τοπικές και συνολικές συνεισφορές	Αιτιακή επίδραση ανακαίνισης
k-means	Περιγραφική τμηματοποίηση	Clusters γνωρισμάτων περιοχής	Φυσικά ή διοικητικά όρια ομάδων

Το ερευνητικό κενό που αντιμετωπίζει το συγκεκριμένο λογισμικό δεν διατυπώνεται ως απουσία παγκοσμίως αντίστοιχων εφαρμογών. Είναι η ανάγκη μιας συνεκτικής υλοποίησης για το διαθέσιμο σύνολο δεδομένων: από την καταχώριση και τη χωρική αναζήτηση μέχρι την πρόβλεψη, την εξήγηση και την παρουσίαση. Η συνεισφορά εξετάζεται με βάση την ορθότητα αυτής της σύνθεσης και την ικανότητα να εκθέτει τους περιορισμούς της.

Η διαφοροποίηση από έναν απλό κατάλογο αγγελιών είναι ότι η εκτίμηση και η ερμηνεία αποτελούν μέρος της ίδιας διαδρομής χρήσης. Η διαφοροποίηση από ένα απομονωμένο notebook είναι ότι υπάρχουν διαρκή δεδομένα, αυθεντικοποιημένοι χρήστες, συμβόλαια HTTP και μοντέλα που επαναφορτώνονται σε ενεργές διεργασίες. Η διαφοροποίηση από πλήρες παραγωγικό AVM είναι εξίσου σημαντική: δεν υπάρχουν ακόμη ολοκληρωμένη ασφάλεια, ανεξάρτητο backtesting και εγγυημένη συναλλακτικότητα ολόκληρης της εκπαίδευσης.

3. Απαιτήσεις, δεδομένα και μεθοδολογία

3.1 Μεθοδολογική προσέγγιση

Η εργασία εξετάζει ένα λογισμικό ως τεχνούργημα εφαρμοσμένης Πληροφορικής. Η μέθοδος περιλαμβάνει διατύπωση απαιτήσεων, αντιστοίχιση απαιτήσεων σε διαδρομές κώδικα, εξέταση συμβολαίων δεδομένων και επαλήθευση με αυτοματοποιημένες δοκιμές. Το σύνολο των αποθηκευμένων μοντέλων αντιμετωπίζεται ως παρατηρούμενο αποτέλεσμα συγκεκριμένης εκτέλεσης, όχι ως νέο πείραμα που επαναλήφθηκε για τη συγγραφή. Κατά την τεκμηρίωση δεν εκτελέστηκε παραγωγική επανεκπαίδευση και δεν μεταβλήθηκαν δεδομένα χρηστών.

Η προσέγγιση συνδυάζει στατική και δυναμική εξέταση. Η στατική εξέταση ακολουθεί τις κλήσεις από το main.go προς τους handlers, τις ενέργειες χρήστη, τα pipelines και τους clients. Η δυναμική εξέταση επαναλαμβάνει τα διαθέσιμα unit και integration-style tests που χρησιμοποιούν ελεγχόμενα δεδομένα. Όπου μια δοκιμή προσομοιώνει το HTTP ή τη βάση, αυτό δηλώνεται ρητά. Η επιτυχία της δεν μετατρέπεται σε ισχυρισμό ότι πραγματοποιήθηκε πραγματική συναλλαγή σε ζωντανό περιβάλλον.

Αντικείμενο της μελέτης είναι το λειτουργικό υποσύνολο κατοικιών. Το Worker διαθέτει γενικότερες ρυθμίσεις για κτίρια, οικόπεδα, αγροτεμάχια και επαγγελματικά ακίνητα, αλλά η κύρια εφαρμογή φορτώνει δύο Phase2 μοντέλα κατοικιών: ένα για sell και ένα για rent. Η ύπαρξη παραγόμενων αρχείων άλλων κατηγοριών δεν αποδεικνύει ότι οι αντίστοιχες διαδρομές είναι προσβάσιμες από την τρέχουσα διεπαφή.

3.2 Λειτουργικές απαιτήσεις και αποδοχή

Η αναζήτηση πρέπει να είναι ελεγχόμενη ως προς το πλήθος αποτελεσμάτων και να διατηρεί το πλαίσιο του χρήστη. Το Browse επιστρέφει έως είκοσι περιλήψεις και πληροφορία ύπαρξης επόμενης σελίδας. Η επιλογή μιας περίληψης ανοίγει λεπτομέρειες χωρίς να χάνει τα εφαρμοσμένα φίλτρα. Η απαίτηση δεν περιλαμβάνει αυτόματη φόρτωση όλης της βάσης ή ακριβές συνολικό πλήθος αποτελεσμάτων.

Το προσωπικό χαρτοφυλάκιο έχει διαφορετικό όριο πρόσβασης από τη δημόσια ως προς τους συνδεδεμένους χρήστες αγορά. Η ιδιόκτητη προβολή πρέπει να επιβεβαιώνει τη σχέση χρήστη και ακινήτου. Η διαγραφή από το χαρτοφυλάκιο πρέπει να αφαιρεί μόνο αυτή τη σχέση. Η ίδια η εγγραφή και τα παράγωγα δεδομένα δεν διαγράφονται. Αυτή η διάκριση προφυλάσσει τις κοινές εγγραφές, αλλά σημαίνει επίσης ότι η ενέργεια δεν ισοδυναμεί με πλήρη διαγραφή προσωπικών δεδομένων.

Η εισαγωγή κατοικίας πρέπει να επιτρέπει απουσία ζητούμενης τιμής. Η πρόβλεψη στηρίζεται στα χαρακτηριστικά και δεν απαιτεί την τιμή ως είσοδο. Αντίθετα, το κατάλοιπο και οι όροι κατάταξης που συγκρίνουν asking και estimate απαιτούν θετική asking. Η σωστή αναπαράσταση της απουσίας είναι NULL ή nil και όχι μηδέν. Ένα μηδενικό ποσό θα μπορούσε να προκαλέσει διαίρεση με μηδέν ή να εμφανίσει πλασματική ευκαιρία.

Πίνακας 3.1. Ιχνηλασιμότητα βασικών απαιτήσεων

Κωδικός	Απαίτηση	Κύρια υλοποίηση	Κριτήριο ελέγχου
A1	Σελιδοποιημένη αγορά	BrowseProperties.go	20 εγγραφές, σταθερή ταξινόμηση, has_next
A2	Ιδιόκτητη προβολή	PropertyDetails.go	Άγνωστο ή ξένο UID επιστρέφει 404
A3	Προαιρετική asking	PropertyValidation.go, CalculateElo.go	Απουσία δεν γίνεται αριθμητικό μηδέν
A4	Ενιαία εξήγηση	explainability και ml_shared	Ίδια 30 χαρακτηριστικά, έλεγχος αθροίσματος
A5	Επιλογή περιοχής	AreaSearch.tsx, AreaSearch.go	Αναμονή 2 s, επιλογή πραγματικού bbox
A6	Επαληθευμένη διεύθυνση	GetGeoData.go	OSM τύπος/ID και canonical όνομα
A7	Ελεγχόμενη δημοσίευση	Tree.py, training.go, main.go	Έλεγχοι πριν από επιμέρους ενεργοποιήσεις

3.3 Ποιότητα και μη λειτουργικές απαιτήσεις

Η συνέπεια έχει πολλές διαστάσεις. Σημασιολογική συνέπεια σημαίνει ότι το f20 είναι Elevator παντού. Αριθμητική συνέπεια σημαίνει ότι η πρόβλεψη ONNX συμφωνεί μέσα σε ανοχή με τη native πρόβλεψη. Συνέπεια ανάγνωσης σημαίνει ότι μια λεπτομερής προβολή δεν συνδυάζει αυθαίρετα γραμμές από δύο διαφορετικά snapshots της βάσης. Συνέπεια δημοσίευσης σημαίνει ότι το ενεργό μοντέλο και τα συνοδευτικά δεδομένα μπορούν να αντιστοιχιστούν στην ίδια γενιά.

Οι τέσσερις διαστάσεις δεν καλύπτονται από έναν μόνο mutex ή ένα hash. Ο mutex προστατεύει κοινή μνήμη εντός διεργασίας. Το hash ελέγχει ταυτότητα bytes. Η συναλλαγή PostgreSQL προστατεύει συγκεκριμένες εντολές βάσης. Η αναγνωσιμότητα των logs βοηθά στη διάγνωση. Καμία από αυτές τις τεχνικές δεν αποτελεί από μόνη της πλήρη εγγύηση ορθότητας κατανεμημένης δημοσίευσης.

Η απλότητα του κώδικα αποτελεί επίσης απαίτηση, επειδή το έργο πρέπει να μπορεί να συντηρηθεί και να εξηγηθεί. Η χρήση μικρών κοινών helpers για την ανάγνωση λεπτομερειών αποτρέπει διπλή υλοποίηση owner και browse λογικής. Αντίστοιχα, η κεντρική λίστα χαρακτηριστικών μειώνει τον κίνδυνο σιωπηρής απόκλισης. Όμως η απλότητα δεν σημαίνει κατάργηση κρίσιμων ελέγχων: ένας έλεγχος πλήθους χαρακτηριστικών ή δικαιώματος ιδιοκτησίας προστατεύει το νόημα της λειτουργίας.

3.4 Προέλευση, πληθυσμός και περιορισμοί δεδομένων

Το schema_dump.sql είναι σχήμα και όχι πλήρες dataset. Δεν επαρκεί για αναπαραγωγή των αριθμητικών αποτελεσμάτων. Τα manifests των ενεργών μοντέλων αναφέρουν 123.449 γραμμές πωλήσεων και 39.781 γραμμές μισθώσεων. Οι αριθμοί αυτοί είναι οι πληθυσμοί της συγκεκριμένης εκπαιδευτικής δημοσίευσης και όχι εγγύηση για το σημερινό πλήθος της βάσης. Νέες καταχωρίσεις ή διαγραφές σχέσεων μπορούν να αλλάξουν άλλες μετρήσεις χωρίς νέα εκπαίδευση.

Οι εντολές επιλογής για εκπαίδευση απαιτούν έγκυρη θετική ζητούμενη τιμή και τα απαραίτητα χαρακτηριστικά. Η επιλογή αυτή δημιουργεί υποπληθυσμό. Κατοικίες χωρίς asking μπορούν να εκτιμηθούν online αλλά δεν συμμετέχουν στην επιβλεπόμενη εκπαίδευση. Δεν

υπάρχει στο εξεταζόμενο bundle πλήρης τεκμηρίωση αρχικής συλλογής, άδειας επαναχρησιμοποίησης, συχνότητας ενημέρωσης, αποδιπλοποίησης και χρονικής κάλυψης. Αυτά παραμένουν προϋποθέσεις για δημόσια διάθεση και ισχυρή επιστημονική αναπαραγωγή.

Το dataset δεν πρέπει να περιγραφεί ως τυχαίο αντιπροσωπευτικό δείγμα όλων των ελληνικών ακινήτων. Τα χαρακτηριστικά μιας βάσης αγγελιών εξαρτώνται από τους διαθέσιμους παρόχους, την ψηφιακή παρουσία, την περιοχή και τον χρόνο παρατήρησης. Ελλείψεις ή πολλαπλές αγγελίες του ίδιου ακινήτου μπορούν να επηρεάσουν την εκπαίδευση και την επικύρωση. Η παρούσα μελέτη δεν εισάγει αυθαίρετους ισχυρισμούς ότι αυτά τα προβλήματα έχουν λυθεί.

3.5 Μονάδες και σημασία των βασικών μεγεθών

Πίνακας 3.2. Λεξικό οικονομικών και στατιστικών μεγεθών

Μέγεθος	Μονάδα	Ερμηνεία
AskingPricePerSqm	€/m²	Ζητούμενη τιμή ανά μονάδα επιφάνειας
PredictedPricePerSqm	€/m²	Πρόβλεψη μοντέλου για το ίδιο μέγεθος
EstimatedTotalPrice	€	Πρόβλεψη ανά m² επί επιφάνεια
ResidualPercentage	ποσοστιαίες μονάδες	100 × (asking - prediction) / asking
GeoPremium	ποσοστιαίες μονάδες	Μέσο κατάλοιπο Attr γειτόνων
GeoPremiumZScore	χωρίς μονάδα	Τυποποιημένο GeoPremium στην αντίστοιχη αγορά
Elo	μονάδες ευρετικής βαθμολογίας	Σταθμισμένος συνδυασμός z-scores
SHAP contribution	€/m²	Προσημασμένη συμμετοχή στη συγκεκριμένη πρόβλεψη
Importance percentage	%	Κανονικοποιημένο μέτρο απόλυτης συνεισφοράς

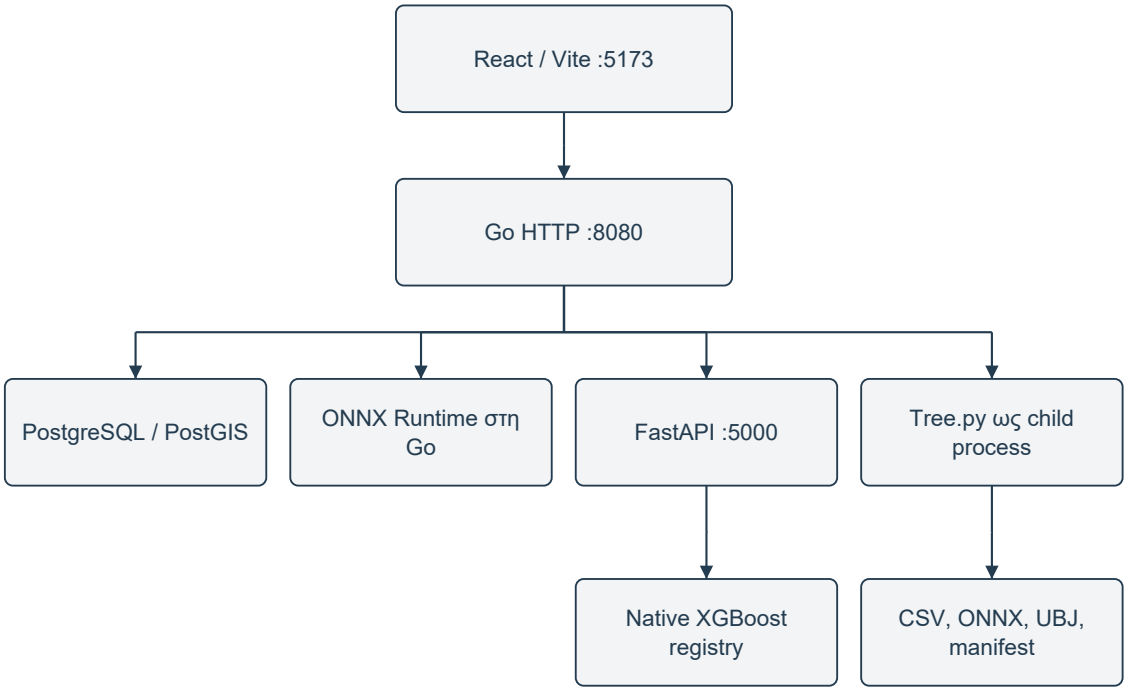
Στις μισθώσεις το ίδιο τεχνικό target unit χρησιμοποιείται για την καταχωρισμένη ζητούμενη μίσθωση ανά τ.μ. Το schema και το manifest δεν αποτελούν από μόνα τους τεκμηρίωση της χρονικής περιόδου της μίσθωσης. Δεν είναι ασφαλές να παρουσιάζεται ένα ποσό ως ετήσιο ή μηνιαίο χωρίς επιβεβαίωση της προέλευσης των καταχωρίσεων. Η διεπαφή διαχωρίζει sell και rent, ώστε να μην αθροίζονται ποσά διαφορετικής οικονομικής φύσης στην ίδια σύγκριση αγοράς.

4. Αρχιτεκτονική, σκελετός και αρχικοποίηση

4.1 Διάσπαση σε υποσυστήματα

Το σύστημα αποτελείται από τέσσερις κύριες εκτελούμενες υπηρεσίες στο περιβάλλον ανάπτυξης: τον Vite server της διεπαφής, τη Go υπηρεσία HTTP, την PostgreSQL/PostGIS και την Python υπηρεσία FastAPI. Ο καθολικός trainer είναι πέμπτη, προσωρινή διεργασία, που ξεκινά από τη Go όταν απαιτείται εκπαίδευση. Δεν πρέπει να συγχέεται με τον ήδη ενεργό Python server. Το ότι και οι δύο γράφονται σε Python δεν συνεπάγεται κοινή μνήμη ή αυτόματη ανταλλαγή αντικειμένων.

Η διεπαφή επικοινωνεί μόνο με τη Go μέσω των routes /register, /User και /analysis. Στην ανάπτυξη οι κλήσεις περνούν από το Vite proxy προς localhost:8080. Η Go εκτελεί SQL, ONNX inference και HTTP κλήσεις προς την Python. Η Python δεν αποτελεί απευθείας browser API για τις κανονικές ενέργειες χρήστη. Αυτή η διάταξη διατηρεί το σημείο αυθεντικοποίησης στη Go, χωρίς όμως να εξαλείφει την ανάγκη προστασίας της εσωτερικής θύρας Python.



Διάγραμμα 4.1. Δομή εκτέλεσης και επικοινωνίας

4.2 Σκελετός αποθετηρίου

```
ptixiaki-bundle/
  Ptixiaki/
    main.go
    Backend/
      Api/                HTTP δεδομένα και λειτουργίες
        UserActions/      portfolio, browse, details, add
        MarketAnalysis/   περιοχή, clustering, Python client
        Elo/              online βαθμολογία
      Auth/               credentials και JWT middleware
      ExternalServices/Geo/
    Database/migrations/
    Worker/
      database/           κανόνες SQL επιλογής
      training/           CSV, encoding, GeoPremium
```

```
    testing/             online feature vector και inference
    Pipelines/           train/predict/persist orchestration
    explainability/      stream, artifacts, client, persistence
    types/               δομές δεδομένων και predictor
    Models/TreeCode/Tree.py
Ptixiaki-ml-microservice/
  main.py
  test_explainability.py
ml_shared/
  explain_artifacts.py
  shap_validation.py
Ptixiaki-frontend/Ptixiaki-Frontend/src/
  components/
  pages/
  App.tsx
schema_dump.sql
```

Ο φάκελος Worker περιέχει κανονικό κώδικα Go που εισάγεται από την κύρια εφαρμογή. Δεν είναι ανεξάρτητος server. Το testing στο όνομα πακέτου Worker/testing είναι ιστορική ονομασία για τη λογική inference και δεν σημαίνει ότι χρησιμοποιείται μόνο από tests. Αντίστοιχα, τα DataCsv και Models/TreeOutputs είναι λειτουργικά artifacts. Η διάκριση πηγαίου κώδικα και παραγόμενων δεδομένων βοηθά στην ασφαλή αλλαγή και στην αναπαραγωγή.

4.3 Η application ως σημείο σύνθεσης

Η δομή application συγκεντρώνει τις εξαρτήσεις με διάρκεια ζωής εφαρμογής. Ο Calculator κρατά το pool βάσης και ένα context. Τα SellTree και RentTree δείχνουν στους ενεργούς predictors. Το AreaCatalog διατηρεί τον κατάλογο διοικητικών περιοχών. Τα δύο mutex έχουν διαφορετική ευθύνη: TrainingMu σειριοποιεί εργασίες εκπαίδευσης, ενώ Mu ελέγχει την πρόσβαση στη γενιά των ενεργών predictors.

Πηγή: Ptixiaki/main.go, γραμμές 36-74. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
36  type application struct {
37      AreaCatalog  mar.AreaCatalog
38      Mu           sync.RWMutex
39      TrainingMu   sync.Mutex
40      Calculator   c.Calculator
41      SellTreeAttr *t.ModelPredictor
42      RentTreeAttr *t.ModelPredictor
43      SellTree     *t.ModelPredictor
44      RentTree     *t.ModelPredictor
45  }
```

```
47  func main() {
```

```
49      app := &application{}
50      err := app.WarmUpApplication()
51      if err != nil {
52          log.Fatalf("Failed to warm up application: %v", err)
53      }
```

```
55      http.HandleFunc("/predict/sell", app.PredictSellHandler)
57      http.HandleFunc("/train", app.TrainTreesHandler)
58      http.HandleFunc("/register/SignUp", app.SignUpHandler)
59      http.HandleFunc("/register/SignIn", app.SignInHandler)
60      http.HandleFunc("/User/AddProperty",
    auth.Authenticate(app.AddPropertyHandler))
61      http.HandleFunc("/User/ViewProperty",
    auth.Authenticate(app.ViewPropertyHandler))
62      http.HandleFunc("/User/BrowseProperties",
    auth.Authenticate(app.BrowsePropertiesHandler))
63      http.HandleFunc("/User/BrowseProperty",
    auth.Authenticate(app.BrowsePropertyHandler))
64      http.HandleFunc("/User/Portfolio", auth.Authenticate(app.PortfolioHandler))
65      http.HandleFunc("/User/DeleteProperty",
```

```
    auth.Authenticate(app.DeletePropertyHandler))
66      http.HandleFunc("/test", app.test)
67      http.HandleFunc("/analysis/autocomplete",
    auth.Authenticate(app.AutocompleteHandler))
68      http.HandleFunc("/analysis/CreateAnalysisForArea",
    auth.Authenticate(app.CreateAnalysisForAreaHandler))
69      err = http.ListenAndServe(":8080", nil)
70      if err != nil {
71          log.Fatal(err)
72      }
74  }
```

Το app := &application{} δημιουργεί αρχικά μια δομή με μηδενικές τιμές. Τα mutex είναι ήδη χρήσιμα με τη μηδενική τους τιμή· δεν απαιτούν ξεχωριστό constructor. Οι δείκτες μοντέλων αρχικά είναι nil. Η WarmUpApplication πρέπει να ολοκληρωθεί πριν ο server αρχίσει να ακούει αιτήματα. Έτσι η κανονική διαδρομή δεν επιτρέπει να φτάσει χρήστης σε έναν εντελώς μη αρχικοποιημένο predictor.

Δεν δημιουργούνται όλες οι δομές του έργου στην εκκίνηση. Οι User, PropertyFeatures και AnalysisPipeline δημιουργούνται ανά αίτημα. Οι TrainingPipeline, parser και propertyStager δημιουργούνται ανά εργασία εκπαίδευσης. Ο διαχωρισμός εμποδίζει την εσφαλμένη εντύπωση ότι κάθε struct αντιστοιχεί σε μόνιμη υπηρεσία. Στη Go μια struct περιγράφει διάταξη δεδομένων· το πού κατασκευάζεται και ποιος κρατά τον δείκτη της καθορίζουν τη διάρκεια ζωής.

Πίνακας 4.1. Κύριες δομές και διάρκεια ζωής

Δομή	Δημιουργία	Τι κατέχει
application	main	Αναφορές κοινών υπηρεσιών και mutex
pgxpool.Pool	WarmUpApplication	Pool συνδέσεων PostgreSQL
Calculator	WarmUpApplication ή τοπική ροή	Pool, context και προετοιμασία inference
ModelPredictor	loadTree	Native session, tensors, buffers, mutex
AreaCatalog	zero value στο application	Προσωρινά αποθηκευμένος κατάλογος ορίων
NominatimClient	global Geocoder	Client, cache και χρονικός περιορισμός
User	authenticated αίτημα	UID χρήστη, όχι ξεχωριστή DB σύνδεση
TrainingPipeline	trainResidenceTrees	Pool και context εκπαίδευσης
AnalysisPipeline	αίτημα Analysis	Pool, context, warnings, URL Python
ExplainModelRegistry	εισαγωγή Python module	Sell/rent native μοντέλα και locks

4.4 Εκκίνηση Go και διαθέσιμα μοντέλα

Η WarmUpApplication ανοίγει την πρόσβαση στη βάση μέσω InitDB1 και αποθηκεύει το pool στον Calculator. Έπειτα καλεί WarmUpTrees. Η τελευταία ορίζει τη διαδρομή της native βιβλιοθήκης onnxengine.dll, αρχικοποιεί το ONNX Runtime και ελέγχει αν υπάρχουν τα δύο σταθερά ONNX αρχεία. Αν υπάρχουν, φορτώνονται. Αν λείπουν, η τρέχουσα υλοποίηση οδηγείται σε εκπαίδευση. Η αυτόματη αυτή διαδρομή είναι σημαντική λειτουργική προειδοποίηση: η εκκίνηση δεν είναι πάντα μόνο read-only φόρτωση.

Πηγή: Ptixiaki/main.go, γραμμές 421-464. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
421  func (a *application) WarmUpApplication() error {
422      con, err := db.InitDB1()
423      if err != nil {
424          return err
425      }
426      Calculator := c.Calculator{
427          Db:  con,
428          Ctx: context.Background(),
429      }
430      a.Calculator = Calculator
431      err = a.WarmUpTrees()
432      if err != nil {
433          return err
434      }
435      return nil
436  }

438  func fileExists(path string) bool {
439      _, err := os.Stat(path)
440      return !errors.Is(err, os.ErrNotExist)
441  }

443  func (a *application) WarmUpTrees() error {

445      onnxruntime_go.SetSharedLibraryPath("onnxengine.dll")

447      err := onnxruntime_go.InitializeEnvironment()
448      if err != nil {
449          return err
450      }

452      flag := false
453      if fileExists(TreePaths[0]) && fileExists(TreePaths[1]) {
454          flag = true
455      }
456      if !flag {
457          err := a.TrainTrees()
458          if err != nil {
459              return fmt.Errorf("failed to train trees: %v", err)
460          }
461          return nil
462      }
463      return a.LoadOrReloadTrees()
464  }
```

Το context.Background δεν συνδέεται με ένα συγκεκριμένο αίτημα. Χρησιμοποιείται ως ρίζα για εργασίες αρχικοποίησης. Αντίθετα, όταν ένας handler λαμβάνει r.Context, οι επιμέρους ενέργειες μπορούν να πληροφορηθούν ότι ο πελάτης ακύρωσε το αίτημα. Η διάκριση δεν είναι θεωρητική: ένας μακρόχρονος SQL υπολογισμός που αγνοεί το context ενδέχεται να συνεχίζει ενώ ο browser έχει αποχωρήσει.

Το πρόγραμμα καταχωρίζει routes μόνο μετά την προθέρμανση και καλεί http.ListenAndServe στη θύρα 8080. Δεν υπάρχει ενιαίο endpoint που να εκκινεί και τις άλλες διεργασίες. Ένα τέτοιο endpoint θα προϋπέθετε ήδη ενεργό server και θα αύξανε τα δικαιώματα που εκτίθενται μέσω HTTP. Η εκκίνηση από ξεχωριστά terminals είναι ο πραγματικός μηχανισμός του τρέχοντος περιβάλλοντος. Ένας μελλοντικός launcher θα ήταν εξωτερικός

διαχειριστής διεργασιών, όχι ιδιότητα του μοντέλου.

4.5 Κατασκευή ModelPredictor και native πόροι

Πηγή: Ptixiaki/Worker/types/TreeType.go, γραμμές 1-16. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
 1  package types

 3  import (
 4      "sync"

 6      "github.com/yalue/onnxruntime_go"
 7  )

 9  type ModelPredictor struct {
10      Mu           sync.Mutex
11      Session      *onnxruntime_go.AdvancedSession
12      InputTensor  *onnxruntime_go.Tensor[float32]
13      OutputTensor *onnxruntime_go.Tensor[float32]
14      InputData    []float32
15      OutputData   []float32
16  }
```

Το ModelPredictor περιέχει τόσο τη συνεδρία εκτέλεσης όσο και επαναχρησιμοποιούμενη μνήμη εισόδου και εξόδου. Το InputData έχει 30 float32 και το OutputData μία τιμή. Η χρήση float32 είναι μέρος του συμβολαίου του μοντέλου. Δεν αρκεί να σταλεί ένα map με σωστά ονόματα: το ONNX session αναμένει πίνακα συγκεκριμένου σχήματος και σωστής σειράς.

Πηγή: Ptixiaki/main.go, γραμμές 466-508. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
466  func loadTree(modelPath string, featureCount int) (*t.ModelPredictor, error) {
467      inputData := make([]float32, featureCount)
468      inputShape := onnxruntime_go.NewShape(1, int64(featureCount))
469      inputTensor, err := onnxruntime_go.NewTensor(inputShape, inputData)
470      if err != nil {
471          return nil, err
472      }

474      outputData := make([]float32, 1)
475      outputShape := onnxruntime_go.NewShape(1, 1)
476      outputTensor, err := onnxruntime_go.NewTensor(outputShape, outputData)
477      if err != nil {
478          inputTensor.Destroy()
479          return nil, err
480      }

482      session, err := onnxruntime_go.NewAdvancedSession(
483          modelPath,
484          []string{"input"},
485          []string{"variable"},
486          []onnxruntime_go.Value{inputTensor},
487          []onnxruntime_go.Value{outputTensor},
488          nil,
489      )
490      if err != nil {
491          inputTensor.Destroy()
492          outputTensor.Destroy()
493          return nil, err
494      }

496      predictor := &t.ModelPredictor{
497          Session:      session,
498          InputTensor:  inputTensor,
499          OutputTensor: outputTensor,
500          InputData:    inputData,
501          OutputData:   outputData,
502      }
503      if _, err := c.PredictPrice(make([]float32, featureCount), predictor); err !=
      nil {
```

```
504          destroyPredictor(predictor)
505          return nil, fmt.Errorf("model validation run failed: %w", err)
506      }
507      return predictor, nil
508  }
```

Η loadTree δημιουργεί πρώτα tensor εισόδου με σχήμα (1,30), έπειτα tensor εξόδου (1,1) και μετά AdvancedSession με ονόματα input και variable. Αν ένα επόμενο βήμα αποτύχει, καταστρέφονται οι πόροι που είχαν ήδη δημιουργηθεί. Αυτό χρειάζεται επειδή οι native πόροι δεν είναι απλώς συνηθισμένα Go slices που αρκεί να ξεχαστούν. Το smoke test με μηδενικό διάνυσμα εξετάζει ότι το μοντέλο μπορεί να εκτελεστεί και να επιστρέψει έγκυρο αποτέλεσμα, όχι ότι προβλέπει σωστά πραγματικά ακίνητα.

Η εφαρμογή φορτώνει sell και rent πριν δημοσιεύσει το ζεύγος. Αν η φόρτωση rent αποτύχει, ο νέος sell predictor καταστρέφεται. Η συμπεριφορά αποτρέπει μια μισή αντικατάσταση στο επίπεδο των Go pointers. Δεν συνεπάγεται ταυτόχρονη συναλλαγή με τα αρχεία και τη βάση, τα οποία έχουν διαφορετικό κύκλο ενημέρωσης. Η έννοια «όλα ή τίποτα» πρέπει πάντοτε να συνοδεύεται από το επίπεδο στο οποίο ισχύει.

4.6 Mutex, παράλληλα αιτήματα και αλλαγή γενιάς

Ένας RWMutex επιτρέπει πολλούς αναγνώστες ταυτόχρονα, αλλά αποκλείει τη λειτουργία εγγραφής όσο υπάρχουν ενεργοί αναγνώστες (Go Authors, χ.χ.-δ). Στο AddPropertyHandler η RLock κρατά τη συγκεκριμένη γενιά μοντέλου ζωντανή μέχρι να ολοκληρωθεί η ροή. Η publishTrees αποκτά την αποκλειστική Lock προτού αλλάξει pointers και καταστρέψει τα παλιά sessions. Χωρίς αυτόν τον συντονισμό ένα αίτημα θα μπορούσε να χρησιμοποιεί native μνήμη που έχει ήδη ελευθερωθεί.

Πηγή: Ptixiaki/main.go, γραμμές 235-248. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
235  func (a *application) AddPropertyHandler(w http.ResponseWriter, r *http.Request)
     {
236      u.ServeAddProperty(w, r, func(ctx context.Context, userUID string, prop
     p.PropertyFeatures, transaction string) (string, error) {
237          a.Mu.RLock()
238          defer a.Mu.RUnlock()
239          var attrTree, predTree *t.ModelPredictor
240          if transaction == "sell" {
241              attrTree, predTree = a.SellTreeAttr, a.SellTree
242          } else {
243              attrTree, predTree = a.RentTreeAttr, a.RentTree
244          }
245          user := u.User{Uid: userUID}
246          return user.AddProperty(a.Calculator.Db, ctx, prop, attrTree, predTree,
     transaction)
247      })
248  }
```

Η κοινή κλειδαριά δεν παγώνει ολόκληρο τον HTTP server. Ανεξάρτητα endpoints μπορούν να εκτελούνται παράλληλα, ενώ περισσότερες προσθήκες μπορούν να κρατούν RLock. Υπάρχει όμως και δεύτερο mutex μέσα σε κάθε ModelPredictor: προστατεύει το γράψιμο στο κοινό InputData, την εκτέλεση και την ανάγνωση του OutputData. Στον ίδιο predictor οι ίδιες οι προβλέψεις σειριοποιούνται. Sell και rent έχουν διαφορετικά αντικείμενα και διαφορετικά εσωτερικά locks.

Αυτή η διάταξη είναι απλή και κατανοητή για το μέγεθος του έργου. Σε υψηλό φορτίο θα μπορούσαν να εξεταστούν ανεξάρτητα tensors ανά αίτημα ή pool sessions, μετά από μέτρηση. Η αλλαγή δεν πρέπει να γίνει μόνο με αφαίρεση του mutex, επειδή δύο goroutines θα μπορούσαν να αναμείξουν χαρακτηριστικά διαφορετικών ακινήτων. Προτεραιότητα είναι η ορθή ιδιοκτησία της μνήμης και κατόπιν η βελτιστοποίηση της παραλληλίας.

4.7 Εκκίνηση Python και ml_shared

Ο Python server κατασκευάζει ExplainModelRegistry κατά την εισαγωγή του module. Στο lifespan επιχειρεί να φορτώσει sell και rent. Αν λείπει έγκυρο ζεύγος UBJ/manifest, καταγράφει προειδοποίηση και η υπηρεσία μπορεί να συνεχίσει, ενώ η εξήγηση παραμένει μη διαθέσιμη. Η κατάσταση αυτή διαφέρει από την εκκίνηση Go, όπου η αποτυχία προθέρμανσης οδηγεί σε τερματισμό.

Πηγή: Ptixiaki-ml-microservice/main.py, γραμμές 273-291. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
275  explain_registry = ExplainModelRegistry()
```

```
278  @asynccontextmanager
279  async def lifespan(_: FastAPI):
280      try:
281          loaded = explain_registry.reload(("sell", "rent"))
282          logger.info("Loaded explainability models at startup: %s", loaded)
283      except (ArtifactValidationError, OSError, ValueError, xgb.core.XGBoostError)
     as exc:
286          logger.warning("Explainability models were not loaded at startup: %s",
     exc)
287      yield
```

```
290  app = FastAPI(lifespan=lifespan)
```

Το ml_shared είναι πακέτο κοινών Python συναρτήσεων. Περιέχει τη λίστα χαρακτηριστικών, τη δημιουργία/επικύρωση manifest, τη φόρτωση native μοντέλου, τους ατομικούς χειρισμούς αρχείων και την αριθμητική ανοχή SHAP. Δεν είναι κοινόχρηστη μνήμη μεταξύ διεργασιών ούτε φάκελος Go cache. Το Tree.py και το FastAPI main.py εισάγουν τις ίδιες συναρτήσεις ώστε να συμφωνούν στο συμβόλαιο, αλλά καθεμία διεργασία διαθέτει τα δικά της αντικείμενα.

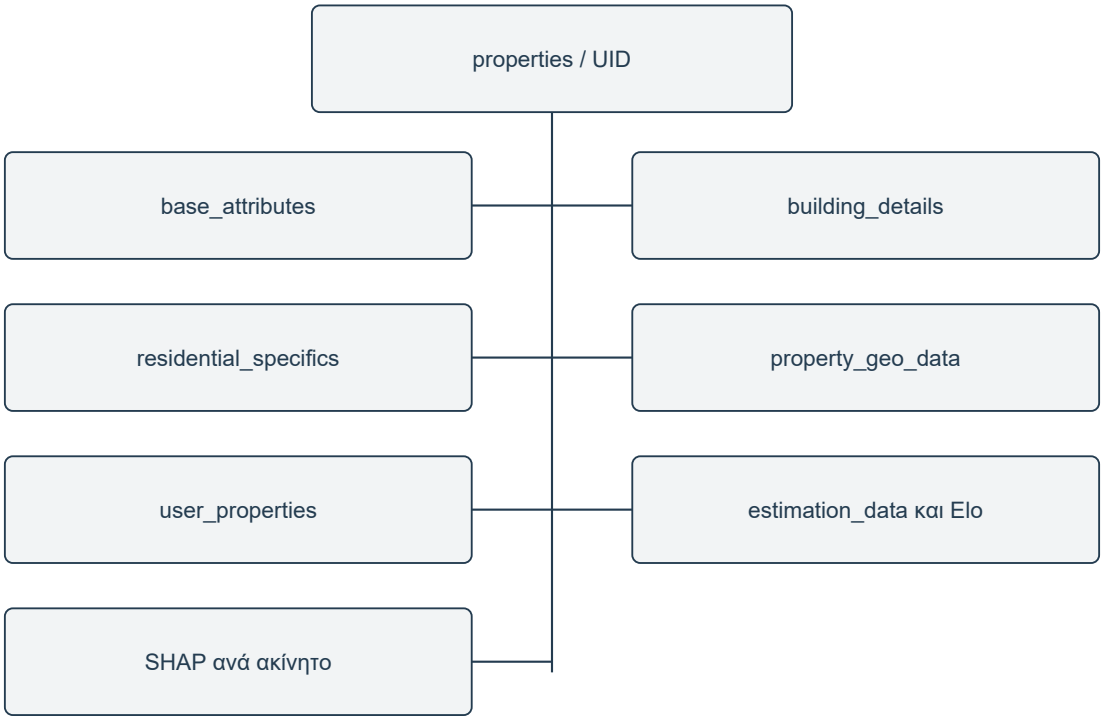
Το .gocache και το .cache είναι βοηθητικοί χώροι εργαλείων, όχι πηγές επιχειρησιακής αλήθειας. Η διαγραφή ενός build cache μπορεί να επιβραδύνει την επόμενη μεταγλώττιση· η απώλεια ενός UBJ ή manifest μπορεί να καταστήσει αδύνατη τη φόρτωση εξηγήσεων. Για τον λόγο αυτό η τεκμηρίωση διαχωρίζει τις διαδρομές κατασκευής από τις διαδρομές των ενεργών artifacts.

5. Σχεδιασμός δεδομένων και συμβόλαιο χαρακτηριστικών

5.1 Οντότητες και διαχωρισμός ευθυνών

Η PostgreSQL αποθηκεύει τόσο πρωτογενή στοιχεία αγγελιών όσο και παράγωγα αποτελέσματα. Η properties είναι η κεντρική οντότητα αναφοράς. Τα base_attributes, building_details και residential_specifics κατανέμουν τα γνωρίσματα σε θεματικές ομάδες. Το property_geo_data συνδέει το ακίνητο με διεύθυνση, γεωγραφικές συντεταγμένες και γεωμετρία. Το user_properties εκφράζει τη σχέση ενός λογαριασμού με μια καταχώριση. Η σχέση αυτή δεν πρέπει να συγχέεται με αποδεδειγμένη νομική κυριότητα.

Τα estimation_data και properties_elo περιέχουν αποτελέσματα υπολογισμών. Η λογική αυτή επιτρέπει στις προβολές να διαβάζουν έτοιμες τιμές, αντί να εκτελούν ακριβή διαδικασία πρόβλεψης σε κάθε κλικ. Ταυτόχρονα δημιουργεί υποχρέωση ενημέρωσης: αλλαγή μοντέλου δεν ανανεώνει από μόνη της όλες τις αποθηκευμένες εκτιμήσεις. Η εκπαίδευση και δημοσίευση πρέπει να ενημερώνουν τις αντίστοιχες παράγωγες εγγραφές.



Διάγραμμα 5.1. Λογικός πυρήνας δεδομένων, όχι πλήρες ER όλων των πινάκων

Το διάγραμμα παρουσιάζει λογικές συνδέσεις και όχι ισχυρισμό ότι κάθε βέλος υλοποιείται ως foreign key. Στο dump υπάρχουν σχέσεις που ελέγχονται κυρίως από την εφαρμογή. Η διάκριση έχει πρακτικό αποτέλεσμα: μια λανθασμένη εγγραφή μπορεί να γίνει δεκτή από τη βάση όταν απουσιάζει κατάλληλος περιορισμός. Η τεκμηρίωση μιας σχέσης οντοτήτων δεν υποκαθιστά έλεγχο DDL.

5.2 Σχήμα μεγαλύτερο από το ενεργό προϊόν

Το schema_dump.sql περιλαμβάνει επιπλέον οικογένειες πινάκων για συνδρομές, χρεώσεις, οργανισμούς, αναφορές, ιστορικό αποτιμήσεων και διαχειριστικές ενέργειες. Δεν προκύπτει ότι όλα αυτά εκτελούνται από την τρέχουσα εφαρμογή. Για παράδειγμα, η ύπαρξη billing_preferences δεν αρκεί για να υποστηριχθεί ότι λειτουργούν πληρωμές. Η μελέτη περιορίζει τους ισχυρισμούς της στις διαδρομές που συνδέονται πραγματικά με handlers ή pipelines.

Υπάρχουν επίσης δύο σχεδιασμοί ταυτότητας: ο παλαιότερος user_credentials, που χρησιμοποιεί το τρέχον authentication, και οι users/auth_providers του ευρύτερου schema. Η συνύπαρξη απαιτεί προσοχή πριν από επέκταση της εφαρμογής. Ένα ξένο κλειδί προς users δεν σημαίνει ότι αντιστοιχεί αυτόματα στο UID που εκδόθηκε από το σημερινό SignIn. Η ενοποίηση ταυτότητας είναι μελλοντική εργασία, όχι ήδη ολοκληρωμένη μετάβαση.

Το dump προϋποθέτει PostgreSQL extensions και ρόλους. Περιλαμβάνει PostGIS, pg_trgm, unaccent, pg_partman και pg_cron, ενώ αναφέρεται και σε συγκεκριμένους owners/ρόλους. Η επαναφορά σε άδεια εγκατάσταση δεν είναι απλή αντιγραφή αρχείου. Η επιτυχής εκτέλεση των unit tests δεν επαληθεύει αυτή τη διαδικασία. Απαιτείται ανεξάρτητος έλεγχος restore σε απομονωμένη βάση.

5.3 Αποθήκευση εξηγήσεων

Η migration 002 δημιουργεί έξι λογικούς πίνακες: properties_analysis_sell/rent, feature_importance_sell/rent και features_interactions_sell/rent. Στους πίνακες ακινήτων κάθε μία από τις τριάντα στήλες impact_per_sqm είναι συνεισφορά τιμής, όχι αρχική τιμή γνωρίσματος. Για παράδειγμα, το square_meters_impact_per_sqm δεν περιέχει εμβαδόν. Περιέχει το ποσό της πρόβλεψης που αποδόθηκε στο χαρακτηριστικό SquareMeters.

Πηγή: Ptixiaki/Database/migrations/002_create_explainability_persistence.sql, γραμμές 4-46. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
 4  CREATE TABLE IF NOT EXISTS public.properties_analysis_sell (
 5      property_uid character varying(255) NOT NULL,
 6      subtype_impact_per_sqm double precision NOT NULL,
 7      admin_name_impact_per_sqm double precision NOT NULL,
 8      square_meters_impact_per_sqm double precision NOT NULL,
 9      age_impact_per_sqm double precision NOT NULL,
10      renov_age_impact_per_sqm double precision NOT NULL,
11      no_view_impact_per_sqm double precision NOT NULL,
12      view_forest_impact_per_sqm double precision NOT NULL,
13      view_mountain_impact_per_sqm double precision NOT NULL,
14      view_sea_impact_per_sqm double precision NOT NULL,
15      view_unlimited_impact_per_sqm double precision NOT NULL,
16      no_facade_impact_per_sqm double precision NOT NULL,
17      facade_corner_impact_per_sqm double precision NOT NULL,
18      facade_frontage_impact_per_sqm double precision NOT NULL,
19      facade_interior_impact_per_sqm double precision NOT NULL,
20      facade_through_impact_per_sqm double precision NOT NULL,
21      heat_type_impact_per_sqm double precision NOT NULL,
22      heat_from_impact_per_sqm double precision NOT NULL,
23      energy_performance_impact_per_sqm double precision NOT NULL,
24      wc_impact_per_sqm double precision NOT NULL,
25      ac_impact_per_sqm double precision NOT NULL,
26      elevator_impact_per_sqm double precision NOT NULL,
27      parking_impact_per_sqm double precision NOT NULL,
28      storage_room_impact_per_sqm double precision NOT NULL,
29      aluminum_frames_impact_per_sqm double precision NOT NULL,
30      bedrooms_impact_per_sqm double precision NOT NULL,
31      pool_impact_per_sqm double precision NOT NULL,
32      garden_impact_per_sqm double precision NOT NULL,
33      floor_level_impact_per_sqm double precision NOT NULL,
34      floor_count_impact_per_sqm double precision NOT NULL,
35      geo_premium_impact_per_sqm double precision NOT NULL,
```

```
36      baseline_price_per_sqm double precision NOT NULL,
37      explained_prediction_per_sqm double precision NOT NULL,
38      additivity_error double precision NOT NULL,
39      model_sha256 character varying(64) NOT NULL,
40      explained_at timestamp with time zone DEFAULT now() NOT NULL,
41      CONSTRAINT properties_analysis_sell_pkey PRIMARY KEY (property_uid),
42      CONSTRAINT properties_analysis_sell_property_uid_fkey
43          FOREIGN KEY (property_uid)
44          REFERENCES public.properties(uid)
45          ON DELETE CASCADE
46  );
```

Οι στήλες baseline_price_per_sqm και explained_prediction_per_sqm επιτρέπουν τον αριθμητικό έλεγχο της αποσύνθεσης. Το additivity_error καταγράφει την απόκλιση του αθροίσματος από τη native πρόβλεψη. Το model_sha256 συνδέει το αποτέλεσμα με το συγκεκριμένο αρχείο UBJ. Το explained_at δίνει χρόνο αναφοράς. Χωρίς αυτά τα μεταδεδομένα, μια λίστα από συνεισφορές θα μπορούσε να φαίνεται πλήρης αλλά να ανήκει σε διαφορετική γενιά από την εμφανιζόμενη σημασία χαρακτηριστικών.

Η χωριστή αποθήκευση sell και rent ακολουθεί τον χωρισμό των μοντέλων. Δεν θεωρείται ότι μια συνεισφορά 20 €/τ.μ. σημαίνει το ίδιο σε δύο διαφορετικές αγορές ή ότι οι αντίστοιχες baseline είναι συγκρίσιμες χωρίς το transaction. Η τρέχουσα προσέγγιση latest-only απλοποιεί τα queries, αλλά χάνει ιστορική δυνατότητα ανακατασκευής παλαιών εξηγήσεων όταν αντικαθίσταται το μοντέλο.

5.4 Τι αποθηκεύει τελικά το global_stats

Το global_stats εξακολουθεί να περιέχει mean και std για τη βαθμολόγηση. Οι σημασιολογικές εγγραφές Price_stats_sell και Price_stats_rent αναφέρονται στα τελικά ποσοστιαία κατάλοιπα, ενώ Geo_stats_sell και Geo_stats_rent στα GeoPremium. Η προσθήκη baseline, training_r2, validation_r2, model_sha256 και model_trained_at δεν καταργεί τις παλιές στατιστικές. Επεκτείνει τις εγγραφές Price_stats με μεταδεδομένα του τελευταίου μοντέλου.

Πίνακας 5.1. Σημασιολογία εγγραφών global_stats

stat_name	mean / std	Πρόσθετα μεταδεδομένα μοντέλου
Price_stats_sell	Τελικά κατάλοιπα πώλησης	Baseline, δύο R^2, hash, χρόνος
Price_stats_rent	Τελικά κατάλοιπα μίσθωσης	Baseline, δύο R^2, hash, χρόνος
Geo_stats_sell	GeoPremium πωλήσεων	Δεν χρησιμοποιούνται ως metadata μοντέλου
Geo_stats_rent	GeoPremium μισθώσεων	Δεν χρησιμοποιούνται ως metadata μοντέλου

Η baseline παραμένει ένα scalar ανά μοντέλο. Δεν απαιτείται πίνακας «baseline ανά χαρακτηριστικό». Οι πληροφορίες κάθε χαρακτηριστικού αποθηκεύονται στους πίνακες importance και contributions. Το UPSERT των μεταδεδομένων ενημερώνει μόνο τις νέες στήλες σε σύγκρουση stat_name, ώστε να μην αντικαθιστά κατά λάθος τα πραγματικά mean/std με τις μηδενικές τιμές fallback της εισαγωγής.

Η εντολή SELECT ... FROM global_stats LIMIT 0 δεν διαβάζει μηδενικές στατιστικές ούτε αδειάζει τον πίνακα. Ζητά από την PostgreSQL να ελέγξει ότι ο πίνακας και οι συγκεκριμένες στήλες υπάρχουν και είναι προσβάσιμες, χωρίς να επιστρέψει εγγραφές. Είναι preflight έλεγχος συμβολαίου. Αν η migration δεν έχει εφαρμοστεί, η διαδικασία αποτυγχάνει πριν δαπανηθεί χρόνος στην ακριβή εκπαίδευση και εξήγηση.

Πηγή: Ptixiaki/Worker/explainability/training.go, γραμμές 184-222. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
184  func preflightPersistenceSchema(ctx context.Context, tx pgx.Tx, tables Tables)
     error {
185      impactColumns := PropertyImpactColumns()
186      quotedImpactColumns := make([]string, len(impactColumns))
187      for i, column := range impactColumns {
188          quotedImpactColumns[i] = pgx.Identifier{column}.Sanitize()
189      }
190      checks := []string{
191          fmt.Sprintf(
192              `SELECT property_uid, %s, baseline_price_per_sqm,
193                  explained_prediction_per_sqm, additivity_error, model_sha256,
     explained_at
194               FROM %s LIMIT 0`,
195              strings.Join(quotedImpactColumns, ", "),
196              pgx.Identifier{tables.PropertyAnalysis}.Sanitize(),
197          ),
198          fmt.Sprintf(
199              `SELECT feature_name, mean_abs_shap_per_sqm,
     mean_signed_shap_per_sqm,
200                  importance_percentage, importance_rank, calculated_at,
     model_sha256
201               FROM %s LIMIT 0`,
202              pgx.Identifier{tables.FeatureImportance}.Sanitize(),
203          ),
204          fmt.Sprintf(
205              `SELECT feature_1, feature_2, mean_signed_interaction_per_sqm,
206                  mean_abs_interaction_per_sqm, importance_rank, calculated_at,
     model_sha256
207               FROM %s LIMIT 0`,
208              pgx.Identifier{tables.FeatureInteractions}.Sanitize(),
209          ),
210          `SELECT stat_name, mean, std, baseline_price_per_sqm, training_r2,
211              validation_r2, model_sha256, model_trained_at
212           FROM global_stats LIMIT 0`,
213      }
214      for _, query := range checks {
215          rows, err := tx.Query(ctx, query)
216          if err != nil {
217              return fmt.Errorf("explainability persistence schema preflight failed
      (apply migration 002): %w", err)
218          }
219          rows.Close()
220      }
221      return nil
222  }
```

5.5 Είσοδος χρήστη και τελικό διάνυσμα

Η PropertyFeatures είναι η δομή μεταφοράς εισόδου και αποθηκευμένων χαρακτηριστικών. Δεν είναι το ίδιο αντικείμενο με το τελικό tensor. Περιέχει στοιχεία που χρειάζονται για τη δημιουργία ή προβολή καταχώρισης, όπως διεύθυνση και provider ID, τα οποία δεν εισάγονται αυτούσια στο XGBoost. Αντίστροφα, τα Age, RenovAge και FloorCount κατασκευάζονται από πιο φυσικές εισόδους, όπως έτη και λίστα ορόφων.

Πηγή: Ptixiaki/Backend/Api/Structs.go, γραμμές 3-45. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
  3  type PropertyFeatures struct {
  4      UID string

  6      SubType   string
  7      AdminName string

  9      SquareMeters      int
```

```
    AskingPricePerSqm *float64 `json:"AskingPricePerSqm,omitempty"`
    ConstructionYear  int
    RenovYear         int
    Distance          int
    IsRenovated       int
    ZoneValue         int
    AddressOSMType    string `json:"AddressOSMType,omitempty"`
    AddressOSMID      int64  `json:"AddressOSMID,omitempty"`
    Address           string
    GeoLat            float64
    GeoLon            float64
    NoView            int
    ViewForest        int
    ViewMountain      int
    ViewSea           int
    ViewUnlimited     int
    NoFacade          int
    FacadeCorner      int
    FacadeFrontage    int
    FacadeInterior    int
    FacadeThrough     int
    HeatType          string
    HeatFrom          string
    EnergyPerformance string
    WC                int
    AC                int
    Elevator          int
    Parking           int
    StorageRoom       int
    AluminumFrames    int
    Bedrooms          int
    Pool              int
    Garden            int
    FloorLevel        []int
}
```

Στο τελικό Phase2 διάνυσμα υπάρχουν ακριβώς τριάντα αριθμητικές θέσεις. Οι κατηγορίες γίνονται κωδικοί μέσω αποθηκευμένου mapping. Οι δυαδικές ενδείξεις διατηρούνται ως αριθμοί. Το GeoPremium προστίθεται τελευταίο. Η asking χρησιμοποιείται ως ετικέτα στην εκπαίδευση και για κατάλοιπα, αλλά όχι ως feature πρόβλεψης. Αν εισαγόταν ως είσοδος, το μοντέλο θα μάθαινε να αντιγράφει την τιμή που υποτίθεται ότι εκτιμά.

Πίνακας 5.2. Ακριβής σειρά χαρακτηριστικών Phase2 από το manifest

Θέση	Όνομα χαρακτηριστικού	Encoded όνομα
0	SubType	f0
1	AdminName	f1
2	SquareMeters	f2
3	Age	f3
4	RenovAge	f4
5	NoView	f5
6	ViewForest	f6
7	ViewMountain	f7
8	ViewSea	f8
9	ViewUnlimited	f9
10	NoFacade	f10
11	FacadeCorner	f11
12	FacadeFrontage	f12
13	FacadeInterior	f13
14	FacadeThrough	f14
15	HeatType	f15
16	HeatFrom	f16
17	EnergyPerformance	f17
18	WC	f18
19	AC	f19
20	Elevator	f20
21	Parking	f21
22	StorageRoom	f22
23	AluminumFrames	f23
24	Bedrooms	f24
25	Pool	f25
26	Garden	f26
27	FloorLevel	f27
28	FloorCount	f28
29	GeoPremium	f29

Η χρήση f0 έως f29 διευκολύνει το συμβόλαιο εξαγωγής, αλλά αφαιρεί τα φυσικά ονόματα από την άμεση όψη του μοντέλου. Το manifest αποκαθιστά τη σύνδεση original_feature_names και model_feature_names. Η Go ελέγχει τη λίστα που παράγει το BuildFeatureVector. Η Python

ελέγχει την ίδια λίστα πριν εκπαίδευση και πριν native φόρτωση. Αυτό είναι ισχυρότερο από έναν απλό έλεγχο ότι το μήκος είναι 30.

5.6 Κατηγορικές μεταβλητές και χρονικές συμβάσεις

Στην καθολική εκπαίδευση το pandas category παράγει λεξικό κατηγορίας προς ακέραιο. Το λεξικό γράφεται ως CSV-compatible mapping, ώστε μια κατηγορία με κόμμα να μη διασπάται λανθασμένα. Στο inference η Go δεν επινοεί νέα αρίθμηση με βάση τη σειρά εμφάνισης. Διαβάζει το mapping της Phase2 που αντιστοιχεί στο transaction. Η επιλογή mapping άλλης φάσης θα μπορούσε να μετατρέψει μια ορθή λεκτική τιμή σε λάθος αριθμό.

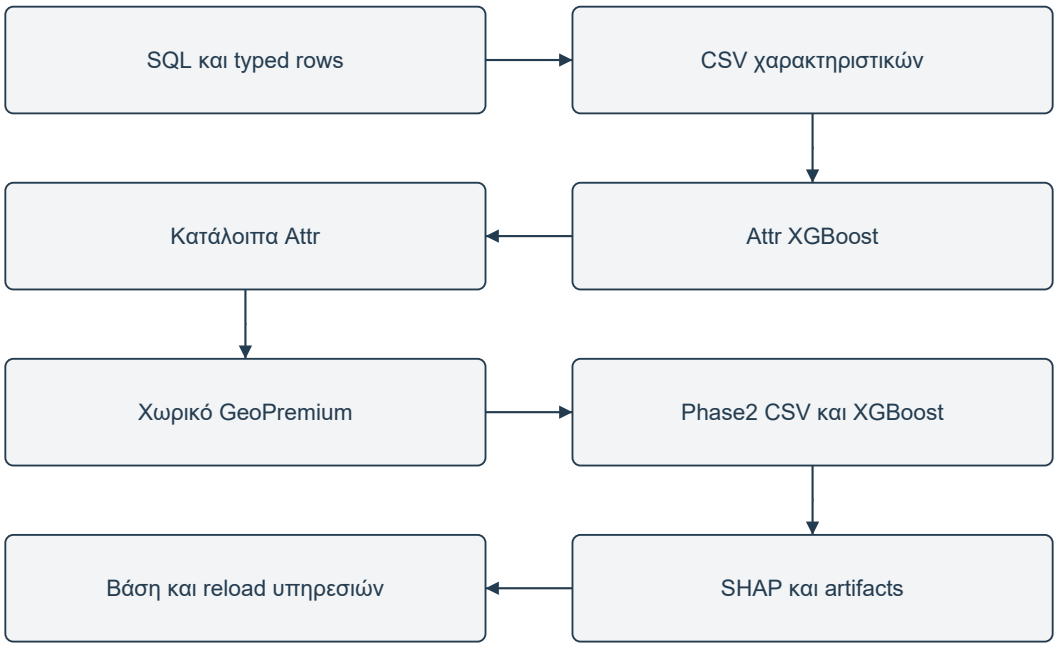
Η αριθμητική κωδικοποίηση δεν είναι one-hot encoding και δεν είναι η native categorical εκπαίδευση της Analysis. Οι ακέραιοι κωδικοί αντιμετωπίζονται ως float32 στο καθολικό μοντέλο, οπότε οι συγκρίσεις δέντρων επιβάλλουν τεχνητή σειρά. Αυτό είναι χαρακτηριστικό της συγκεκριμένης υλοποίησης και πιθανός στόχος μελλοντικής βελτίωσης. Δεν αλλάζει σιωπηρά στην τεκμηρίωση, επειδή αλλαγή encoding θα απαιτούσε νέο training και νέο συμβόλαιο.

Οι μετασχηματισμοί ηλικίας χρησιμοποιούν σημεία με σταθερή χρονική αναφορά 2026. Η σταθερά επιτρέπει συμφωνία με το αποθηκευμένο μοντέλο αλλά προκαλεί γήρανση του συμβολαίου όσο περνά ο χρόνος. Η προτεινόμενη εξέλιξη είναι ρητό reference_year στο manifest και κοινός μετασχηματισμός. Δεν είναι ορθό να αλλάζει μόνο η online ηλικία με το σημερινό έτος ενώ το μοντέλο και η βάση εξηγήσεων παραμένουν στην παλιά αναφορά.

6. Η καθολική αλυσίδα εκπαίδευσης

6.1 Συντονισμός στη Go

Η TrainJob οργανώνει μια αλληλουχία με εξαρτήσεις. Πρώτα εξάγονται δεδομένα σε CSV. Έπειτα εκτελείται το μοντέλο Attr. Από τα κατάλοιπά του δημιουργείται GeoPremium, το οποίο συγχωνεύεται σε δεύτερο CSV. Στη συνέχεια εκπαιδεύεται η Phase2. Για τις κατοικίες sell και rent ενεργοποιείται το streaming πρωτόκολλο SHAP και η ελεγχόμενη αποθήκευση. Η σειρά δεν μπορεί να αντιστραφεί, επειδή η δεύτερη φάση χρειάζεται έξοδο της πρώτης.



Διάγραμμα 6.1. Ροή εκπαίδευσης δύο φάσεων

Πηγή: Ptixiaki/Worker/Pipelines/TrainingPipeline.go, γραμμές 16-68. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
16  type TrainingPipeline struct {
17      Db  *pgxpool.Pool
18      Ctx context.Context
19  }
```

```
21  func InitTrainingPipeline(db *pgxpool.Pool, ctx context.Context) TrainingPipeline
     {
22      return TrainingPipeline{
23          Db:  db,
24          Ctx: ctx,
25      }
26  }
27  func (T *TrainingPipeline) TrainJob(ItemType string, SubType string, Transaction
    string) error {
```

```
30      err := tr.GenerateTrainingData(T.Ctx, T.Db, ItemType, SubType, Transaction)
31      if err != nil {
32          return err
33      }
```

```
36      Creator := tr.AttrTreeConfigs{}
37      Creator.Initialize(ItemType, SubType)
38      Phase1Params := Creator.BuildPythonArgs(Transaction, ItemType, SubType,
    "Attr")
39      scriptPath := "Worker/Models/TreeCode/Tree.py"
```

```
41      err = RunPythonTree(T.Ctx, scriptPath, Phase1Params)
42      if err != nil {
43          return err
44      }
```

```
47      InputJsonPath := BuildOutputPath(Creator.BasePathOutputs, ItemType, SubType,
    Transaction, "JsonOutputs", "Attr")
48      GeoPremOutputDir := filepath.Join(Creator.BasePathOutputs, "GeoPremiums")
```

```
50      err = tr.CalcGeoPremium(T.Ctx, T.Db, ItemType, SubType, Transaction,
    InputJsonPath, GeoPremOutputDir)
51      if err != nil {
52          return err
53      }
```

```
56      OriginalCsvPath := BuildCsvPath(Creator.BasePathCsvs, ItemType, SubType,
    Transaction)
57      GeoJsonFileName := fmt.Sprintf("GeoPrem_%s_%s_%s.json", ItemType, SubType,
    Transaction)
58      GeoJsonPath := filepath.Join(GeoPremOutputDir, GeoJsonFileName)
59      NewCsvPath := BuildCsvPath(Creator.BasePathCsvs, ItemType, SubType,
    Transaction+"_Phase2")
```

```
61      err = tr.MergeGeoPremiumToCSV(OriginalCsvPath, GeoJsonPath, NewCsvPath)
62      if err != nil {
63          return err
64      }
```

```
68      Phase2Params := Creator.BuildPythonArgs(Transaction, ItemType, SubType,
    "Phase2")
```

Η TrainingPipeline περιέχει pool και context, όχι τα εκπαιδευμένα δέντρα. Τα δέντρα υπάρχουν προσωρινά μέσα στην Python διεργασία και στη συνέχεια αποθηκεύονται. Η Go δεν λαμβάνει Python object ως επιστρεφόμενη τιμή της συνάρτησης exec. Λαμβάνει exit status και streams bytes. Τα αρχεία και το πρωτόκολλο stdout είναι οι πραγματικές γέφυρες μεταξύ των δύο περιβαλλόντων.

Η εκπαίδευση sell και rent εκτελείται διαδοχικά από trainResidenceTrees. Το TrainingMu επιτρέπει μόνο μία τέτοια εργασία κάθε φορά μέσα στη διεργασία. Η επιλογή αποτρέπει ταυτόχρονη αντικατάσταση των ίδιων paths από δύο αιτήματα. Δεν είναι κατανεμημένο lock: αν ξεκινήσουν δύο ανεξάρτητες Go διεργασίες στον ίδιο φάκελο, δεν μοιράζονται την ίδια κλειδαριά.

6.2 Προετοιμασία δεδομένων στο Tree.py

Το Tree.py χρησιμοποιεί argparse για να λάβει paths, target, dropped columns, κατηγορίες και υπερπαραμέτρους. Το if __name__ == '__main__' εκκινεί τη main μόνο όταν το αρχείο εκτελείται ως πρόγραμμα. Οι ίδιες συναρτήσεις μπορούν επομένως να εισαχθούν από tests χωρίς να ξεκινήσει ολόκληρη η παραγωγική εκπαίδευση. Η αναζήτηση του bundle από το __file__ επιτρέπει την εισαγωγή ml_shared χωρίς εξάρτηση από το working directory της Go.

Το pandas.read_csv παράγει DataFrame. Η validate_target απαιτεί αριθμητικές και πεπερασμένες τιμές και αποκλείει μηδενικό παρονομαστή καταλοίπου. Για τα explainable Phase2 μοντέλα ελέγχονται και μοναδικά, μη κενά UID. Αν δύο γραμμές είχαν ίδιο UID, η βάση δεν θα μπορούσε να αποθηκεύσει δύο διαφορετικές εξηγήσεις ως την «τελευταία» εξήγηση του ίδιου ακινήτου.

Πηγή: Ptixiaki/Worker/Models/TreeCode/Tree.py, γραμμές 230-292. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
230  def prepare_features(
231      df: pd.DataFrame,
232      args: argparse.Namespace,
233  ) -> tuple[pd.DataFrame, list[str], dict[str, dict[str, int]]]:
234      dropped = [column.strip() for column in args.columns_dropped.split(",") if
     column.strip()]
235      present_dropped = [column for column in dropped if column in df.columns]
236      features = df.drop(columns=present_dropped + [args.target_col]).copy()
```

```
238      if features.shape[1] == 0:
```

```
239         raise ValueError("training matrix has no feature columns")
240     if not features.columns.is_unique:
241         raise ValueError("training feature columns must be unique")
```

```
243     original_feature_names = [str(column) for column in features.columns]
244     if is_explainable_phase2(args):
245         if original_feature_names != EXPECTED_RESIDENCE_PROPERTY_PHASE2_FEATURES:
246             raise ValueError(
247                 "Phase2 residence-properties feature order mismatch; "
248                 f"expected {EXPECTED_RESIDENCE_PROPERTY_PHASE2_FEATURES}, got
    {original_feature_names}"
249             )
```

```
251     configured_categoricals = [
252         column.strip() for column in args.categorical_cols.split(",") if
    column.strip()
253     ]
254     actual_categoricals = [
255         column for column in configured_categoricals if column in
    features.columns
256     ]
257     categorical_mappings: dict[str, dict[str, int]] = {}
```

```
259     for column in actual_categoricals:
260         features[column] = features[column].astype("category")
261         categories = [str(category) for category in
    features[column].cat.categories]
262         categorical_mappings[column] = {
263             category: index for index, category in enumerate(categories)
264         }
265         features[column] = features[column].cat.codes.astype("float32")
```

```
267     for column in features.columns:
268         if column not in actual_categoricals:
269             try:
270                 features[column] = pd.to_numeric(
271                     features[column], errors="raise"
272                 ).astype("float32")
273             except (TypeError, ValueError) as exc:
274                 raise ValueError(f"feature column {column!r} must be numeric")
    from exc
```

```
276     values = features.to_numpy(dtype=np.float32, copy=False)
277     if not np.isfinite(values).all():
278         bad_columns = [
279             str(column)
280             for column in features.columns
281             if not np.isfinite(
282                 features[column].to_numpy(dtype=np.float32, copy=False)
283             ).all()
284         ]
285         raise ValueError(f"training matrix contains non-finite values in columns:
     {bad_columns}")
```

```
287     model_feature_names = [f"f{index}" for index in range(features.shape[1])]
288     features.columns = model_feature_names
289     if list(features.columns) != model_feature_names:
290         raise ValueError("failed to establish the exact f0..fN model feature
    order")
```

```
292     return features, original_feature_names, categorical_mappings
```

Η prepare_features διατηρεί την αρχική σειρά στηλών, ελέγχει το ακριβές συμβόλαιο, μετατρέπει κατηγορίες, ελέγχει τα υπόλοιπα αριθμητικά πεδία και τελικά μετονομάζει σε f0...fN. Το copy δημιουργεί χωριστή δομή εργασίας από το αρχικό DataFrame. Το astype('float32') ευθυγραμμίζει την αναπαράσταση με το ONNX. Ο έλεγχος isfinite αποτρέπει NaN και άπειρες τιμές που θα δυσκόλευαν τόσο το training όσο και την αποθήκευση JSON.

Δεν εκτελείται τυποποίηση όλων των αριθμητικών εισόδων στον καθολικό trainer. Τα δέντρα λειτουργούν με κατώφλια και δεν χρειάζονται την ίδια κλιμάκωση που απαιτεί μια ευκλείδεια

απόσταση k-means. Η Analysis έχει δική της ροή StandardScaler. Οι δύο περιπτώσεις πρέπει να διακρίνονται, επειδή μια περιγραφή «όλα τα δεδομένα κανονικοποιούνται» θα ήταν ανακριβής.

6.3 Δύο μοντέλα για δύο διαφορετικές μετρικές

Η calculate_validation_r2 κάνει train_test_split με seed 42 και ποσοστό 20%. Εκπαιδεύει προσωρινό validation_model μόνο στο training μέρος και προβλέπει το validation μέρος. Κατόπιν η main δημιουργεί νέο μοντέλο και το εκπαιδεύει σε όλες τις γραμμές. Το δεύτερο είναι το μοντέλο που εξάγεται. Η διαδικασία δεν συνεχίζει την εκπαίδευση του πρώτου αντικειμένου και δεν αποθηκεύει το validation μοντέλο.

Πηγή: Ptixiaki/Worker/Models/TreeCode/Tree.py, γραμμές 295-318. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
295  def calculate_validation_r2(
296      features: pd.DataFrame,
297      target: pd.Series,
298      args: argparse.Namespace,
299  ) -> float:
300      if len(features) < 4:
301          raise ValueError("at least four rows are required for deterministic
     validation R²")
```

```
303      train_x, validation_x, train_y, validation_y = train_test_split(
304          features,
305          target,
306          test_size=args.validation_fraction,
307          random_state=VALIDATION_SEED,
308      )
309      if len(train_x) < 2 or len(validation_x) < 2:
310          raise ValueError("validation split must contain at least two training and
      validation rows")
```

```
312      validation_model = new_model(args)
313      validation_model.fit(train_x, train_y)
314      validation_predictions = validation_model.predict(validation_x)
315      score = float(r2_score(validation_y, validation_predictions))
316      if not np.isfinite(score):
317          raise ValueError("deterministic validation R² is not finite")
318      return score
```

Ο training R^2 υπολογίζεται στις γραμμές που το τελικό μοντέλο έχει ήδη δει. Ο validation R^2 προκύπτει από διαφορετικό μοντέλο και μη χρησιμοποιημένες άμεσα γραμμές στο δικό του fit. Η διαφορά μεταξύ 0,711019 και 0,563221 στις πωλήσεις είναι συνεπής με αυτή τη διάκριση. Δεν υπάρχει λόγος να συμπίπτουν. Η διαφορά από μόνη της δεν αποδεικνύει συγκεκριμένη αιτία υπερπροσαρμογής, αλλά επιβάλλει να μη διαφημίζεται το training score ως αναμενόμενη επίδοση σε νέα ακίνητα.

Το split είναι αναπαραγώγιμο για δεδομένη είσοδο και σειρά γραμμών. Δεν αποτελεί πλήρη επιστημονική αναπαραγωγή όταν λείπουν το αρχικό dataset, η χρονική έκδοσή του και η διαδικασία συλλογής. Επιπλέον, όπως αναλύεται στην αξιολόγηση, το Phase2 input έχει ήδη παραχθεί με target-derived GeoPremium πριν από το split. Ο όρος validation χρησιμοποιείται εδώ όπως αποθηκεύεται στον κώδικα, με σαφή επιφύλαξη για την ανεξαρτησία του πειράματος.

6.4 GeoPremium: κατασκευή και ερμηνεία

Για κάθε γραμμή το Attr μοντέλο δίνει μια πρόβλεψη. Το κατάλοιπο μετρά την απόκλιση από την asking, χρησιμοποιώντας την asking στον παρονομαστή. Το spatial engine αναζητεί έως πενήντα γειτονικές καταχωρίσεις εντός τριακοσίων μέτρων και υπολογίζει μέσο και πληθυσμιακή τυπική απόκλιση των διαθέσιμων καταλοίπων. Ο μέσος γίνεται GeoPremium. Οι χωρικές αναζητήσεις της μαζικής διαδικασίας εκτελούνται σε batches 5.000 ακινήτων.

Η μέτρηση σε μέτρα προϋποθέτει σωστή χρήση γεωγραφικού τύπου ή κατάλληλου προβολικού συστήματος. Η ST_DWithin με geography ερμηνεύει την απόσταση σε μέτρα, ενώ με geometry χρησιμοποιεί τις μονάδες του αντίστοιχου συστήματος αναφοράς (PostGIS Project, χ.χ.). Αυτό διαφέρει από ένα απλό φίλτρο ορθογωνίου σε μοίρες γεωγραφικού μήκους και πλάτους.

$$r_i^{(Attr)} = 100 \cdot (A_i - P_i^{(Attr)}) / A_i \quad (6.1)$$

$$Geo_i = (1 / |N_i|) \cdot \Sigma[j \in N_i] \, r_j^{(Attr)} \quad (6.2)$$

$$\sigma_i = \sqrt{(1 / |N_i|) \cdot \Sigma[j \in N_i] \, (r_j^{(Attr)} - Geo_i)^2} \quad (6.3)$$

Θετικό GeoPremium σημαίνει ότι οι γειτονικές asking υπερβαίνουν κατά μέσο όρο τις αντίστοιχες Attr προβλέψεις. Δεν σημαίνει ότι προστίθενται αυτόματα τόσα ευρώ ή τόσα τοις εκατό στην τελική πρόβλεψη. Η Phase2 μαθαίνει πώς να χρησιμοποιεί το χαρακτηριστικό μαζί με τα υπόλοιπα. Η SHAP συνεισφορά του GeoPremium είναι διαφορετικό μέγεθος και εκφράζεται σε €/τ.μ.

Υπάρχει μια λεπτή διαφορά μεταξύ μαζικής και online διαδρομής. Η μαζική αναζήτηση περιορίζει πρώτα χωρικούς υποψηφίους και μετά διατηρεί όσους έχουν κατάλοιπο στον χάρτη αποτελεσμάτων. Η online διαδρομή περιορίζει πρώτα τα κατάλληλα διαθέσιμα κατάλοιπα και μετά επιλέγει έως πενήντα. Οι τελικοί γείτονες μπορούν επομένως να διαφέρουν. Σε έλλειψη γειτόνων η offline λογική μπορεί να χρησιμοποιήσει μηδέν, ενώ η online ροή μπορεί να επιστρέψει σφάλμα. Η ίδια μαθηματική ιδέα δεν συνεπάγεται απολύτως ίδιο πληθυσμό αναφοράς.

6.5 Phase2 και παραγωγή των artifacts

Μετά τη συγχώνευση GeoPremium, η Phase2 εκπαιδεύεται σε τριάντα χαρακτηριστικά και εξάγεται σε ONNX. Το ONNX επιτρέπει inference μέσα στη Go χωρίς μεταφορά κάθε πρόβλεψης στον Python server. Παράλληλα, η native αναπαράσταση UBJ διατηρεί το δέντρο σε μορφή που φορτώνεται από XGBoost για τις εξηγήσεις. Πρόκειται για δύο αναπαραστάσεις του ίδιου τελικού fit, όχι για δύο ανεξάρτητα εκπαιδευμένα μοντέλα.

Πηγή: Ptixiaki/Worker/Models/TreeCode/Tree.py, γραμμές 692-729. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
692     log(f"Training final {base_name} model on all {len(features)} rows...")
693     stage_started = time.perf_counter()
694     emit_progress(args, "training", 0, 1, stage_started)
695     model = new_model(args)
696     model.fit(features, target)
697     predictions = np.asarray(model.predict(features), dtype=np.float64)
698     if predictions.shape != (len(df),) or not np.isfinite(predictions).all():
699         raise ValueError("final model predictions are missing or non-finite")
700     training_r2 = float(r2_score(target, predictions))
701     if not np.isfinite(training_r2):
702         raise ValueError("all-row training R² is not finite")
703     log(f"Final all-row {base_name}: R2={training_r2:.6f}")
704     emit_progress(args, "training", 1, 1, stage_started)
```

```
706     output_df = pd.DataFrame(index=df.index)
707     if "UID" in df.columns:
708         output_df["UID"] = df["UID"]
709     output_df["AskingPricePerSqm"] = target
710     output_df["PredictedPricePerSqm"] = predictions
712     output_df["ResidualPercentage"] = ((target - predictions) / target) * 100.0
713     if not np.isfinite(
714         output_df["ResidualPercentage"].to_numpy(dtype=np.float64)
715     ).all():
716         raise ValueError("residual percentages are non-finite")
717     prediction_json = output_df.to_json(orient="records", indent=2)
```

```
719     log("Converting final model to ONNX format...")
```

```
720     stage_started = time.perf_counter()
721     emit_progress(args, "onnx", 0, 1, stage_started)
722     initial_types = [("input", FloatTensorType([None, features.shape[1]]))]
723     onnx_model = onnxmltools.convert_xgboost(
724         model,
725         initial_types=initial_types,
726         target_opset=15,
727     )
728     onnx_bytes = onnx_model.SerializeToString()
729     emit_progress(args, "onnx", 1, 1, stage_started)
```

Η εξαγωγή σε ONNX είναι συγκεκριμένο στάδιο και δεν περιλαμβάνει όλο το υπόλοιπο κόστος SHAP. Τα progress records δίνουν διαφορετικά stage names για preparation, validation, training, onnx, artifacts, shap_properties, shap_interactions και activate. Έτσι μια μεγάλη αναμονή μπορεί να αποδοθεί στο πραγματικό βήμα. Η υψηλή CPU κατά τις αλληλεπιδράσεις δεν αποδεικνύει από μόνη της ατέρμονο βρόχο ή πρόβλημα κατηγορικών μεταβλητών.

Τα τελικά ονόματα είναι σταθερά και αποτελούν συμβόλαιο. Για τη συναλλαγή sell, το stem είναι Phase2_residence_properties_sell. Το αρχείο Tree.onnx χρησιμοποιείται από Go, το Tree.ubj από Python και το Manifest.json συνδέει δεδομένα, σχήμα και ταυτότητα. Τα mappings και οι προβλέψεις JSON παραμένουν ξεχωριστά outputs. Το GeoPrem_residence_properties_sell.json είναι προϊόν της ενδιάμεσης χωρικής φάσης.

6.6 Μαθηματική κατάταξη Elo

Η βαθμολογία χρησιμοποιεί τρία τυποποιημένα μεγέθη: το τελικό κατάλοιπο ως προς την αγορά, το ίδιο κατάλοιπο ως προς τοπικούς συγκρίσιμους και το GeoPremium ως προς την αγορά. Για τα δύο πρώτα οι αρνητικές αποκλίσεις αυξάνουν τη βαθμολογία, επειδή αντιστοιχούν σε asking μικρότερη από την πρόβλεψη. Για το GeoPremium ο θετικός τυποποιημένος όρος αυξάνει τη βαθμολογία.

$$Z_r = (r - \mu_r) / \sigma_r \; ; \; Z_{local} = (r - \mu_{local}) / \sigma_{local} \quad (6.4)$$

$$Z_{geo} = (Geo - \mu_{geo}) / \sigma_{geo} \quad (6.5)$$

$$Elo = 10000 - 3000 \cdot Z_r - 2000 \cdot Z_{local} + 3000 \cdot Z_{geo} \quad (6.6)$$

Το τοπικό σύνολο της βαθμολογίας διαφέρει από το σύνολο GeoPremium: χρησιμοποιεί έως είκοσι γείτονες εντός εκατόν πενήντα μέτρων και τελικά κατάλοιπα Phase2. Δεν πρέπει να χρησιμοποιηθεί η GeoStd ως υποκατάστατο της τυπικής απόκλισης τοπικών τελικών καταλοίπων. Η πρώτη χαρακτηρίζει την κατασκευή του χωρικού feature, η δεύτερη τη σύγκριση της συμφωνίας ως προς τη γειτονιά.

Πηγή: Ptixiaki/Backend/Api/Elo/CalculateElo.go, γραμμές 16-47. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
16  func CalculateElo(ctx context.Context, pool Queryer, uid, transaction string)
    (float64, float64, error) {
17      if err := explain.ValidateTransaction(transaction); err != nil {
18          return 0, 0, err
19      }
20      var geoPremium float64
21      var residual *float64
22      err := pool.QueryRow(ctx, "SELECT geo_premium, residual_percentage FROM
    estimation_data WHERE uid = $1", uid).Scan(&geoPremium, &residual)
23      if err != nil {
24          return 0, 0, fmt.Errorf("read property scoring data: %w", err)
25      }
26      var geoMean, geoStd float64
27      err = pool.QueryRow(ctx, "SELECT mean, std FROM global_stats WHERE stat_name
    = $1", "Geo_stats_"+transaction).Scan(&geoMean, &geoStd)
28      if err != nil {
29          return 0, 0, fmt.Errorf("read global GeoPremium statistics: %w", err)
30      }
```

```
31      geoZ := p.CalculateZScore(geoPremium, geoMean, geoStd)
34      if residual == nil {
35          return p.CalculateEloScore(0, 0, geoZ), geoZ, nil
36      }
37      var residualMean, residualStd float64
38      err = pool.QueryRow(ctx, "SELECT mean, std FROM global_stats WHERE stat_name
    = $1", "Price_stats_"+transaction).Scan(&residualMean, &residualStd)
39      if err != nil {
40          return 0, 0, fmt.Errorf("read global residual statistics: %w", err)
41      }
42      localZ, err := CalculateLocalDealValue(ctx, pool, uid, transaction,
    *residual)
43      if err != nil {
44          return 0, 0, err
45      }
46      return p.CalculateEloScore(p.CalculateZScore(*residual, residualMean,
    residualStd), localZ, geoZ), geoZ, nil
47  }
```

Με σταθερά θετική πρόβλεψη P και στατιστικά αναφοράς, το r = 100 - 100P/A αυξάνει όταν αυξάνει η θετική asking A. Επομένως, μικρότερη asking οδηγεί σε μικρότερο r και, λόγω των αρνητικών βαρών, μεγαλύτερο Elo. Αυτό είναι ο επιθυμητός προσανατολισμός. Αν όμως η asking πλησιάζει το μηδέν, το κατάλοιπο γίνεται πολύ αρνητικό και η βαθμολογία μπορεί να εκτιναχθεί. Ο κώδικας δεν τεκμηριώνει οικονομικά βαθμονομημένο άνω όριο ή πιθανότητα επενδυτικής επιτυχίας.

Χωρίς asking, το residual είναι nil. Παραλείπονται οι δύο όροι συμφωνίας και απομένει 10000 + 3000 · Z_geo. Δεν υπολογίζεται ότι το ακίνητο είναι δωρεάν ούτε ότι η asking ισούται με την estimate. Με μηδενική τυπική απόκλιση ο αντίστοιχος z-score γίνεται μηδέν. Η αποθήκευση στο properties_elo είναι ακέραιη, άρα η δεκαδική τιμή των logs και η εμφανιζόμενη τιμή μπορεί να διαφέρουν στο κλασματικό μέρος.

Η ονομασία Elo είναι εσωτερική ονομασία ευρετικής κατάταξης. Δεν υλοποιείται το κλασικό σύστημα διαδοχικών αναμετρήσεων. Τα βάρη 3000, 2000 και 3000 είναι επιλογές σχεδιασμού που χρειάζονται ανεξάρτητη αξιολόγηση. Για αποφυγή υπερβολικής εμπιστοσύνης, η βαθμολογία πρέπει να παρουσιάζεται μαζί με asking, estimate και εξηγήσεις, όχι ως αυτόνομη ετυμηγορία «καλής αγοράς».

7. Εξηγησιμότητα, ροές και δημοσίευση μοντέλων

7.1 Γιατί υπάρχει ξεχωριστό πακέτο explainability

Το πακέτο Worker/explainability δεν εκπαιδεύει το δέντρο. Συγκεντρώνει το συμβόλαιο που συνδέει εκπαίδευση, Go, Python και βάση. Το contract.go περιγράφει χαρακτηριστικά, ονόματα, μονάδες, timeouts και ανοχές. Το records.go διαβάζει και επικυρώνει εγγραφές από τον trainer. Το training.go οργανώνει staging και δημοσίευση στη βάση. Το artifacts.go ελέγχει τα τοπικά UBJ/manifest. Το client.go χειρίζεται την HTTP επικοινωνία για reload και online explanation. Το progress.go μετατρέπει αριθμητικά γεγονότα προόδου σε ανθρώπινα logs.

Ο διαχωρισμός είναι χρήσιμος επειδή τα ίδια λάθη μπορούν να εμφανιστούν σε διαφορετικά σημεία. Αν η σειρά χαρακτηριστικών αλλάξει, πρέπει να εντοπιστεί τόσο κατά την εκπαίδευση όσο και κατά την online εξήγηση. Αν μια απάντηση περιέχει NaN, δεν πρέπει να φτάσει στη βάση. Αν η Python φόρτωσε μοντέλο από λάθος φάκελο, μια επιτυχής HTTP απόκριση δεν αρκεί. Το πακέτο συγκεντρώνει τους ελέγχους ώστε οι handlers να μην επαναλαμβάνουν πολύπλοκη λογική.

Πηγή: Ptixiaki/Worker/explainability/contract.go, γραμμές 12-43. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
12  const (
13      RecordPrefix = "SHAP_RECORD\t"

15      FeatureCount = 30

17      DefaultServiceBaseURL = "http://localhost:5000"
18      ExplainPropertyPath   = "/ExplainProperty"
19      ReloadModelsPath      = "/ReloadExplainModels"

21      HTTPTimeout                   = 20 * time.Second
22      TrainingProcessTimeout        = 2 * time.Hour
23      MaxResponseBytes              = 2 << 20
24      ScannerMaxTokenBytes          = 16 << 20
25      PropertyStageBatchSize        = 500
26      TrainingSHAPBatchSize         = 256
27      TrainingInteractionSampleSize = 2000
28      TrainingInteractionSampleSeed = 42
29      ValidationFraction            = 0.2
30      TargetName                    = "AskingPricePerSqm"
31      TargetUnit                    = "EUR_per_square_meter"

36      AdditivityAbsTolerance = 1e-3
37      AdditivityRelTolerance = 1e-5

41      ONNXNativeAbsTolerance = 0.10
42      ONNXNativeRelTolerance = 1e-4
43  )
```

Τα όρια είναι διαφορετικών ειδών. Το TrainingProcessTimeout αφορά μια ολόκληρη child process και είναι δύο ώρες. Το HTTPTimeout αφορά το σύντομο εσωτερικό αίτημα reload/explain και είναι είκοσι δευτερόλεπτα. Το TrainingSHAPBatchSize 256 καθορίζει πόσες γραμμές εξηγούνται μαζί στην Python. Το PropertyStageBatchSize 500 καθορίζει πόσες γραμμές συγκεντρώνει η Go πριν από COPY στη βάση. Τα δύο batch sizes δεν χρειάζεται να συμπίπτουν, επειδή ελέγχουν διαφορετικά κόστη.

7.2 Τοπικές συνεισφορές και καθολική σημασία

Το native booster.predict με pred_contribs=True επιστρέφει m+1 τιμές ανά γραμμή. Οι πρώτες m είναι οι συνεισφορές και η τελευταία η baseline. Ο trainer μετατρέπει τις εξόδους σε float64 για συσσώρευση και ελέγχει το σχήμα και την περατότητα. Η χρήση float64 στην άθροιση δεν αναιρεί ότι η βασική πρόβλεψη προέρχεται από float32 αριθμητική. Μειώνει πρόσθετο σφάλμα συσσώρευσης, δεν μετατρέπει το μοντέλο σε απεριόριστης ακρίβειας υπολογισμό.

Πηγή: Ptixiaki/Worker/Models/TreeCode/Tree.py, γραμμές 486-507. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
486     for start in range(0, len(features), args.shap_batch_size):
487         end = min(start + args.shap_batch_size, len(features))
488         batch_values = features.iloc[start:end].to_numpy(dtype=np.float32,
    copy=False)
489         matrix = xgb.DMatrix(batch_values, feature_names=model_feature_names)
```

```
491         native_predictions = np.asarray(
492             booster.predict(matrix, validate_features=True), dtype=np.float64
493         ).reshape(-1)
494         contributions_with_bias = np.asarray(
495             booster.predict(matrix, pred_contribs=True, validate_features=True),
496             dtype=np.float64,
497         )
498         expected_contribution_shape = (end - start, feature_count + 1)
499         if contributions_with_bias.shape != expected_contribution_shape:
500             raise ValueError(
501                 "unexpected pred_contribs shape: "
502                 f"{contributions_with_bias.shape}, expected
    {expected_contribution_shape}"
503             )
504         if not np.isfinite(native_predictions).all() or not np.isfinite(
505             contributions_with_bias
506         ).all():
507             raise ValueError("native predictions or SHAP contributions contain
    non-finite values")
```

Για ένα ακίνητο, το τοπικό ποσοστό σημασίας υπολογίζεται από την απόλυτη τιμή της κάθε συνεισφοράς ως ποσοστό του αθροίσματος των απόλυτων συνεισφορών. Το πρόσημο διατηρείται ξεχωριστά στο impact_per_sqm. Έτσι ένα χαρακτηριστικό μπορεί να έχει υψηλό ποσοστό σημασίας και αρνητική επίδραση. Το ποσοστό δεν είναι πιθανότητα ούτε ποσοστό της συνολικής τιμής, επειδή η baseline δεν συμμετέχει στον παρονομαστή.

$$\text{LocalImportance}_j(x) = 100 \cdot |\varphi_j(x)| \,/\, \Sigma_{k=1..m} |\varphi_k(x)| \quad (7.1)$$

Η καθολική σημασία προκύπτει από τον μέσο όρο των απόλυτων συνεισφορών σε όλες τις γραμμές εκπαίδευσης. Αποθηκεύεται και ο προσημασμένος μέσος, αλλά δεν χρησιμοποιείται ως μέγεθος ισχύος επειδή θετικές και αρνητικές τιμές μπορούν να αλληλοαναιρεθούν. Ένα χαρακτηριστικό με μέσο +2 μπορεί να έχει μεγάλες αποκλίσεις +300 και -296 σε διαφορετικά ακίνητα. Ο μέσος απόλυτων τιμών αποτυπώνει καλύτερα το μέγεθος της συμμετοχής.

$$I_j = (1 / n) \cdot \Sigma_{i=1..n} |\varphi_j(x_i)| \quad (7.2)$$

$$\text{GlobalImportance}_j = 100 \cdot I_j \,/\, \Sigma_{k=1..m} I_k \quad (7.3)$$

Στη συγκεκριμένη υλοποίηση οι απλές συνεισφορές και η καθολική σημασία υπολογίζονται σε κάθε γραμμή του dataset. Η δειγματοληψία των 2.000 ακινήτων δεν περιορίζει την κάλυψη properties_analysis. Αυτό είναι ουσιαστικό για τη λειτουργία των λεπτομερειών: ο χρήστης μπορεί να επιλέξει ακίνητο που δεν ανήκε στο δείγμα αλληλεπιδράσεων και παρ’ όλα αυτά να έχει αποθηκευμένη εξήγηση.

7.3 Αλληλεπιδράσεις χαρακτηριστικών και κόστος

Οι αλληλεπιδράσεις SHAP αποδίδουν τη συμμετοχή συνδυασμών χαρακτηριστικών. Στην κατοικία, για παράδειγμα, η σχέση ορόφου και ανελκυστήρα μπορεί να έχει διαφορετική σημασία από την ανεξάρτητη παρουσία των δύο. Στο native output υπάρχει τετραγωνικός πίνακας ανά ακίνητο. Η υλοποίηση διατηρεί μόνο τα μοναδικά ζεύγη i < j και όχι τη διαγώνιο. Για τριάντα χαρακτηριστικά προκύπτουν 30 × 29 / 2 = 435 ζεύγη.

$$\text{PairCount} = m \cdot (m - 1) / 2 = 435, \text{ για } m = 30 \quad (7.4)$$

Το μέγεθος του πλήρους tensor για 123.449 ακίνητα και 31 × 31 τιμές είναι περίπου 118,6 εκατομμύρια αριθμοί πριν από επιπλέον βοηθητική μνήμη. Ακόμη και όταν η επεξεργασία γίνεται τμηματικά, ο υπολογισμός όλων των αλληλεπιδράσεων παραμένει σημαντικά ακριβότερος από τις απλές συνεισφορές. Γι’ αυτό η εφαρμογή χρησιμοποιεί έως 2.000 ακίνητα ανά αγορά, με ομοιόμορφη δειγματοληψία χωρίς επανάθεση και seed 42.

Πηγή: Ptixiaki/Worker/Models/TreeCode/Tree.py, γραμμές 95-108. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
 95  def interaction_sample_indices(uids: list[str], sample_size: int, seed: int) ->
     np.ndarray:
 96      if not uids or sample_size <= 0 or seed < 0:
 97          raise ValueError(
 98              "interaction sampling requires rows, a positive sample size and a
     nonnegative seed"
 99          )
100      if len(uids) <= sample_size:
101          return np.arange(len(uids))
104      ordered = np.asarray(sorted(range(len(uids)), key=uids.__getitem__))
105      selected = np.random.default_rng(seed).choice(
106          len(uids), size=sample_size, replace=False
107      )
108      return ordered[np.sort(selected)]
```

Πριν από τη δειγματοληψία γίνεται ταξινόμηση με σταθερό UID. Έτσι η ίδια ομάδα επιλέγεται όταν αλλάξει μόνο η σειρά επιστροφής της βάσης. Αν ο πληθυσμός είναι μικρότερος από το όριο, χρησιμοποιούνται όλες οι γραμμές. Η αναπαραγωγιμότητα του δείγματος δεν αποτελεί μέτρο ακρίβειας της εκτίμησης αλληλεπιδράσεων. Δεν υπολογίζονται διαστήματα εμπιστοσύνης ή επαναλαμβανόμενες δειγματοληψίες για σταθερότητα κατάταξης.

Αποθηκεύονται ο προσημασμένος και ο απόλυτος μέσος της μιας εκτός διαγωνίου θέσης κάθε ζεύγους. Δεν διπλασιάζεται αυθαίρετα η τιμή επειδή ο πίνακας είναι συμμετρικός. Για σωστή ερμηνεία πρέπει να είναι σαφές ποια σύμβαση αναφέρει η εφαρμογή. Οι αλληλεπιδράσεις παραμένουν ιδιότητες του μοντέλου και του πληθυσμού αναφοράς, όχι εκτιμήσεις ανεξάρτητων παρεμβάσεων στην πραγματική κατοικία.

7.4 Αριθμητική προσθετικότητα και ανοχές

Η βασική ταυτότητα SHAP ελέγχεται πρακτικά με ανοχή. Η απολύτως ακριβής ισότητα είναι ακατάλληλη για float32. Αν μεγάλες θετικές και αρνητικές συνεισφορές σχεδόν ακυρώνονται, η τελική πρόβλεψη μπορεί να είναι μικρή, ενώ το ενδιάμεσο άθροισμα έχει πολύ μεγαλύτερη κλίμακα. Μια σχετική ανοχή που εξαρτάται μόνο από την πρόβλεψη θα μπορούσε να απορρίψει κανονικό αριθμητικό θόρυβο.

$$\varepsilon_{allowed} = 0{,}001 + 0{,}00001 \cdot \max(|P|, |B| + \Sigma |\varphi_j|) \quad (7.5)$$

Η συγκεκριμένη ανοχή λαμβάνει υπόψη το μέγεθος των προστιθέμενων όρων. Το αποθηκευόμενο additivity_error είναι reconstructed minus prediction. Ο έλεγχος συγκρίνει την απόλυτη απόκλιση με το όριο της ίδιας γραμμής. Στο μήνυμα αποτυχίας επιλέγεται γραμμή που πράγματι παραβιάζει το όριό της και όχι απλώς η γραμμή με το μεγαλύτερο απόλυτο σφάλμα. Διαφορετικά θα μπορούσε να εμφανιστεί μήνυμα όπου το αναφερόμενο error είναι μικρότερο από το allowed, παρότι άλλη γραμμή απέτυχε.

Πηγή: ml_shared/shap_validation.py, γραμμές 1-34. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
  1  """Numerical validation shared by offline and on-demand native TreeSHAP."""

  3  from __future__ import annotations

  5  import numpy as np

  8  ADDITIVITY_ATOL = 0.001
  9  ADDITIVITY_RTOL = 0.00001

 12  def additivity_tolerance(
```

```
    predictions,
    baselines,
    contributions,
    *,
    atol: float = ADDITIVITY_ATOL,
    rtol: float = ADDITIVITY_RTOL,
):
    """Allow float32 rounding on the terms being summed, even near cancellation.
    The native outputs are float32 despite our float64 reconstruction. Scaling
    only by the prediction makes the check arbitrarily strict when large signed
    terms cancel. Use their L1 magnitude (or prediction, whichever is larger).
    This does not adjust the native prediction, baseline, or contributions.
    """
    if not np.isfinite(atol) or not np.isfinite(rtol) or atol < 0 or rtol < 0:
        raise ValueError("SHAP tolerances must be finite and nonnegative")
    contributions = np.asarray(contributions, dtype=np.float64)
    term_magnitude = np.abs(np.asarray(baselines, dtype=np.float64)) + np.abs(
        contributions
    ).sum(axis=-1, dtype=np.float64)
    scale = np.maximum(np.abs(np.asarray(predictions, dtype=np.float64)),
term_magnitude)
    return atol + rtol * scale
```

Ο έλεγχος ONNX προς native prediction είναι διαφορετικός. Χρησιμοποιεί ανοχή 0,10 €/τ.μ. απόλυτη ή 10^{-4} σχετική. Εξετάζει αν οι δύο μορφές μοντέλου δίνουν συμβατές προβλέψεις στην ίδια είσοδο. Δεν πρέπει να συγχέεται με την ανοχή της εσωτερικής προσθετικότητας. Το πρώτο αφορά μεταφορά μοντέλου, το δεύτερο άθροισμα εξηγήσεων. Και τα δύο ελέγχουν αριθμητική συνέπεια, όχι ακρίβεια ως προς την πραγματική αγορά.

7.5 Πρωτόκολλο stdout και προοδευτική επεξεργασία

Ο trainer στέλνει μία JSON εγγραφή ανά γραμμή με prefix SHAP_RECORD και tab. Οι τύποι είναι meta, property, importance, interaction και progress. Τα ανθρώπινα μηνύματα της Python γράφονται στο stderr. Η Go απαιτεί καθαρό πρωτόκολλο στο stdout· μια απρόσμενη μη κενή γραμμή αντιμετωπίζεται ως σφάλμα. Η ολοκλήρωση επιβεβαιώνεται με λήξη της διεργασίας, επιτυχές exit status και Finalize ελέγχους πλήθους και πληρότητας, όχι με υπόθεση ότι «η έξοδος σταμάτησε άρα όλα πήγαν καλά».

Πηγή: Ptixiaki/Worker/explainability/records.go, γραμμές 12-66. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
type MetaRecord struct {
    Type          string `json:"type"`
    SchemaVersion int    `json:"schema_version"`
    Transaction   string `json:"transaction"`
    Target        struct {
        Name string `json:"name"`
        Unit string `json:"unit"`
    } `json:"target"`
    FeatureNames           []string `json:"feature_names"`
    ModelFeatureNames      []string `json:"model_feature_names"`
    FeatureCount           int      `json:"feature_count"`
    PropertyCount          int      `json:"property_count"`
    InteractionCount       int      `json:"interaction_count"`
    InteractionSampleCount int      `json:"interaction_sample_count"`
    InteractionSampleSeed  int      `json:"interaction_sample_seed"`
    BaselinePricePerSqm    float64  `json:"baseline_price_per_sqm"`
    TrainingR2             float64  `json:"training_r2"`
    ValidationR2           float64  `json:"validation_r2"`
    ModelSHA256            string   `json:"model_sha256"`
    TrainedAt              string   `json:"trained_at"`
}
type PropertyRecord struct {
    Type                  string    `json:"type"`
```

```
36      UID                   string    `json:"uid"`
37      Contributions         []float64 `json:"contributions"`
38      PredictionPricePerSqm float64   `json:"prediction_price_per_sqm"`
39      AdditivityError       float64   `json:"additivity_error"`
40  }
```

```
42  type ImportanceRecord struct {
43      Type        string  `json:"type"`
44      FeatureName string  `json:"feature_name"`
45      MeanAbs     float64 `json:"mean_abs"`
46      MeanSigned  float64 `json:"mean_signed"`
47      Percentage  float64 `json:"percentage"`
48      Rank        int     `json:"rank"`
49  }
```

```
51  type InteractionRecord struct {
52      Type       string  `json:"type"`
53      Feature1   string  `json:"feature_1"`
54      Feature2   string  `json:"feature_2"`
55      MeanAbs    float64 `json:"mean_abs"`
56      MeanSigned float64 `json:"mean_signed"`
57      Rank       int     `json:"rank"`
58  }
```

```
60  type TrainingRecords struct {
61      Meta          MetaRecord
62      TrainedAt     time.Time
63      Importance    []ImportanceRecord
64      Interactions  []InteractionRecord
65      PropertyCount int
66  }
```

Το meta δηλώνει transaction, αριθμό ιδιοτήτων, χαρακτηριστικών και ζευγών, ονόματα, baseline, μετρικές και hash. Το property μεταφέρει UID, τριάντα συνεισφορές, πρόβλεψη και σφάλμα αθροίσματος. Το importance μεταφέρει ένα συγκεντρωτικό αποτέλεσμα χαρακτηριστικού. Το interaction ένα ζεύγος. Το progress δεν αποθηκεύεται ως επιχειρησιακή πληροφορία· μετατρέπεται σε log με ολοκληρωμένο πλήθος, ποσοστό, elapsed και προσεγγιστικό ETA.

Η ροή αποφεύγει τη δημιουργία ενός τεράστιου αρχείου SHAP που θα έπρεπε να διαβαστεί ολόκληρο. Δεν σημαίνει μηδενική μνήμη. Η Python διατηρεί το dataset και το μοντέλο, ενώ ο parser κρατά στοιχεία για έλεγχο μοναδικότητας UID και συγκεντρωτικές εγγραφές. Το κέρδος είναι ότι δεν χρειάζεται να κρατηθεί στη Go όλος ο πίνακας n × 30 συνεισφορών πριν αρχίσει η αποθήκευση.

7.6 Reader, pipe, Scanner και context

Το io.Reader είναι μικρό interface με μέθοδο Read (Go Authors, χ.χ.-α). Περιγράφει κάτι από το οποίο μπορούν να διαβαστούν bytes, χωρίς να δεσμεύει τον χρήστη σε αρχείο, HTTP body ή μνήμη. Το io.ReadCloser προσθέτει Close, επειδή ορισμένες πηγές κατέχουν πόρο που πρέπει να απελευθερωθεί. Ένα HTTP response body πρέπει να κλείνει ώστε η σύνδεση να μπορεί να επαναχρησιμοποιηθεί. Η ίδια γενική ιδέα επιτρέπει στο json.Decoder να διαβάζει από πολλές πηγές.

Η StdoutPipe δημιουργεί σύνδεση ανάμεσα στη stdout της child process και στον γονέα (Go Authors, χ.χ.-β). Η Go δεν μπορεί να περιμένει πρώτα τον τερματισμό και μετά να αρχίσει να διαβάζει απεριόριστη έξοδο: αν γεμίσει ο pipe buffer, η Python θα περιμένει να καταναλωθούν bytes. Γι’ αυτό ξεκινά η διεργασία με Start, η Go διαβάζει κατά τη λειτουργία της και τελικά καλεί Wait. Η ροή παρέχει φυσικό backpressure όταν ο καταναλωτής είναι πιο αργός.

Πηγή: Ptixiaki/Worker/explainability/training.go, γραμμές 93-153. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
93      stager := &propertyStager{ctx: runCtx, tx: tx, rows: make([][]any, 0,
    PropertyStageBatchSize)}
94      parser, err := NewStreamParser(transaction, stager.add)
```

```
95      if err != nil {
96          return err
97      }
98      progress := &progressLogger{transaction: transaction, printf: log.Printf}
99      parser.SetProgressHandler(progress.report)
```

```
101     cmdArgs := append([]string{scriptPath}, args...)
102     cmd := exec.CommandContext(runCtx, "python", cmdArgs...)
103     stdout, err := cmd.StdoutPipe()
104     if err != nil {
105         return fmt.Errorf("open Python stdout: %w", err)
106     }
107     cmd.Stderr = os.Stderr
```

```
109     log.Printf("[TRAIN %s] Launching Python training with explanations: %s",
    transaction, scriptPath)
110     if err := cmd.Start(); err != nil {
111         return fmt.Errorf("start Python training: %w", err)
112     }
```

```
114     scanner := bufio.NewScanner(stdout)
115     scanner.Buffer(make([]byte, 64*1024), ScannerMaxTokenBytes)
116     var protocolErr error
117     metaLogged := false
118     for scanner.Scan() {
119         line := scanner.Text()
120         recognized, parseErr := parser.ParseLine(line)
121         if parseErr != nil {
122             protocolErr = parseErr
123             break
124         }
125         if !recognized && strings.TrimSpace(line) != "" {
126             protocolErr = fmt.Errorf("unexpected non-protocol Python stdout: %q",
     line)
127             break
128         }
129         if parser.meta != nil && !metaLogged {
130             log.Printf("[TRAIN %s] Explaining all %d properties; global
    interactions use %d properties (seed %d)",
131                 transaction, parser.meta.PropertyCount,
    parser.meta.InteractionSampleCount, parser.meta.InteractionSampleSeed)
132             metaLogged = true
133         }
134     }
135     if protocolErr == nil {
136         protocolErr = scanner.Err()
137     }
138     if protocolErr != nil && cmd.Process != nil {
139         _ = cmd.Process.Kill()
140     }
141     waitErr := cmd.Wait()
142     if protocolErr != nil {
143         return fmt.Errorf("invalid Python explanation stream: %w", protocolErr)
144     }
145     if waitErr != nil {
146         if errors.Is(runCtx.Err(), context.DeadlineExceeded) {
147             return fmt.Errorf("Python explanation training timed out after %s",
    TrainingProcessTimeout)
148         }
149         return fmt.Errorf("Python training process failed: %w", waitErr)
150     }
```

```
152     records, err := parser.Finalize()
153     if err != nil {
```

Ο Scanner χωρίζει τη ροή σε γραμμές (Go Authors, χ.χ.-γ). Το make([]byte, 64*1024) δίνει αρχικό buffer 64 KiB και το ScannerMaxTokenBytes ανώτατο επιτρεπόμενο μέγεθος μίας εγγραφής 16 MiB. Δεν δεσμεύονται από την αρχή 16 MiB για κάθε γραμμή και δεν αφορά το

μέγεθος ολόκληρης της εκπαίδευσης. Αν μια γραμμή ξεπεράσει το όριο, ο Scanner επιστρέφει σφάλμα. Αυτό αποτρέπει απεριόριστη κατανάλωση μνήμης από κατεστραμμένη έξοδο.

Το cmd.Stderr = os.Stderr συνδέει το κανάλι σφαλμάτων του παιδιού με το αντίστοιχο του γονέα. Δεν αναλύει JSON και δεν τυπώνει το stderr μέσω progress.go. Η απαίτηση «τα progress logs να τυπώνονται από Go» καλύπτεται από τα structured progress records. Τα διαγνωστικά Python εξακολουθούν να εμφανίζονται στο ίδιο terminal μέσω ξεχωριστής σύνδεσης. Ο διαχωρισμός διατηρεί το πρωτόκολλο καθαρό και επιτρέπει να φαίνεται traceback όταν αποτύχει ο trainer.

7.7 Staging και δημοσίευση στη βάση

Η RunTrainingAndPersist ανοίγει συναλλαγή και δημιουργεί προσωρινούς πίνακες. Το propertyStager κρατά έως 500 εγγραφές, τις περνά με CopyFrom και επαναχρησιμοποιεί το slice. Το rows[:0] μηδενίζει το μήκος χωρίς να απαιτεί νέα κατανομή για κάθε batch. Ο parser λαμβάνει τη μέθοδο stager.add ως callback. Έτσι η λογική επαλήθευσης δεν χρειάζεται να γνωρίζει λεπτομέρειες SQL: όταν μια property record είναι έγκυρη, την παραδίδει στον αποθηκευτικό μηχανισμό.

Πηγή: Ptixiaki/Worker/explainability/training.go, γραμμές 18-57. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
18  type propertyStager struct {
19      ctx   context.Context
20      tx    pgx.Tx
21      rows  [][]any
22      count int
23  }
```

```
25  func (s *propertyStager) add(record PropertyRecord) error {
26      s.rows = append(s.rows, []any{
27          record.UID,
28          record.Contributions,
29          record.PredictionPricePerSqm,
30          record.AdditivityError,
31      })
32      if len(s.rows) >= PropertyStageBatchSize {
33          return s.flush()
34      }
35      return nil
36  }
```

```
38  func (s *propertyStager) flush() error {
39      if len(s.rows) == 0 {
40          return nil
41      }
42      count, err := s.tx.CopyFrom(
43          s.ctx,
44          pgx.Identifier{"shap_property_stage"},
45          []string{"property_uid", "contributions", "prediction_price_per_sqm",
    "additivity_error"},
46          pgx.CopyFromRows(s.rows),
47      )
48      if err != nil {
49          return err
50      }
51      if count != int64(len(s.rows)) {
52          return fmt.Errorf("staged %d property rows, expected %d", count,
    len(s.rows))
53      }
54      s.count += int(count)
55      s.rows = s.rows[:0]
56      return nil
57  }
```

Μετά την επιτυχή ολοκλήρωση, η Finalize ελέγχει ότι υπάρχουν όλα τα αναμενόμενα στοιχεία. Το υπόλοιπο batch γίνεται flush. Ακολουθεί staging των συγκεντρωτικών εγγραφών και το beforePublish callback που ενημερώνει εκτιμήσεις, στατιστικά και Elo. Στη συνέχεια αντικαθίστανται οι πίνακες εξηγήσεων της συγκεκριμένης συναλλαγής και γίνεται commit. Αν η συναλλαγή εξηγήσεων αποτύχει, τα προσωρινά δεδομένα δεν δημοσιεύονται.

Πηγή: Ptixiaki/Worker/explainability/training.go, γραμμές 155-182. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
155     }
156     log.Printf("[TRAIN %s] Python finished; validated %d property, %d importance
    and %d interaction records. Finishing database staging", transaction,
    records.PropertyCount, len(records.Importance), len(records.Interactions))
157     if err := stager.flush(); err != nil {
158         return fmt.Errorf("flush property explanation staging: %w", err)
159     }
160     if stager.count != records.PropertyCount {
161         return fmt.Errorf("staged property count mismatch: got %d, expected %d",
    stager.count, records.PropertyCount)
162     }
163     if err := stageAggregateRecords(runCtx, tx, records); err != nil {
164         return err
165     }
166     if beforePublish != nil {
167         log.Printf("[TRAIN %s] Persisting predictions, global statistics and
    Elo", transaction)
168         if err := beforePublish(runCtx); err != nil {
169             return fmt.Errorf("complete dependent training persistence before
    explanation publication: %w", err)
170         }
171     }
172     log.Printf("[TRAIN %s] Publishing explanation tables and model statistics",
    transaction)
173     if err := publishStagedRecords(runCtx, tx, transaction, records); err != nil
    {
174         return err
175     }
176     if err := tx.Commit(runCtx); err != nil {
177         return fmt.Errorf("commit explanation publication: %w", err)
178     }
180     log.Printf("[TRAIN %s] Explanation publication committed successfully
    (elapsed %s)", transaction, time.Since(started).Round(time.Second))
181     return nil
182 }
```

Το callback δεν χρησιμοποιεί αυτόματα την ίδια pgx.Tx: καλείται με context και οι παλαιότερες λειτουργίες γράφουν μέσω του pool. Άρα η συναλλαγή SHAP δεν περικλείει όλη τη δημοσίευση των derived δεδομένων. Επίσης τα αρχεία ενεργοποιούνται στην Python πριν επιστρέψει επιτυχής exit status. Η εργασία δεν περιγράφει αυτή τη ροή ως μια καθολική ACID συναλλαγή. Η ακριβής εγγύηση είναι στενότερη: staging και αντικατάσταση των explanation tables μέσα σε μία συναλλαγή, μαζί με επιμέρους ελέγχους σε άλλα επίπεδα.

7.8 UBJ, manifest και SHA-256

Το UBJ είναι η δυαδική JSON μορφή αποθήκευσης του XGBoost. Περιέχει την αναπαράσταση του μοντέλου που φορτώνεται με load_model. Δεν είναι αρχείο SHAP, αρχείο predictions ή αντίγραφο ολόκληρης της Python μνήμης. Σύμφωνα με την επίσημη τεκμηρίωση XGBoost, η αποθήκευση μοντέλου διαφέρει από ένα μη σταθερό memory snapshot (XGBoost Developers, χ.χ.). Στο έργο η φυσική αναπαράσταση επιλέγεται ώστε το native booster να μπορεί να υπολογίζει συνεισφορές για νέες εισόδους.

Το manifest δημιουργείται από το Tree.py μέσω της build_explain_manifest του ml_shared. Περιλαμβάνει schema_version, model_format, model_file, transaction, target/unit, feature_count, δύο λίστες ονομάτων, categorical mappings, χρόνο, πλήθος γραμμών,

υπερπαραμέτρους και R^2. Με ενεργές αλληλεπιδράσεις περιλαμβάνει και το sampling metadata. Πρόκειται για περιγραφή του artifact και του συμβολαίου, όχι για το ίδιο το δέντρο.

Πηγή: Ptixiaki/Worker/Models/TreeCode/Tree.py, γραμμές 386-451. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
386 def stage_native_explainability_artifacts(
387     *,
388     model: xgb.XGBRegressor,
389     args: argparse.Namespace,
390     original_feature_names: list[str],
391     categorical_mappings: dict[str, dict[str, int]],
392     training_r2: float,
393     validation_r2: float,
394     row_count: int,
395     activation_id: str,
396 ) -> tuple[dict[str, Any], list[tuple[Path, Path]]]:
397     transaction = EXPLAINABLE_PHASE2_MODELS[args.item_type_subtype]
398     native_directory = Path(args.native_model_dir or args.output_onnx_dir)
399     native_directory.mkdir(parents=True, exist_ok=True)
400     model_filename, manifest_filename = explain_artifact_filenames(transaction)
401     model_path = native_directory / model_filename
402     manifest_path = native_directory / manifest_filename
403     staged_model_path = staging_path(model_path, activation_id)
404     staged_manifest_path = staging_path(manifest_path, activation_id)

406     booster = model.get_booster()
407     model_feature_names = [f"f{index}" for index in
    range(len(original_feature_names))]
408     validate_native_feature_contract(booster, model_feature_names)

410     try:
411         atomic_save_xgboost_model(booster, staged_model_path)
412         model_sha256 = sha256_file(staged_model_path)
413         trained_at = utc_now_iso()
414         manifest = build_explain_manifest(
415             transaction=transaction,
416             feature_names=original_feature_names,
417             encoded_feature_names=model_feature_names,
418             categorical_mappings=categorical_mappings,
419             target_name=args.target_col,
420             target_unit=args.target_unit.strip(),
421             trained_at=trained_at,
422             model_sha256=model_sha256,
423             row_count=row_count,
424             training_r2=training_r2,
425             validation_r2=validation_r2,
426             validation_fraction=args.validation_fraction,
427             hyperparameters=model_hyperparameters(args),
428         )
429         if args.emit_shap_records:
430             manifest["interaction_sampling"] = {
431                 "method": (
432                     "uniform_without_replacement"
433                     if row_count > args.interaction_sample_size
434                     else "full_population"
435                 ),
436                 "population_count": row_count,
437                 "sample_count": min(row_count, args.interaction_sample_size),
438                 "seed": args.interaction_sample_seed,
439             }
440         atomic_write_json(staged_manifest_path, manifest)
441     except BaseException:
442         for staged_path in (staged_model_path, staged_manifest_path):
443             try:
444                 staged_path.unlink()
445             except FileNotFoundError:
446                 pass
447         raise
```

```
448      return manifest, [
449          (staged_model_path, model_path),
450          (staged_manifest_path, manifest_path),
451      ]
```

Το SHA-256 είναι ντετερμινιστικό αποτύπωμα των bytes του UBJ. Δημιουργείται αρχικά μετά την αποθήκευση του staged μοντέλου και γράφεται στο manifest. Η Go το υπολογίζει ξανά από το τοπικό UBJ και συγκρίνει. Η Python το υπολογίζει ξανά κατά τη φόρτωση. Οι υπολογισμοί δεν δημιουργούν νέες εκδόσεις του μοντέλου: επαναλαμβάνουν τον ίδιο έλεγχο ταυτότητας. Αν τα bytes είναι ίδια, το αναμενόμενο αποτέλεσμα είναι ίδιο.

Πηγή: ml_shared/explain_artifacts.py, γραμμές 81-86. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
 81  def sha256_file(path: os.PathLike[str] | str) -> str:
 82      digest = hashlib.sha256()
 83      with open(path, "rb") as source:
 84          for chunk in iter(lambda: source.read(1024 * 1024), b""):
 85              digest.update(chunk)
 86      return digest.hexdigest()
```

Το hash δεν είναι ψηφιακή υπογραφή και δεν προστατεύει από πρόσωπο που μπορεί να αντικαταστήσει και το UBJ και το manifest. Επίσης το συγκεκριμένο hash αφορά το UBJ, όχι ολόκληρο το dataset ή το ONNX. Το manifest schema_version περιγράφει τη μορφή μεταδεδομένων. Δεν πρέπει να συγχέεται με την ταυτότητα ενός fit. Η ταυτότητα της τελευταίας native δημοσίευσης αναγνωρίζεται κυρίως από model_sha256 και trained_at.

7.9 Latest-only ενεργοποίηση αρχείων

Η Python γράφει προσωρινά αρχεία στον ίδιο φάκελο, με μοναδικό activation ID. Μόνο μετά τους ελέγχους και τον υπολογισμό SHAP αρχίζει η προώθηση στα τελικά paths. Τα προηγούμενα αρχεία μεταφέρονται προσωρινά σε backup names και αντικαθίστανται. Αν υπάρξει εξαίρεση μέσα στην ελεγχόμενη διαδικασία, αφαιρούνται οι νέες προωθήσεις και επανέρχονται τα backups. Με επιτυχία τα backups διαγράφονται, σύμφωνα με την επιλογή διατήρησης μόνο της τελευταίας έκδοσης.

Πηγή: Ptixiaki/Worker/Models/TreeCode/Tree.py, γραμμές 350-383. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
350  def activate_staged_artifacts(
351      artifacts: list[tuple[Path, Path]],
352      activation_id: str,
353  ) -> None:
354      """Promote one latest-only artifact set and restore the old set on error."""
```

```
356      backups: list[tuple[Path, Path]] = []
357      promoted: list[Path] = []
358      try:
359          for staged_path, final_path in artifacts:
360              backup_path = final_path.with_name(
361                  f".{final_path.stem}.{activation_id}.backup{final_path.suffix}"
362              )
363              if final_path.exists():
364                  os.replace(final_path, backup_path)
365                  backups.append((backup_path, final_path))
366              os.replace(staged_path, final_path)
367              promoted.append(final_path)
368      except BaseException:
369          for final_path in reversed(promoted):
370              try:
371                  final_path.unlink()
372              except FileNotFoundError:
373                  pass
374          for backup_path, final_path in reversed(backups):
375              if backup_path.exists():
376                  os.replace(backup_path, final_path)
```

```
377          raise
378      else:
379          for backup_path, _ in backups:
380              try:
381                  backup_path.unlink()
382              except FileNotFoundError:
383                  pass
```

Το os.replace είναι ατομικό για τη συγκεκριμένη αντικατάσταση αρχείου στο ίδιο filesystem, όχι για ολόκληρη τη λίστα αρχείων. Υπάρχει σύντομο μεταβατικό διάστημα και δεν έχει υλοποιηθεί πλήρες crash-recovery journal για διακοπή ρεύματος μεταξύ ενεργειών. Το manifest προωθείται τελευταίο ώστε ένας loader να απορρίψει ζεύγος που δεν ταιριάζει. Η συμπεριφορά είναι rollback-safe απέναντι σε ελεγχόμενα exceptions, όχι απόδειξη αδιαίρετης δημοσίευσης όλων των resources υπό κάθε αστοχία.

7.10 Reload: ποιος ειδοποιεί ποιον

Μετά τα δύο TrainJob, η Go προφορτώνει τα νέα ONNX sessions και τα ελέγχει. Έπειτα η LoadLocalModelHashes επαληθεύει UBJ και manifest και παράγει map sell/rent προς hash. Το αίτημα προς /ReloadExplainModels περιέχει μόνο τη λίστα transactions και τα expected_model_sha256. Δεν μεταφέρει δυαδικά μοντέλα μέσω HTTP. Ο Python server διαβάζει τα αρχεία από τις δικές του ρυθμισμένες διαδρομές στο κοινό filesystem.

Πηγή: Ptixiaki/main.go, γραμμές 358-406. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
358  func (a *application) trainResidenceTrees(ctx context.Context) error {
359      a.TrainingMu.Lock()
360      defer a.TrainingMu.Unlock()
```

```
362      trainer := tr.InitTrainingPipeline(a.Calculator.Db, ctx)
363      err := trainer.TrainJob("residence", "properties", "sell")
364      if err != nil {
365          return fmt.Errorf("sell training failed: %w", err)
366      }
367      err = trainer.TrainJob("residence", "properties", "rent")
368      if err != nil {
369          return fmt.Errorf("rent training failed: %w", err)
370      }
```

```
372      sellTree, rentTree, err := loadTreePair()
373      if err != nil {
374          return fmt.Errorf("validate newly trained ONNX models: %w", err)
375      }
376      expectedHashes, err := explain.LoadLocalModelHashes(
377          filepath.Dir(TreePaths[0]),
378          []string{"sell", "rent"},
379      )
380      if err != nil {
381          destroyPredictor(sellTree)
382          destroyPredictor(rentTree)
383          return fmt.Errorf("validate newly trained native model publications: %w",
      err)
384      }
```

```
386      reloadCtx, cancelReload := context.WithTimeout(ctx, explain.HTTPTimeout)
387      reloadResponse, err := explain.NewClient().ReloadModels(
388          reloadCtx,
389          []string{"sell", "rent"},
390          expectedHashes,
391      )
392      cancelReload()
393      if err != nil {
394          destroyPredictor(sellTree)
395          destroyPredictor(rentTree)
396          return fmt.Errorf("reload Python explanation models: %w", err)
397      }
```

```
399      a.publishTrees(sellTree, rentTree)
400      log.Printf(
401          "Go and Python sell/rent models reloaded successfully (sell=%s rent=%s)",
402          reloadResponse.Models[0].ModelSHA256,
403          reloadResponse.Models[1].ModelSHA256,
404      )
405      return nil
406  }
```

Οι υποψήφιοι φάκελοι προκύπτουν πρώτα από PTIXIAKI_MODEL_ROOT και μετά από σταθερές εναλλακτικές σχετικές με την υπηρεσία και το bundle. Αν βρεθεί πλήρες ζεύγος αρχείων, ελέγχονται manifest, hash, feature contract και smoke test. Το expected hash προστατεύει από σιωπηρή φόρτωση άλλου, έγκυρου αλλά παλιότερου αντιγράφου σε διαφορετικό φάκελο. Η σύγκριση δεν εξασφαλίζει από μόνη της ποιότητα μοντέλου· εξασφαλίζει συμφωνία στην ταυτότητα που ζήτησε η Go.

Πηγή: Ptixiaki-ml-microservice/main.py, γραμμές 159-189. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
159      def reload(
160          self,
161          transactions: Sequence[str],
162          expected_model_sha256: dict[str, str] | None = None,
163      ) -> list[dict[str, Any]]:
164          requested = list(transactions)
165          if not requested or len(set(requested)) != len(requested):
166              raise ValueError("transactions must be non-empty and unique")
```

```
170          replacements = {
171              transaction: self._find_and_load(transaction) for transaction in
     requested
172          }
173          if expected_model_sha256 is not None:
174              if set(expected_model_sha256) != set(requested):
175                  raise ValueError(
176                      "expected_model_sha256 keys must exactly match transactions"
177                  )
178              for transaction, registered in replacements.items():
179                  actual_hash = registered.artifact.manifest["model_sha256"]
180                  if expected_model_sha256[transaction] != actual_hash:
181                      raise ArtifactValidationError(
182                          f"{transaction} model hash does not match the Worker
     publication"
183                      )
184          with self._lock:
185              updated = dict(self._models)
186              updated.update(replacements)
187              self._models = updated
```

```
189          return [self._summary(transaction, replacements[transaction]) for
     transaction in requested]
```

Η registry κατασκευάζει πρώτα όλα τα replacements. Αν ένα αποτύχει, δεν αλλάζει το ενεργό map. Όταν είναι όλα έγκυρα, αποκτά lock και αντικαθιστά το map με ενημερωμένο αντίγραφο. Μετά επιστρέφει summaries με hash, χρόνο, πλήθος χαρακτηριστικών και artifact directory. Η Go ελέγχει την απάντηση και καλεί publishTrees. Δεν ξανακάνει training στη reload· μετατρέπει αποθηκευμένα artifacts σε ζωντανά αντικείμενα μνήμης.

Πηγή: Ptixiaki/main.go, γραμμές 520-546. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
520  func loadTreePair() (*t.ModelPredictor, *t.ModelPredictor, error) {
521      sellTree, err := loadTree(TreePaths[0], explain.FeatureCount)
522      if err != nil {
523          return nil, nil, fmt.Errorf("load sell tree: %w", err)
524      }
525      rentTree, err := loadTree(TreePaths[1], explain.FeatureCount)
526      if err != nil {
527          destroyPredictor(sellTree)
```

```
528           return nil, nil, fmt.Errorf("load rent tree: %w", err)
529       }
530       return sellTree, rentTree, nil
531   }
```

```
533   func (a *application) publishTrees(sellTree, rentTree *t.ModelPredictor) {
534       a.Mu.Lock()
535       oldSell := a.SellTree
536       oldRent := a.RentTree
537       a.SellTree = sellTree
538       a.RentTree = rentTree
```

```
543       destroyPredictor(oldSell)
544       destroyPredictor(oldRent)
545       a.Mu.Unlock()
546   }
```

Η Python αλλάζει πριν από τη Go. Αν η Python ολοκληρώσει αλλά χαθεί η HTTP απάντηση, η Go μπορεί να θεωρήσει το reload αποτυχημένο και να κρατήσει τους παλιούς predictors. Αυτό είναι πραγματικό όριο συντονισμού δύο διεργασιών. Η λειτουργική παραδοχή της εργασίας είναι ότι δεν εξυπηρετούνται κανονικές εκτιμήσεις κατά την εκπαίδευση. Για παραγωγική λειτουργία θα χρειαζόταν ενιαίο generation pointer ή σαφές πρωτόκολλο ενεργοποίησης και αποκατάστασης.

8. Backend, αιτήματα χρήστη και ανάλυση περιοχής

8.1 Authentication και όρια πρόσβασης

Οι διαδρομές /User και /analysis τυλίγονται με το υπάρχον AuthMiddleware. Η λογική διαβάζει JWT από cookie, επικυρώνει την υπογραφή και τοποθετεί το user_uid στο context. Ο handler δεν πρέπει να εμπιστεύεται UID χρήστη που θα έστελνε ο browser μέσα στο JSON. Το context λειτουργεί ως εσωτερική μεταφορά της ταυτότητας που έχει ελέγξει το middleware. Η αυθεντικοποίηση απαντά ποιος είναι ο χρήστης· ο έλεγχος ιδιοκτησίας απαντά τι δικαιούται να διαβάσει ή να αφαιρέσει.

Το SignIn ελέγχει αποθηκευμένο bcrypt hash. Το cookie έχει HttpOnly, Secure και SameSite ρυθμίσεις. Αυτό δεν αρκεί για παραγωγική ασφάλεια. Στο υπάρχον έργο υπάρχουν ενσωματωμένα μυστικά, signup-controlled admin flag και απροστάτευτες διαδρομές συντήρησης. Τα μυστικά δεν παρατίθενται στην εργασία. Πριν από οποιαδήποτε δημόσια έκθεση απαιτούνται αντικατάσταση και περιστροφή τους, περιβαλλοντική διαμόρφωση, ασφαλής σύνδεση βάσης και σαφής πολιτική εξουσιοδότησης.

Η public ως προς τους συνδεδεμένους χρήστες BrowseProperty επιτρέπει πρόσβαση σε καταχωρίσεις άλλων χρηστών από σχεδιασμό. Αντίθετα, η ViewProperty απαιτεί user_properties σύνδεση. Η διαφορά δεν πρέπει να αντιμετωπίζεται ως αυθαίρετη παράκαμψη εξουσιοδότησης. Παρ’ όλα αυτά, το τρέχον browse predicate δεν εφαρμόζει το is_published της σχέσης χρήστη. Επομένως η αποθήκευση false κατά την προσθήκη δεν εγγυάται απόκρυψη της καταχώρισης από την αγορά. Η πολιτική δημοσίευσης παραμένει ζήτημα προς οριστικοποίηση.

8.2 Αποκωδικοποίηση εισόδου και προαιρετικές τιμές

Το Add endpoint δέχεται POST, transaction sell ή rent και ένα JSON αντικείμενο έως 1 MiB. Η κλήση http.MaxBytesReader περιορίζει τα bytes που θα διαβάσει ο decoder. Το 1 << 20 είναι μετατόπιση bit και ισούται με 1.048.576. Δεν δηλώνει εκατομμύριο χαρακτηριστικά ή πλήθος γραμμών. Ο περιορισμός προστατεύει έναν μικρό λειτουργικό πόρο: το αίτημα περιγράφει μία κατοικία, όχι ένα αρχείο dataset.

Πηγή: Ptixiaki/Backend/Api/PropertyValidation.go, γραμμές 13-25. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
13  func DecodePropertyInput(reader io.Reader) (PropertyFeatures, error) {
14      var prop PropertyFeatures
15      decoder := json.NewDecoder(reader)
16      decoder.DisallowUnknownFields()
17      if err := decoder.Decode(&prop); err != nil {
18          return prop, err
19      }
20      var extra any
21      if err := decoder.Decode(&extra); err != io.EOF {
22          return prop, fmt.Errorf("request must contain exactly one JSON object")
23      }
24      return prop, ValidatePropertyInput(prop)
25  }
```

Η πρώτη Decode γεμίζει την PropertyFeatures. Το DisallowUnknownFields απορρίπτει άγνωστα ονόματα, ώστε ένα τυπογραφικό λάθος να μη χαθεί σιωπηρά. Η δεύτερη Decode ελέγχει ότι μετά το πρώτο αντικείμενο απομένει μόνο τέλος ροής. Το io.EOF εδώ είναι αναμενόμενη ένδειξη ολοκλήρωσης, όχι λειτουργική αποτυχία. Ένα δεύτερο JSON αντικείμενο δεν γίνεται δεκτό ως τμήμα του ίδιου αιτήματος.

Το AskingPricePerSqm είναι δείκτης σε float64. Αν το πεδίο λείπει ή είναι null, ο δείκτης παραμένει nil. Αν δοθεί αριθμός, υπάρχει τιμή που μπορεί να ελεγχθεί ως θετική και πεπερασμένη. Η χρήση δείκτη εκφράζει τρεις διαφορετικές περιπτώσεις που θα συγχέονταν σε ένα απλό float64: απουσία, μηδέν και θετικό ποσό. Η απουσία επιτρέπεται, το μηδέν δεν θεωρείται έγκυρη ζητούμενη τιμή.

8.3 Διεύθυνση, επιλογή provider και αποθήκευση

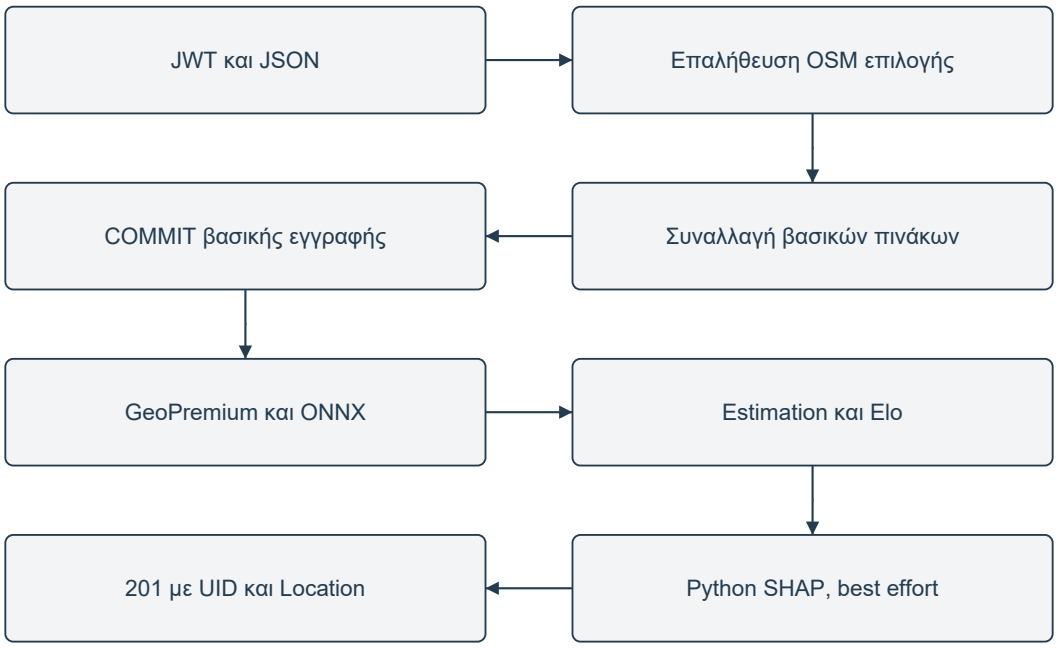
Η φόρμα δεν αποστέλλει απλώς ελεύθερο κείμενο που θα γεωκωδικοποιηθεί αυθαίρετα αργότερα. Ο χρήστης επιλέγει αποτέλεσμα Nominatim. Το αίτημα περιλαμβάνει AddressOSMType και AddressOSMID, μαζί με το canonical display name. Ο server επαληθεύει το επιλεγμένο αντικείμενο μέσω cache ή lookup. Οι caller-provided συντεταγμένες και παράγωγα πεδία μηδενίζονται πριν από τη δημιουργία, ώστε η χωρική πληροφορία να προέρχεται από τον επαληθευμένο provider.

Πηγή: Ptixiaki/Backend/Api/UserActions/AddPropertyHandler.go, γραμμές 17-65. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
17  func ServeAddProperty(w http.ResponseWriter, r *http.Request, create
    PropertyCreator) {
18      if r.Method != http.MethodPost {
19          w.Header().Set("Allow", http.MethodPost)
20          http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
21          return
22      }
23      userUID, ok := r.Context().Value("user_uid").(string)
24      if !ok || strings.TrimSpace(userUID) == "" {
25          http.Error(w, "authentication required", http.StatusUnauthorized)
26          return
27      }
28      transaction := r.URL.Query().Get("transaction")
29      if err := explain.ValidateTransaction(transaction); err != nil {
30          http.Error(w, err.Error(), http.StatusBadRequest)
31          return
32      }
33      prop, err := p.DecodePropertyInput(http.MaxBytesReader(w, r.Body, 1<<20))
34      if err != nil {
35          http.Error(w, err.Error(), http.StatusBadRequest)
36          return
37      }
38      if prop.AddressOSMID <= 0 || (prop.AddressOSMType != "node" &&
    prop.AddressOSMType != "way" && prop.AddressOSMType != "relation") {
39          propertyAddressError(w, "address_selection_required")
40          return
41      }
42      prop.Address = strings.TrimSpace(prop.Address)
43      prop.UID = ""
44      prop.ZoneValue = 0
45      prop.GeoLat = 0
46      prop.GeoLon = 0
47      prop.Distance = 0
48      prop.AdminName = ""
49      uid, err := create(r.Context(), userUID, prop, transaction)
50      if errors.Is(err, geo.ErrAddressSelection) {
51          propertyAddressError(w, "address_selection_invalid")
52          return
53      }
54      if err != nil {
55          log.Printf("failed to add property: %v", err)
56          http.Error(w, "could not finish adding property; check your portfolio
    before trying again", http.StatusInternalServerError)
57          return
58      }
59      w.Header().Set("Content-Type", "application/json")
60      w.Header().Set("Location", "/User/ViewProperty?uid="+uid)
61      w.WriteHeader(http.StatusCreated)
62      if err := json.NewEncoder(w).Encode(map[string]string{"message": "Property
    added successfully", "uid": uid}); err != nil {
63          log.Printf("failed to encode add-property response: %v", err)
64      }
65  }
```

Ο έλεγχος σταθερού OSM ID αποτρέπει τη χρήση μιας παλιάς επιλογής αφού ο χρήστης τροποποιήσει το κείμενο. Η απάντηση 422 με κωδικό address_selection_required ή address_selection_invalid επιτρέπει στη διεπαφή να ζητήσει νέα επιλογή. Μια νέα οδός δεν χρειάζεται να υπάρχει ήδη σε ακίνητα της βάσης για να επιλεγεί. Η τιμή ζώνης, ωστόσο, εξακολουθεί να λαμβάνεται από το πλησιέστερο υπάρχον ακίνητο χωρίς πλήρες μοντέλο επίσημων ζωνών· δεν παρουσιάζεται ως επίσημη αντικειμενική αξία.

Η InsertProperty εκτελεί σε μία συναλλαγή τις εισαγωγές βασικής εγγραφής, χαρακτηριστικών, κτιριακών στοιχείων, γεωγραφίας, κατοικίας και σχέσης χρήστη. Μετά κάνει commit. Ακολουθούν πρόβλεψη, estimation_data, Geo z-score και Elo μέσω του pool. Η βασική εγγραφή έχει επομένως ήδη αποθηκευτεί όταν αρχίζουν οι παράγωγοι υπολογισμοί. Αυτό εξηγεί γιατί μια αποτυχία στο δεύτερο μέρος δεν δικαιολογεί αυτόματο retry από τον browser.



Διάγραμμα 8.1. Προσθήκη κατοικίας και σημεία αποθήκευσης

Η τελική επιτυχής απάντηση είναι HTTP 201 με message και uid, καθώς και Location προς /User/ViewProperty?uid=.... Η παλιότερη απαίτηση για σκέτο μήνυμα έχει πλέον επεκταθεί ώστε η φόρμα να ανοίγει τη νέα καταχώριση. Δεν επιστρέφεται όλο το detail στο POST. Η λεπτομερής προβολή γίνεται με χωριστό read endpoint, διατηρώντας σαφές συμβόλαιο ανά λειτουργία.

8.4 Online εξήγηση της νέας κατοικίας

Μετά την πρόβλεψη η εφαρμογή κρατά το ακριβές διάνυσμα που χρησιμοποιήθηκε. Το ίδιο prediction.Features αποστέλλεται στην Python, μαζί με το transaction. Δεν αποστέλλεται η αρχική ακατέργαστη PropertyFeatures για να ξανακωδικοποιηθεί ανεξάρτητα. Η επιλογή μειώνει τον κίνδυνο διαφορετικών mappings ή ηλικιών σε δύο γλώσσες. Η Python επιστρέφει συνεισφορές της native αναπαράστασης και η Go ελέγχει συμφωνία με την ONNX πρόβλεψη.

Πηγή: Ptixiaki/Backend/Api/UserActions/DataBaseActions.go, γραμμές 268-287. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
269      explanationCtx, cancelExplanation := context.WithTimeout(ctx,
     explain.HTTPTimeout)
270      defer cancelExplanation()
271      if err := explain.ExplainAndPersist(
272          explanationCtx,
273          db,
274          explain.NewClient(),
```

```
275          PropertyUid,
276          transaction,
277          prediction.Features,
278          EstimatedPrice,
279      ); err != nil {
280          log.Printf("property %s was added successfully but its %s explanation was
      not persisted: %v", PropertyUid, transaction, err)
281      } else {
282          log.Printf("Success UPSERT property explanation for uid=%s
     transaction=%s", PropertyUid, transaction)
283      }
```

```
285      log.Println("Property inserted completely and successfully with UID:",
     PropertyUid)
286      return PropertyUid, result
287  }
```

Η αποτυχία SHAP είναι best effort: καταγράφεται αλλά δεν αναιρεί την ήδη επιτυχημένη εισαγωγή και πρόβλεψη. Το detail μπορεί να επιστρέψει explanation:null. Αυτό είναι προτιμότερο από την επινόηση μηδενικών συνεισφορών, αλλά απαιτεί σαφή εμφάνιση μη διαθεσιμότητας. Η ανάγνωση detail δεν καλεί εκ νέου την Python. Αν λείπει εξήγηση, δεν δημιουργείται σιωπηρά κατά το GET.

Υπάρχει ειδική εκκρεμότητα για ακίνητα χωρίς asking. Μπορούν να έχουν online explanation, αλλά αποκλείονται από supervised training. Η επόμενη πλήρης αντικατάσταση properties_analysis μπορεί να αφαιρέσει την προηγούμενη online εγγραφή τους, επειδή δεν περιλαμβάνονται στο νέο training population. Η παρατήρηση δείχνει γιατί «τελευταίο μοντέλο» και «εξήγηση κάθε αποθηκευμένου ακινήτου» δεν είναι ταυτόσημες εγγυήσεις.

8.5 Browse: φίλτρα, ορθογώνιο και pagination

Το Browse endpoint δέχεται προαιρετικά τα τέσσερα όρια γεωγραφικού ορθογωνίου. Αν λείπουν όλα, χρησιμοποιούνται παγκόσμια όρια και εξακολουθεί να απαιτείται γεωγραφική εγγραφή. Αν δοθεί μόνο μέρος τους, επιστρέφεται σφάλμα. Η SQL χρησιμοποιεί ST_MakeEnvelope σε SRID 4326 και τον τελεστή &&. Πρόκειται για ορθογωνική επιλογή, όχι ακριβή ένταξη στο πολύγωνο του δήμου. Ένα ακίνητο κοντά σε γωνία του bbox μπορεί να ανήκει σε γειτονικό διοικητικό όριο.

Τα φίλτρα κατασκευάζονται από εσωτερική επιτρεπόμενη λίστα στηλών. Οι τιμές τοποθετούνται ως parameters και όχι ως ανεξέλεγκτο SQL κείμενο. Το API υποστηρίζει construction_year, renovation_year, estimated_price_per_sqm, elo, asking_price_per_sqm και square_meters, με min/max. Η διεπαφή εκθέτει το απλούστερο υποσύνολο που ζητήθηκε: περιοχή, transaction, asking και εμβαδόν. Οι συνθήκες συνδυάζονται με AND και είναι συμπεριληπτικές στα άκρα.

Πηγή: Ptixiaki/Backend/Api/UserActions/BrowseProperties.go, γραμμές 91-131. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
 91      args := []any{bounds[0], bounds[1], bounds[2], bounds[3], transaction}
 92      query := `SELECT p.uid, ed.predicted_price_per_sqm, ` + browseAskingPrice +
     `,
 93      b.square_meters, geo.address_text, e.elo, ed.geo_premium,
 94      bd.construction_year, bd.renovation_year
 95      FROM properties p
 96      JOIN property_geo_data geo ON geo.property_uid = p.uid
 97      LEFT JOIN base_attributes b ON b.property_uid = p.uid
 98      LEFT JOIN building_details bd ON bd.property_uid = p.uid
 99      LEFT JOIN estimation_data ed ON ed.uid = p.uid
100      LEFT JOIN properties_elo e ON e.property_uid = p.uid
101      WHERE ` + browseResidencePredicate + `
102      AND geo.geom && ST_MakeEnvelope($1, $2, $3, $4, 4326)
103      AND p.transaction_type = $5`
```

```
105      for _, filter := range []struct {
106          name, column string
107          integer      bool
108      }{
```

```
109         {"construction_year", "bd.construction_year", true},
110         {"renovation_year", "bd.renovation_year", true},
111         {"estimated_price_per_sqm", "ed.predicted_price_per_sqm", false},
112         {"elo", "e.elo", true},
113         {"asking_price_per_sqm", browseAskingPrice, false},
114         {"square_meters", "b.square_meters", true},
115     } {
116         for _, bound := range []struct{ suffix, operator string }{{"min", ">="},
    {"max", "<="}} {
117             name := filter.name + "_" + bound.suffix
118             if raw := values.Get(name); raw != "" {
119                 value, err := strconv.ParseFloat(raw, 64)
120                 if err != nil || math.IsNaN(value) || math.IsInf(value, 0) ||
    (filter.integer && value != math.Trunc(value)) {
121                     return "", nil, 0, fmt.Errorf("invalid %s", name)
122                 }
123                 args = append(args, value)
124                 query += fmt.Sprintf("\n AND %s %s $%d::double precision",
    filter.column, bound.operator, len(args))
125             }
126         }
127     }
128     args = append(args, (index-1)*BrowsePageSize)
129     query += fmt.Sprintf("\n ORDER BY e.elo DESC NULLS LAST, p.uid ASC LIMIT 21
    OFFSET $%d", len(args))
130     return query, args, index, nil
131 }
```

Η σελίδα είναι one-based. Το offset υπολογίζεται ως (index - 1) × 20. Η SQL ζητά 21 εγγραφές, ώστε η επιπλέον να καθορίζει has_next χωρίς COUNT ολόκληρου του αποτελέσματος. Οι χρήστες λαμβάνουν έως είκοσι. Η ταξινόμηση είναι κατά Elo φθίνουσα, NULLS LAST και UID για ισοβαθμίες. Χωρίς τον δεύτερο όρο, τα ισόβαθμα ακίνητα θα μπορούσαν να αλλάζουν θέση ανά query.

Η offset pagination δεν εγγυάται σταθερό snapshot ανάμεσα σε δύο διαφορετικά HTTP αιτήματα. Αν αλλάξουν βαθμολογίες ή προστεθούν εγγραφές ανάμεσα στη σελίδα 1 και 2, μπορεί να μετακινηθούν αποτελέσματα. Η υλοποίηση είναι επαρκώς απλή για το ζητούμενο περιβάλλον, αλλά μια μελλοντική αγορά υψηλής μεταβλητότητας θα μπορούσε να χρησιμοποιήσει keyset pagination ή ρητό snapshot. Δεν απαιτείται να προστεθεί τέτοια πολυπλοκότητα για να περιγραφεί έντιμα ο σημερινός περιορισμός.

8.6 Portfolio, εικόνες και αναγνωστικό snapshot

Το Portfolio επιστρέφει property_count, total_estimated_value και λίστα περιλήψεων. Η συνολική τιμή προκύπτει μόνο όπου υπάρχουν και estimate ανά τ.μ. και εμβαδόν. Η απουσία εκτίμησης δεν γίνεται μηδενική αξία στην κάρτα, αλλά η συγκεκριμένη εγγραφή δεν συμβάλλει στο άθροισμα. Επομένως το συνολικό ποσό αφορά το εκτιμημένο διαθέσιμο υποσύνολο. Η κοινή σύνοψη δεν περιέχει transaction ανά ακίνητο, κάτι που απαιτεί προσοχή αν ένα χαρτοφυλάκιο αναμειγνύει πωλήσεις και μισθώσεις.

Πηγή: Ptixiaki/Backend/Api/UserActions/Portfolio.go, γραμμές 16-31. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
16  type PortfolioPropertyOverview struct {
17      UID                 string   `json:"uid"`
18      EstimatedPricePerSqm *float64 `json:"estimated_price_per_sqm"`
19      AskingPricePerSqm    *float64 `json:"asking_price_per_sqm"`
20      SquareMeters         *int     `json:"square_meters"`
21      Address              *string  `json:"address"`
22      Image                *string  `json:"image,omitempty"`
23      Elo                  *int     `json:"elo"`
24      GeoPremium           *float64 `json:"geo_premium"`
25  }
```

```
27  type PortfolioResponse struct {
```

```
28      PropertyCount       int                          `json:"property_count"`
29      TotalEstimatedValue float64
    `json:"total_estimated_value"`
30      Properties          []PortfolioPropertyOverview `json:"properties"`
31  }
```

Οι εικόνες ανακτώνται ομαδικά για τα UID της σελίδας ή του χαρτοφυλακίου. Μια URL επιστρέφεται ως URL, ενώ αποθηκευμένα bytes μετατρέπονται σε data URL με κατάλληλο MIME type. Δεν υπάρχει εδώ upload φωτογραφιών. Στο Browse η απουσία είναι ρητό image:null. Σε παλαιότερα owner contracts το πεδίο παραλείπεται μέσω omitempty. Το frontend δέχεται και τις δύο μορφές απουσίας και εμφανίζει το κοινό SVG fallback.

Πηγή: Ptixiaki/Backend/Api/UserActions/PropertyDetails.go, γραμμές 24-62. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
24  type PropertyDetails struct {
25      UID                 string                    `json:"uid"`
26      Transaction         string                    `json:"transaction"`
27      Features            p.PropertyFeatures        `json:"features"`
28      EstimatedPricePerSqm *float64
    `json:"estimated_price_per_sqm"`
29      EstimatedTotalPrice  *float64                 `json:"estimated_total_price"`
30      AskingPricePerSqm    *float64
    `json:"asking_price_per_sqm,omitempty"`
31      AskingTotalPrice     *float64
    `json:"asking_total_price,omitempty"`
32      Elo                 *int                      `json:"elo"`
33      GeoPremium          *float64                  `json:"geo_premium"`
34      GeoPremiumZScore    *float64                  `json:"geo_premium_zscore"`
35      ResidualPercentage  *float64                  `json:"residual_percentage"`
36      Image               *string                   `json:"image,omitempty"`
37      Explanation         *PropertyExplanation      `json:"explanation"`
38      FeatureImportance   []GlobalFeatureImportance `json:"feature_importance"`
39  }
40  type PropertyExplanation struct {
41      BaselinePricePerSqm       float64
    `json:"baseline_price_per_sqm"`
42      ExplainedPredictionPerSqm float64
    `json:"explained_prediction_per_sqm"`
43      AdditivityError           float64               `json:"additivity_error"`
44      ModelSHA256               string                `json:"model_sha256"`
45      ExplainedAt               time.Time             `json:"explained_at"`
46      MatchesImportanceModel    *bool
    `json:"matches_importance_model"`
47      Contributions             []PropertyContribution `json:"contributions"`
48  }
49  type PropertyContribution struct {
50      FeatureName               string  `json:"feature_name"`
51      ImpactPerSqm              float64 `json:"impact_per_sqm"`
52      LocalImportancePercentage float64 `json:"local_importance_percentage"`
53  }
54  type GlobalFeatureImportance struct {
55      FeatureName          string    `json:"feature_name"`
56      MeanAbsSHAPPerSqm    float64   `json:"mean_abs_shap_per_sqm"`
57      MeanSignedSHAPPerSqm float64   `json:"mean_signed_shap_per_sqm"`
58      ImportancePercentage float64   `json:"importance_percentage"`
59      Rank                 int       `json:"rank"`
60      ModelSHA256          string    `json:"model_sha256"`
61      CalculatedAt         time.Time `json:"calculated_at"`
62  }
```

Το detail περιλαμβάνει features, estimate ανά τ.μ. και σύνολο, optional asking, Elo, GeoPremium, residual, image, explanation και feature_importance. Η τοπική σημασία βρίσκεται μέσα στις contributions, ενώ η καθολική σημασία είναι ξεχωριστός πίνακας. Το matches_importance_model δείχνει αν τα hashes εξήγησης και importance συμφωνούν. Δεν πιστοποιεί ότι κάθε άλλο πεδίο της βάσης έχει ίδια ηλικία, αλλά βοηθά να εντοπιστεί

συγκεκριμένη αναντιστοιχία.

Η readPropertyDetails χρησιμοποιεί read-only Repeatable Read συναλλαγή. Σύμφωνα με τη συμπεριφορά PostgreSQL αυτού του επιπέδου απομόνωσης, οι αναγνώσεις της ίδιας συναλλαγής βλέπουν συνεπές snapshot (PostgreSQL Global Development Group, χ.χ.). Αυτό αποτρέπει το ενδιάμεσο πέρασμα σε νεότερες εγγραφές importance ενώ έχουν ήδη διαβαστεί παλιές contributions. Δεν κλειδώνει ολόκληρη τη βάση ούτε συντονίζει τη μνήμη των μοντέλων σε άλλες διεργασίες.

Πηγή: Ptixiaki/Backend/Api/UserActions/PropertyDetails.go, γραμμές 66-93. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
66  func (u *User) ViewProperty(db PropertyReader, ctx context.Context, uid string)
    (PropertyDetails, error) {
67      if strings.TrimSpace(uid) == "" || strings.TrimSpace(u.Uid) == "" {
68          return PropertyDetails{}, pgx.ErrNoRows
69      }
70      return readPropertyDetails(db, ctx, propertyDetailsQuery, uid, u.Uid)
71  }
```

```
74  func ViewBrowseProperty(db PropertyReader, ctx context.Context, uid string)
    (PropertyDetails, error) {
75      return readPropertyDetails(db, ctx, browsePropertyDetailsQuery, uid)
76  }
```

```
78  func readPropertyDetails(db PropertyReader, ctx context.Context, query string,
    args ...any) (PropertyDetails, error) {
79      tx, err := db.BeginTx(ctx, pgx.TxOptions{IsoLevel: pgx.RepeatableRead,
    AccessMode: pgx.ReadOnly})
80      if err != nil {
81          return PropertyDetails{}, err
82      }
83      defer func() { _ = tx.Rollback(context.Background()) }()
84      detail, err := loadPropertyDetails(ctx, tx, query, args...)
85      if err != nil {
86          return PropertyDetails{}, err
87      }
88      if err := tx.Commit(ctx); err != nil {
89          return PropertyDetails{}, err
90      }
91      return detail, nil
92  }
```

8.7 Περιοχή, autocomplete και Nominatim

Η αναζήτηση διοικητικής περιοχής χρησιμοποιεί το application-scoped AreaCatalog. Διαβάζει admin_boundaries, κανονικοποιεί ελληνικούς τόνους και υποστηρίζει επιλεγμένες μεταγραφές και aliases. Ο κατάλογος επαναχρησιμοποιείται για μία ώρα. Η απαίτηση autocomplete δύο δευτερολέπτων ικανοποιείται με αναζήτηση στον κατάλογο αυτόν, όχι με επαναλαμβανόμενες κλήσεις στο δημόσιο Nominatim.

Η διεύθυνση χρησιμοποιεί διαφορετικό mode του ίδιου frontend component. Η επίσημη πολιτική Nominatim απαγορεύει autocomplete και θέτει απόλυτο μέγιστο ένα αίτημα ανά δευτερόλεπτο για τη δημόσια υπηρεσία (OpenStreetMap Foundation, χ.χ.). Η υλοποίηση απαιτεί Enter ή Find address, χρησιμοποιεί User-Agent, cache και δυνατότητα αλλαγής base URL με NOMINATIM_URL. Ο περιορισμός είναι ανά εφαρμογή, όχι ανά ξεχωριστό χρήστη, και η δημόσια υπηρεσία δεν πρέπει να δέχεται εμπιστευτικές πληροφορίες.

Ο NominatimClient κρατά mutex κατά τον έλεγχο cache και την εξωτερική κλήση. Το χρονικό διάστημα υπολογίζεται από lastRequest. Το timer μπορεί να διακοπεί από context cancellation. Η cache είναι περιορισμένη σε 256 entries με διάρκεια μίας ώρας· η απομάκρυνση όταν γεμίσει δεν είναι υλοποίηση LRU. Στην επιτυχή search αποθηκεύονται και lookup keys του OSM αντικειμένου, ώστε η αμέσως επόμενη προσθήκη συνήθως να επαληθεύει την ίδια επιλογή χωρίς δεύτερη εξωτερική κλήση.

8.8 Ανάλυση περιοχής και προσωρινά μοντέλα

Το CreateAnalysisForArea δέχεται αριθμητικό BoundingBox με MinLat, MaxLat, MinLon και MaxLon. Δεν δέχεται μόνο το λεκτικό όνομα. Το frontend έχει ήδη μετατρέψει την επιλεγμένη πρόταση στα όρια. Η Go φορτώνει κατάλληλα ακίνητα, τυποποιεί τέσσερα γνωρίσματα και εκτελεί k-means. Εξετάζονται υποψήφια K έως 10, με elbow κριτήριο, διακριτά farthest-point αρχικά κέντρα και όριο εκατό επαναλήψεων. Ίδιες γραμμές δεν πρέπει να προκαλούν ατέρμονη προσπάθεια παραγωγής διαφορετικών κέντρων.

Πηγή: Ptixiaki/Backend/Api/MarketAnalysis/AnalysisPipeline.go, γραμμές 36-73. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
36  func (ap *AnalysisPipeline) RunPipeline(bbox BoundingBox) ([]DataPoint,
    []Cluster, *MLResponse, *MLResponse, error) {
37      ap.Warnings = nil
38      log.Printf("[ANALYSIS] Loading properties inside selected area")
39      Properties, err := ap.GetAreaDataPoints(ap.Db, ap.Ctx, bbox)
40      if err != nil {
41          return nil, nil, nil, nil, fmt.Errorf("load area properties: %w", err)
42      }
43      log.Printf("[ANALYSIS] Found %d properties; calculating clusters",
    len(Properties))
44      if len(Properties) == 0 {
45          return []DataPoint{}, []Cluster{}, nil, nil, nil
46      }

48      clustersData := CreateClusters(Properties, 10)
49      clusters := CalculateClusterStats(clustersData)
50      Points, finalClusters := EnrichDataAndFindExtremes(clusters, clustersData)

52      var uids []string
53      for _, p := range Points {
54          uids = append(uids, p.Uid)
55      }

57      ModeledProperties, err := ap.GetDataForMicroService(uids)
58      if err != nil {
59          return nil, nil, nil, nil, fmt.Errorf("prepare model properties: %w",
    err)
60      }

62      log.Printf("[ANALYSIS] Prepared %d model rows", len(ModeledProperties))

64      r1, r2, err := ap.TrainSeparateModels(ModeledProperties)
65      if err != nil {
66          return nil, nil, nil, nil, err
67      }

69      PrintMLReport("SALES", r1)
70      PrintMLReport("RENTALS", r2)

72      return Points, finalClusters, r1, r2, nil
73  }
```

Μετά τα clusters, τα δεδομένα χωρίζονται σε sell και rent για περιγραφικά στατιστικά και αναφορές. Οι ονομασίες Top5 και Bottom5 αναφέρονται σε άκρα περίπου 5% των ομάδων και όχι σε σταθερά πέντε ακίνητα. Τα αποτελέσματα περιλαμβάνουν μέσους, τυπικές αποκλίσεις και z-scores. Οι ομάδες είναι χαρακτηριστικών κατοικίας και όχι γεωγραφικά πολύγωνα, ενώ δεν παρέχονται συμπληρωμένες συντεταγμένες ανά σημείο για πραγματικό χάρτη clusters.

Για κάθε αγορά με τουλάχιστον δέκα modeled rows η Go καλεί /TrainAndExplain. Η Python δημιουργεί νέο XGBRegressor, με native categorical υποστήριξη και StandardScaler για αριθμητικές στήλες, τον εκπαιδεύει και υπολογίζει in-sample R^2. Παράγει feature_importances_, SHAP για τις γραμμές, τις πέντε ισχυρότερες αλληλεπιδράσεις και εξηγήσεις των πρώτων δέκα γραμμών. Το προσωρινό δέντρο δεν αποθηκεύεται ως το καθολικό UBJ και δεν αντικαθιστά τη registry.

Πηγή: Ptixiaki-ml-microservice/main.py, γραμμές 420-465. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
420  def train_and_explain(payload: TrainRequest):
421      if len(payload.data) < 10:
422          raise HTTPException(status_code=400, detail="Dataset must have at least
     10 points.")
424      df = pd.DataFrame(payload.data)
425      uids = df["UID"].tolist() if "UID" in df.columns else [str(i) for i in
     range(len(df))]
426      df = df.drop(columns=payload.drop_columns, errors="ignore")
428      y = df[payload.target_column]
429      X = df.drop(columns=[payload.target_column], errors="ignore")
431      for column in payload.categorical_columns:
432          if column in X.columns:
433              X[column] = X[column].astype("category")
435      numerical_columns = X.select_dtypes(include=["int64", "float64"]).columns
436      scaler = StandardScaler()
437      if len(numerical_columns) > 0:
438          X[numerical_columns] = scaler.fit_transform(X[numerical_columns])
440      model = xgb.XGBRegressor(
441          max_depth=payload.max_depth,
442          min_child_weight=payload.min_child_weight,
443          learning_rate=payload.learning_rate,
444          n_estimators=payload.n_estimators,
445          enable_categorical=True,
446          tree_method="hist",
447      )
448      model.fit(X, y)
450      predictions = model.predict(X)
451      r2 = float(r2_score(y, predictions))
453      feature_importances = pd.DataFrame(
454          {
455              "feature": X.columns,
456              "importance": float(0),
457          }
458      )
459      feature_importances["importance"] = model.feature_importances_
460      feature_importances = feature_importances.sort_values(
461          "importance", ascending=False
462      ).to_dict(orient="records")
464      explainer = shap.TreeExplainer(model)
465      shap_values = explainer.shap_values(X)
```

Η importance της τοπικής αναφοράς προέρχεται από το model.feature_importances_ του XGBoost και δεν είναι το ίδιο μέγεθος με το full-population mean-absolute SHAP των καθολικών πινάκων. Οι αλληλεπιδράσεις της περιοχής υπολογίζονται στο σύνολο των επιλεγμένων γραμμών, όχι με το όριο 2.000 του Worker. Η λεπτομέρεια εξηγεί γιατί μεγάλη περιοχή μπορεί να παραμένει ακριβή παρότι έχει βελτιστοποιηθεί η καθολική εκπαίδευση.

Η αποτυχία ενός επιμέρους μοντέλου δεν αφαιρεί υποχρεωτικά όλη την ανάλυση. Οι warnings επιτρέπουν να επιστραφούν clusters και η διαθέσιμη άλλη αγορά. Άδεια περιοχή επιστρέφει κενές λίστες χωρίς άσκοπη κλήση Python. Η Go χρησιμοποιεί request context και timeout τεσσάρων λεπτών, ενώ ο browser περιμένει έως πέντε λεπτά. Η ακύρωση του HTTP δεν εγγυάται ότι διακόπηκε ένας ήδη εκτελούμενος CPU υπολογισμός Python.

9. Διεπαφή React και διαδρομές χρήσης

9.1 Κατάσταση εφαρμογής και πλοήγηση

Η διεπαφή χρησιμοποιεί React και TypeScript με Vite. Δεν υπάρχει πρόσθετος router ή καθολικό state store. Η App διατηρεί αν εμφανίζεται SignIn ή SignUp και αν έχει ολοκληρωθεί η σύνδεση στη συγκεκριμένη μνήμη του browser. Η Workspace παρακολουθεί το location.hash με useSyncExternalStore και επιλέγει Browse, Portfolio, Analysis ή λεπτομέρειες. Το πλεονέκτημα είναι μικρή εξάρτηση και άμεση αντιστοίχιση διαδρομών σε components.

Πηγή: Ptixiaki-frontend/Ptixiaki-Frontend/src/components/Workspace.tsx, γραμμές 15-36. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
16  function subscribeToNavigation(onChange: () => void) {
17    window.addEventListener('hashchange', onChange)
18    return () => window.removeEventListener('hashchange', onChange)
19  }
```

```
21  export default function Workspace({ onSignIn = () => {} }: { onSignIn?: () =>
    void }) {
22    const hash = useSyncExternalStore(subscribeToNavigation, () =>
    window.location.hash)
23    const [path, query = ''] = hash.slice(1).split('?')
24    const isAddProperty = path === '/portfolio/add'
25    const isPortfolio = path === '/portfolio' || path === '/portfolio/property' ||
    isAddProperty
26    const isProperty = path === '/portfolio/property' || path ===
    '/browse/property'
27    const active = isPortfolio ? 'Portfolio' : path === '/analysis' ? 'Analysis' :
    'Browse'
28    const parameters = new URLSearchParams(query)
29    const uid = isProperty ? parameters.get('uid') : null
30    const pageTitle = uid ? 'Property details' : isAddProperty ? 'Add property' :
    active
31    parameters.delete('uid')
32    const browseBackHref = parameters.size ? `#/browse?${parameters}` : '#/browse'
33    const [open, setOpen] = useState(false)
34    const toggle = useRef<HTMLButtonElement>(null)
35    const main = useRef<HTMLElement>(null)
```

Η κατάσταση signedIn δεν αποτελεί έλεγχο ασφαλείας. Η Go παραμένει υπεύθυνη για το cookie. Με refresh η React μπορεί να επιστρέψει στη φόρμα σύνδεσης παρότι υπάρχει ακόμη cookie, επειδή δεν έχει υλοποιηθεί πλήρης ανάκτηση session κατά την αρχικοποίηση. Αν ένα αίτημα επιστρέψει 401, εμφανίζεται δυνατότητα νέας σύνδεσης. Δεν υπάρχει ολοκληρωμένος server-side μηχανισμός logout/revocation στο εξεταζόμενο προϊόν.

Το αριστερό μενού ανοίγει με burger button. Υπάρχουν aria-expanded, aria-controls, χειρισμός Escape και μεταφορά focus. Όταν το μενού είναι ανοικτό, το κύριο περιεχόμενο γίνεται inert. Αυτές είναι συγκεκριμένες πρόνοιες προσβασιμότητας, όχι ισχυρισμός πλήρους πιστοποίησης WCAG. Η συνολική αξιολόγηση προσβασιμότητας θα απαιτούσε ξεχωριστό έλεγχο πληκτρολογίου, screen reader, αντίθεσης και πολλών μεγεθών οθόνης.

9.2 Ένα συμβόλαιο δεδομένων για κάρτες και λεπτομέρειες

Οι TypeScript τύποι PropertyOverview, PropertyDetailData, Contribution και GlobalImportance περιγράφουν τα δεδομένα που περιμένει η διεπαφή. Τα nullable πεδία αποτρέπουν το να αντιμετωπίζεται η έλλειψη ως αριθμητικό μηδέν. Οι μορφοποιητές money, number και signed μετατρέπουν τα διαθέσιμα δεδομένα σε αναγνώσιμα ποσά και πρόσημα. Το UID παραμένει το σταθερό αναγνωριστικό πλοήγησης, όχι το εμφανιζόμενο address string.

Πηγή: Ptixiaki-frontend/Ptixiaki-Frontend/src/property.ts, γραμμές 1-59. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
 1  export type PropertyOverview = {
 2    uid: string
 3    address: string | null
 4    square_meters: number | null
 5    estimated_price_per_sqm: number | null
 6    asking_price_per_sqm?: number | null
 7    elo: number | null
 8    geo_premium: number | null
 9    image?: string | null
10  }
12  export type PortfolioData = {
13    property_count: number
14    total_estimated_value: number
15    properties: PropertyOverview[]
16  }
18  export type PropertyFeatures = {
19    UID: string
20    Address: string
21    SubType: string
22    SquareMeters: number
23    [name: string]: string | number | number[] | null | undefined
24  }
26  export type Contribution = {
27    feature_name: string
28    impact_per_sqm: number
29    local_importance_percentage: number
30  }
32  export type GlobalImportance = {
33    feature_name: string
34    mean_abs_shap_per_sqm: number
35    mean_signed_shap_per_sqm: number
36    importance_percentage: number
37    rank: number
38    model_sha256: string
39    calculated_at: string
40  }
42  export type PropertyDetailData = Omit<PropertyOverview, 'address' |
    'square_meters'> & {
43    transaction: string
44    features: PropertyFeatures
45    estimated_total_price: number | null
46    asking_total_price?: number | null
47    geo_premium_zscore: number | null
48    residual_percentage: number | null
49    explanation: {
50      baseline_price_per_sqm: number
51      explained_prediction_per_sqm: number
52      additivity_error: number
53      model_sha256: string
54      explained_at: string
55      matches_importance_model: boolean | null
56      contributions: Contribution[]
57    } | null
58    feature_importance: GlobalImportance[]
59  }
```

Η TypeScript περιγραφή δεν επικυρώνει από μόνη της το JSON κατά την εκτέλεση. Το as T δηλώνει στον compiler την αναμενόμενη μορφή· δεν επιθεωρεί κάθε πεδίο. Οι έλεγχοι server και οι συμβατικές δοκιμές είναι επομένως κρίσιμοι. Ένας μελλοντικός schema validator θα μπορούσε να προσθέσει runtime προστασία, αλλά δεν παρουσιάζεται ως ήδη εγκατεστημένος.

9.3 Autocomplete με αποφυγή παλιών απαντήσεων

Το AreaSearch χρησιμοποιεί useEffect που εξαρτάται από query, selected, disabled και mode. Στο area mode προγραμματίζει αίτημα μετά από 2.000 ms. Αν ο χρήστης πληκτρολογήσει ξανά, το cleanup ακυρώνει τον timer και τον προηγούμενο AbortController. Έτσι η αναμονή μετρά από το τελευταίο πλήκτρο. Μια παλιά αργή απάντηση δεν πρέπει να αντικαταστήσει τις προτάσεις μιας νεότερης αναζήτησης.

Πηγή: Ptixiaki-frontend/Ptixiaki-Frontend/src/components/AreaSearch.tsx, γραμμές 29-68. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
29    useEffect(() => {
30      if (query.trim().length < 2 || selected || disabled || (isAddress &&
    searchVersion === 0)) {
31        return
32      }

34      const controller = new AbortController()
35      const timer = setTimeout(async () => {
36        setSearching(true)
37        setError(null)
38        setStatus(isAddress ? 'Finding addresses…' : 'Finding areas…')

40        try {
41          const url =
    `/analysis/autocomplete?q=${encodeURIComponent(query.trim())}${isAddress ?
    '&kind=address' : ''}`
42          const suggestions = await getPropertyData<AreaSuggestion[] | null>(url,
    controller.signal)

44          if (!controller.signal.aborted) {
45            setOptions(suggestions ?? [])
46            setStatus(suggestions?.length
47              ? `Choose an ${noun} from the suggestions.`
48              : isAddress
49                ? 'No matching addresses found. Include the city or try the Greek
    street name.'
50                : 'No matching areas found. Try its Greek name or a larger
    municipality.')
51          }
52        } catch (error) {
53          if (!controller.signal.aborted) {
54            setError(error as PropertyRequestError)
55            setStatus('')
56          }
57        } finally {
58          if (!controller.signal.aborted) {
59            setSearching(false)
60          }
61        }
62      }, isAddress ? 0 : 2000)

64      return () => {
65        clearTimeout(timer)
66        controller.abort()
67      }
68    }, [query, selected, disabled, isAddress, noun, searchVersion])
```

Η πληκτρολόγηση και η επιλογή είναι διαφορετικές καταστάσεις. Το changeQuery καθαρίζει την προηγούμενη επιλογή. Το choose αποθηκεύει ολόκληρη την πρόταση με bbox ή OSM στοιχεία. Αυτό αποτρέπει την αποστολή ορίων Πειραιά ενώ το πεδίο έχει αλλάξει σε άλλη περιοχή. Το ίδιο component υποστηρίζει πλήκτρα επάνω/κάτω, Enter και Escape, αλλά το address mode απαιτεί ρητή ενεργοποίηση αναζήτησης αντί για debounce κλήσεων στον δημόσιο provider.

Η προηγούμενη αποτυχία όπου το όνομα Pireaus αντιστοιχούσε σε μικρό τοπικό αντικείμενο άλλης πόλης δείχνει γιατί ένα λεκτικό αποτέλεσμα δεν αρκεί. Ο κατάλογος διοικητικών ορίων και η προτεραιότητα δήμων μειώνουν τέτοια σφάλματα για την επιλογή περιοχής. Η κάλυψη παραμένει περιορισμένη σε όσα όρια υπάρχουν στη βάση και στα υποστηριζόμενα aliases. Δεν

υπάρχει γενική εγγύηση κατανόησης κάθε ανορθογραφίας ή αγγλικής μετάφρασης.

9.4 Browse και διατήρηση φίλτρων

Το Browse διαβάζει αρχικές τιμές από URLSearchParams. Η φόρμα διατηρεί προσωρινές εισόδους μέχρι Apply. Η εφαρμογή επαναφέρει index=1 όταν αλλάζουν φίλτρα, επειδή η παλιά σελίδα μπορεί να μην υπάρχει στον νέο πληθυσμό. Το area_name αποθηκεύεται για εμφάνιση στο URL αλλά αφαιρείται από τα API parameters· το backend χρειάζεται αριθμητικά όρια, όχι το label.

Πηγή: Ptixiaki-frontend/Ptixiaki-Frontend/src/pages/Browse.tsx, γραμμές 57-98. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
57    function applyFilters(event: FormEvent<HTMLFormElement>) {
58      event.preventDefault()
59      setFilterError('')

61      if (areaQuery.trim() && !area) {
62        setFilterError('Choose an area from the suggestions, or clear the area to
    search everywhere.')
63        return
64      }

66      const fields = new FormData(event.currentTarget)
67      const next = new URLSearchParams()
68      next.set('transaction', String(fields.get('transaction')))
69      next.set('index', '1')

71      for (const name of ['asking_price_per_sqm', 'square_meters']) {
72        const minimum = String(fields.get(`${name}_min`) ?? '').trim()
73        const maximum = String(fields.get(`${name}_max`) ?? '').trim()

75        if (minimum && maximum && Number(minimum) > Number(maximum)) {
76          setFilterError('The minimum must not be greater than the maximum.')
77          return
78        }

80        if (minimum) {
81          next.set(`${name}_min`, minimum)
82        }
83        if (maximum) {
84          next.set(`${name}_max`, maximum)
85        }
86      }

88      if (area) {
89        const bounds = areaBounds(area)
90        next.set('min_lat', String(bounds.MinLat))
91        next.set('max_lat', String(bounds.MaxLat))
92        next.set('min_lon', String(bounds.MinLon))
93        next.set('max_lon', String(bounds.MaxLon))
94        next.set('area_name', area.display_name)
95      }

97      window.location.hash = `/browse?${next}`
98    }
```

Όταν ο χρήστης επιλέξει ακίνητο, το detail link κρατά τα φίλτρα και προσθέτει uid. Η Workspace αφαιρεί μόνο το uid για να κατασκευάσει backHref. Έτσι η επιστροφή διατηρεί transaction, ranges και index. Η συμπεριφορά αυτή είναι ουσιαστική για πραγματική περιήγηση: ο χρήστης δεν χρειάζεται να ξαναχτίσει την αναζήτηση μετά από κάθε εξέταση κατοικίας.

Οι κάρτες αξιοποιούν την ίδια λογική overview με το Portfolio. Η εικόνα, όπου υπάρχει, είναι δεδομένο της καταχώρισης. Αν δεν υπάρχει ή δεν φορτώνεται, εμφανίζεται κοινό SVG. Το fallback δηλώνει οπτικά απουσία εικόνας και δεν πρέπει να ερμηνεύεται ως φωτογραφία πραγματικού ακινήτου. Η σελιδοποίηση χρησιμοποιεί has_next, επομένως η διεπαφή δεν υπόσχεται γνωστό τελευταίο αριθμό σελίδας.

9.5 Προσθήκη και λεπτομέρειες χαρτοφυλακίου

Η φόρμα AddProperty συγκεντρώνει τύπο κατοικίας, επιφάνεια, κατασκευή, ανακαίνιση, όροφο, παροχές και προαιρετική asking ανά τ.μ. Η επιλεγμένη διεύθυνση συνοδεύεται από provider στοιχεία. Η υποβολή αποστέλλει JSON και περιμένει έως δύο λεπτά. Με επιτυχία χρησιμοποιεί το επιστρεφόμενο UID για owner detail. Δεν δημιουργεί το UID στον browser ούτε θεωρεί τη δική της επιτυχία υποβολής απόδειξη ολοκλήρωσης χωρίς HTTP απάντηση.

Σε αβέβαιη αποτυχία, όπως timeout μετά από commit, εμφανίζεται σύσταση ελέγχου του Portfolio πριν από επανάληψη. Δεν υπάρχει idempotency key που να καθιστά ασφαλές ένα αυτόματο δεύτερο POST. Η επιλογή αυτή αναγνωρίζει το πραγματικό όριο συναλλακτικότητας του backend. Μια πιο σύνθετη ανθεκτική ροή θα απαιτούσε ξεχωριστές καταστάσεις pending/ready/failed και επαναλήψεις παράγωγης επεξεργασίας πάνω στο ίδιο UID.

Η σελίδα λεπτομερειών διαχωρίζει signed contribution, local importance και global importance. Το θετικό πρόσημο δηλώνει αύξηση της μοντελοποιημένης τιμής σε σχέση με τη baseline, όχι ότι το χαρακτηριστικό είναι αντικειμενικά «καλό». Οι τιμές και τα κείμενα πρέπει να διαβάζονται μαζί με transaction και μονάδα. Μια μικρή απόκλιση αθροίσματος εμφανίζεται ως διαγνωστική πληροφορία, όχι ως ποσοστό σφάλματος της εκτίμησης στην πραγματική αγορά.

9.6 Analysis και καταστάσεις αποτυχίας

Η Analysis ξεκινά μόνο αφού επιλεγεί πραγματική περιοχή. Κατά την αναμονή ενημερώνει ότι δημιουργούνται ομάδες και τοπικά μοντέλα. Το Stop waiting ακυρώνει την αναμονή του browser και επισημαίνει ότι η εργασία backend μπορεί να συνεχίζει. Με επιτυχία αποθηκεύονται μαζί αποτέλεσμα και όνομα περιοχής, ώστε η επικεφαλίδα να αναφέρεται στην περιοχή που όντως αναλύθηκε και όχι σε νεότερο κείμενο του input.

Τα components αποτελεσμάτων εμφανίζουν σύνοψη, clusters, περιγραφικά διαγράμματα, ακραίες καταχωρίσεις, αναφορές sell/rent, importance, ζεύγη αλληλεπιδράσεων και παραδείγματα SHAP. Δεν κατασκευάζουν γεωγραφικά polygons για τις ομάδες. Αν ένα μοντέλο λείπει αλλά υπάρχουν clusters, οι warnings επιτρέπουν μερική χρήσιμη προβολή. Η κενή περιοχή διακρίνεται από HTTP αποτυχία, γιατί η απουσία δεδομένων είναι νόμιμο αποτέλεσμα αναζήτησης.

Η απλή αρχιτεκτονική της διεπαφής αποφεύγει επιπλέον libraries για κάθε γράφημα, αλλά έχει κόστος συντήρησης των χειροποίητων components. Οι κοινές HTTP helpers, η επαναχρησιμοποίηση PropertyCard/PropertyDetail και οι δοκιμές συμπεριφοράς μειώνουν τον διπλασιασμό. Η αξιολόγηση στο επόμενο κεφάλαιο εξετάζει τι ελέγχεται αυτοματοποιημένα, χωρίς να εξισώνει τα mock requests με πραγματική λειτουργία όλων των υπηρεσιών.

10. Αξιολόγηση, αποτελέσματα και απειλές εγκυρότητας

10.1 Τρεις διαφορετικές έννοιες επιτυχίας

Η αξιολόγηση διακρίνει λειτουργική ορθότητα, προβλεπτική επίδοση και όφελος χρήστη. Η λειτουργική ορθότητα εξετάζει αν οι διαδρομές τηρούν τα συμβόλαιά τους: σωστή σελιδοποίηση, εξουσιοδότηση, αντιστοίχιση χαρακτηριστικών, προσθετικότητα, reload και χειρισμός αποτυχίας. Η προβλεπτική επίδοση εξετάζει τη διαφορά εκτιμήσεων και πραγματικών ετικετών σε κατάλληλα ανεξάρτητα δεδομένα. Το όφελος χρήστη εξετάζει αν κάποιος λαμβάνει καλύτερες αποφάσεις επειδή χρησιμοποιεί την εφαρμογή.

Τα επίπεδα συνδέονται αλλά δεν ταυτίζονται. Ένα λανθασμένα αξιολογημένο μοντέλο μπορεί να σερβίρεται από άριστα δοκιμασμένο API. Ένα ακριβές μοντέλο μπορεί να παρουσιαστεί με ασαφείς μονάδες και να παραπλανήσει. Μια εύχρηστη εφαρμογή μπορεί να μην επιφέρει οικονομικό όφελος. Για τον λόγο αυτό, το αποτέλεσμα της πτυχιακής δεν συνοψίζεται σε έναν ενιαίο αριθμό «ακρίβειας».

10.2 Αναπαραγώγιμοι έλεγχοι λογισμικού

Στις 19 Οκτωβρίου 2026 εκτελέστηκαν εκ νέου οι διαθέσιμες μη παραγωγικές σουίτες. Το go test ./... ολοκληρώθηκε επιτυχώς με GOCACHE μέσα στο bundle. Η αρχική εκτέλεση είχε επιτυχή package tests αλλά exit failure λόγω μη επιτρεπόμενης εγγραφής στο εξωτερικό cache trim.txt· η επανάληψη με writable cache διευκρίνισε ότι επρόκειτο για περιβαλλοντικό ζήτημα. Δεν παρουσιάζεται το πρώτο exit status ως αποτυχία επιχειρησιακής λογικής ούτε αποκρύπτεται.

Το frontend πέρασε npm run lint, npm run build και 85 tests σε 8 αρχεία. Το build περιλαμβάνει TypeScript έλεγχο και Vite παραγωγή. Η Python πέρασε και τα 10 tests του test_explainability.py στο system Python. Ένα από αυτά εκπαιδεύει μικρό συνθετικό Phase2 μοντέλο σε προσωρινό χώρο για να ελέγξει όλη τη διαδικασία εξαγωγής. Δεν επανεκπαιδεύει τα παραγωγικά μοντέλα και δεν αντικαθιστά τη βάση εξηγήσεων.

Πίνακας 10.1. Αποτελέσματα επαλήθευσης της 19/10/2026

Πεδίο	Εντολή ή αντικείμενο	Αποτέλεσμα	Όριο συμπεράσματος
Go	go test ./...	Επιτυχία	Δεν ισοδυναμεί με δοκιμή ζωντανής βάσης
Frontend lint	npm run lint	Επιτυχία	Έλεγχος κανόνων πηγής
TypeScript/build	npm run build	Επιτυχία	Έλεγχος τύπων και κατασκευή assets
UI συμπεριφορά	npm test -- --run	85/85, 8 αρχεία	HTTP mocks, όχι πλήρες browser E2E
Python	unittest test_explainability.py	10/10	Συνθετικά δεδομένα και contracts
PostgreSQL restore	schema_dump.sql	Δεν εκτελέστηκε	Εκκρεμεί έλεγχος κενής εγκατάστασης
Πλήρης εκπαίδευση	/train	Δεν εκτελέστηκε	Οι μετρικές προέρχονται από manifests

Οι Go δοκιμές καλύπτουν μαθηματικά Elo, feature contracts, parsing mappings, streaming και counts, validation εισόδου, HTTP clients, reload helpers, browse pagination/filters και owner/browse detail. Οι Python έλεγχοι καλύπτουν hash corruption, σχήμα manifest,

ONNX/native parity, sampling, αριθμητική ακύρωση και αληθή failing-row diagnostics. Η frontend σουίτα ελέγχει φόρμες, αναζήτηση, επιλογή περιοχής, πλοήγηση, περιλήψεις και αποτυχίες αιτημάτων.

Υπάρχουν πακέτα χωρίς tests και συμπεριφορές που δεν έχουν ελεγχθεί end-to-end. Δεν εκτελέστηκε stress test, go test -race, penetration test, αυτοματοποιημένη αξιολόγηση προσβασιμότητας ή production load test για τη συγγραφή. Οι ιστορικές καταγραφές του έργου αναφέρουν προηγούμενο πραγματικό browse και ανάλυση Πειραιά, αλλά δεν χρησιμοποιούνται ως νέα μέτρηση της παρούσας εκτέλεσης. Η διάκριση αποτρέπει την ανακύκλωση παλιών ελέγχων ως δήθεν τρέχουσας πιστοποίησης.

10.3 Διαθέσιμα αποτελέσματα μοντέλων

Οι τιμές του επόμενου πίνακα προέρχονται από τα τελικά Phase2 manifests της 15ης Σεπτεμβρίου 2026. Επιλέχθηκαν τα κανονικά Manifest.json των sell/rent και όχι προσωρινά staged αρχεία. Η ύπαρξη hash και χρόνου επιτρέπει ταυτοποίηση του μοντέλου, αλλά δεν αντικαθιστά πρόσβαση στο dataset ή αναπαραγωγή του training. Οι στρογγυλοποιήσεις γίνονται μόνο για την παρουσίαση.

Πίνακας 10.2. Μετρικές των αποθηκευμένων Phase2 μοντέλων

Αγορά	Γραμμές	Training R^2	Validation R^2	Διαφορά
Πώληση	123.449	0,711019	0,563221	0,147798
Μίσθωση	39.781	0,458086	0,008920	0,449166

$$R^2 = 1 - \Sigma[i] (y_i - \hat{y}_i)^2 / \Sigma[i] (y_i - \bar{y})^2 \quad (10.1)$$

Το R^2 συγκρίνει τετραγωνικό σφάλμα με την πρόβλεψη μιας σταθεράς ίσης με τον μέσο του αξιολογούμενου συνόλου. Μπορεί να είναι αρνητικό και δεν είναι ποσοστό ακριβών αγγελιών. Το 0,711 δεν σημαίνει ότι το 71,1% των κατοικιών εκτιμάται σωστά ούτε ότι το μέσο σφάλμα είναι 28,9%. Η επίσημη τεκμηρίωση διευκρινίζει τη σημασία του δείκτη και τις περιπτώσεις μη πεπερασμένου αποτελέσματος (scikit-learn Developers, χ.χ.).

Η επίδοση validation του rent είναι περίπου 0,009. Στο συγκεκριμένο κριτήριο είναι πολύ κοντά στην πρόβλεψη σταθερού μέσου του συνόλου αξιολόγησης. Αυτό είναι ουσιαστικό αρνητικό αποτέλεσμα και δεν πρέπει να κρυφτεί πίσω από το training 0,458. Πιθανές εξηγήσεις είναι θόρυβος ετικετών, μη ομοιογένεια, ακραίες τιμές, ελλείψεις χαρακτηριστικών ή υπερπροσαρμογή. Η παρούσα μελέτη δεν αποδίδει αιτιότητα σε μία από αυτές χωρίς νέα πειράματα.

Για τις πωλήσεις ο validation δείκτης είναι υψηλότερος, αλλά δεν αρκεί ως τελική πιστοποίηση. Η σχέση με target-derived GeoPremium μπορεί να τον κάνει αισιόδοξο. Η έλλειψη MAE, RMSE και κατανομών σφάλματος ανά περιοχή δυσκολεύει την πρακτική ερμηνεία. Ένα ίδιο R^2 μπορεί να συνυπάρχει με πολύ διαφορετικά σφάλματα σε φθηνές και ακριβές κατοικίες.

10.4 Αριθμητικό παράδειγμα ερμηνείας, όχι εμπειρικό εύρημα

Για να φανεί η κατεύθυνση της κατάταξης, θεωρείται τεχνητή περίπτωση πρόβλεψης P = 2.500 €/τ.μ., συνολικού μέσου καταλοίπου 0, συνολικής τυπικής απόκλισης 20, τοπικού μέσου 0, τοπικής τυπικής απόκλισης 10 και Geo z-score 0,5. Οι αριθμοί επιλέγονται αποκλειστικά για επίδειξη των εξισώσεων. Δεν προέρχονται από τη βάση και δεν αποτελούν σύσταση τιμολόγησης.

Πίνακας 10.3. Παράδειγμα επίδρασης asking με σταθερή πρόβλεψη

Asking €/m²	Κατάλοιπο %	Global z	Local z	Elo
2.000	-25,00	-1,25	-2,50	20.250
2.500	0,00	0,00	0,00	11.500
3.000	16,67	0,8333	1,6667	περίπου 5.667
Απουσία	NULL	Παραλείπεται	Παραλείπεται	11.500

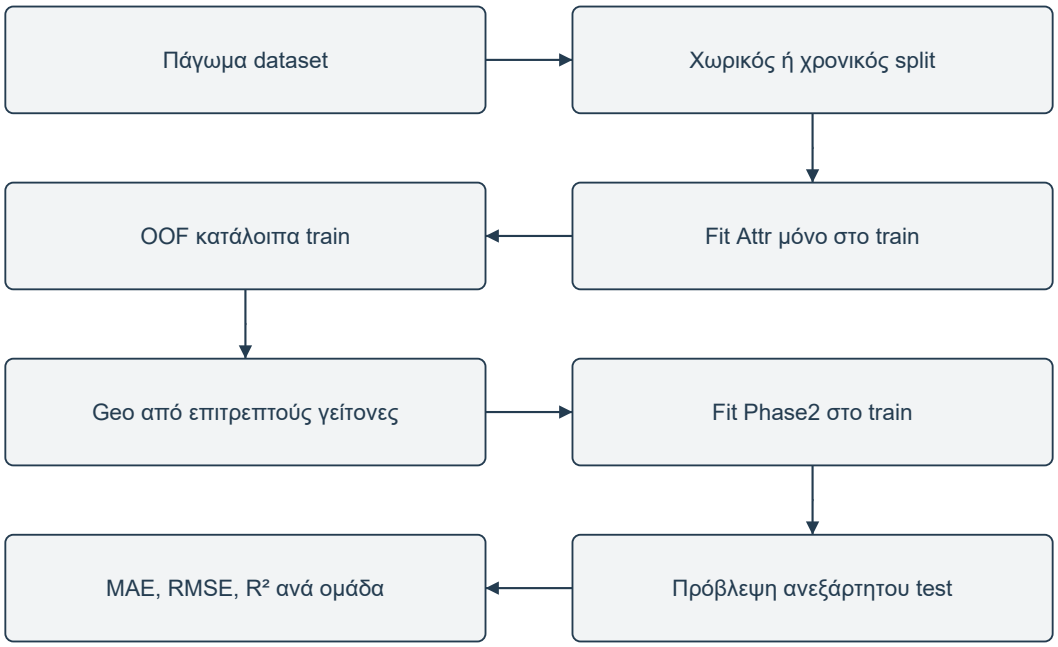
Η τεχνητή σύμπτωση Elo της απουσίας asking με την περίπτωση asking ίσης με estimate οφείλεται στους μηδενικούς μέσους που επιλέχθηκαν. Στα πραγματικά δεδομένα ένα μηδενικό κατάλοιπο δεν σημαίνει απαραίτητα μηδενικό z-score. Η απουσία δεν εξισώνεται γενικά με συμφωνία τιμών. Το παράδειγμα δείχνει επίσης ότι η κλίμακα Elo δεν έχει αυτονόητο ανώτατο όριο ή ερμηνεία πιθανότητας.

Για τις εξηγήσεις μπορεί να θεωρηθεί δεύτερο ανεξάρτητο παράδειγμα: baseline 2.000 €/τ.μ., τρεις συνεισφορές +300, +250 και -50, με όλες τις άλλες μηδενικές. Η πρόβλεψη είναι 2.500. Το αρνητικό χαρακτηριστικό έχει τοπική σημασία 50/600 = 8,33%, όχι -2% της τιμής. Ο διαχωρισμός προσήμου, μεγέθους και baseline αποτρέπει μια συνηθισμένη λανθασμένη ανάγνωση των ποσοστών.

10.5 Προτεινόμενο πρωτόκολλο ανεξάρτητης αξιολόγησης

Η επόμενη πειραματική φάση πρέπει να παγώσει dataset, χρόνο αναφοράς, ταυτότητα καταχώρισης και πρωτόκολλο αποδιπλοποίησης. Κατοικίες που εμφανίζονται πολλές φορές ή με πολύ παρόμοια χαρακτηριστικά δεν πρέπει να κατανεμηθούν ανεξέλεγκτα σε training και test. Ο διαχωρισμός πρέπει να προηγείται κάθε μετασχηματισμού που μαθαίνει από ετικέτες. Οι επιλογές υπερπαραμέτρων πρέπει να γίνονται χωρίς πρόσβαση στο τελικό test.

Για κάθε fold, το Attr πρέπει να εκπαιδεύεται μόνο στο training μέρος. Τα training GeoPremium μπορούν να προκύπτουν από out-of-fold κατάλοιπα ώστε η ίδια γραμμή να μην αξιολογείται από μοντέλο που την έχει ήδη δει. Για τις test γραμμές, οι γείτονες πρέπει να προέρχονται από πληροφορία που θα ήταν διαθέσιμη στον πραγματικό χρόνο πρόβλεψης. Ολόκληρη η Phase2 αλυσίδα πρέπει να επαναλαμβάνεται μέσα στο fold και όχι να μοιράζεται προϋπολογισμένα GeoPremium ολόκληρου του dataset.



Διάγραμμα 10.1. Προτεινόμενη αξιολόγηση χωρίς προϋπολογισμό στο σύνολο

Η σύγκριση πρέπει να περιλαμβάνει τουλάχιστον σταθερό μέσο/διάμεσο ανά αγορά, απλή γεωγραφική διάμεσο, γραμμικό ή κανονικοποιημένο ηδονικό μοντέλο, Attr XGBoost και Phase2 XGBoost. Η διαφορά Attr προς Phase2 είναι ablation test της χωρικής πληροφορίας. Αν η Phase2 δεν βελτιώνει ανεξάρτητο error, η αύξηση πολυπλοκότητας δεν δικαιολογείται από τα διαθέσιμα αποτελέσματα. Δεν έχουν παραχθεί τέτοιες συγκρίσεις στην παρούσα έκδοση.

MAE και RMSE πρέπει να αναφέρονται σε €/τ.μ. και, όπου χρειάζεται, σε συνολικά ευρώ. Χρήσιμη είναι η ανάλυση ανά transaction, περιοχή, τύπο κατοικίας, κλίμακα τιμής, ηλικία και πυκνότητα συγκρίσιμων. Για την εφαρμογή που επιδιώκει να αναδείξει υπερτιμημένες αγγελίες απαιτούνται επιπλέον κατάλληλες ετικέτες τελικής συμφωνίας ή ανεξάρτητης αποτίμησης. Διαφορετικά το «σωστά εντοπισμένη υπερτίμηση» ορίζεται κυκλικά ως διαφωνία με το ίδιο μοντέλο.

10.6 Απειλές εγκυρότητας και υπεύθυνη χρήση

Η εσωτερική εγκυρότητα περιορίζεται από τη διαρροή χωρικής ετικέτας, πιθανές πολλαπλές αγγελίες και διαφορετική επιλογή γειτόνων offline/online. Η εξωτερική εγκυρότητα περιορίζεται από άγνωστη αντιπροσωπευτικότητα και χρονική κάλυψη. Η εγκυρότητα κατασκευής περιορίζεται επειδή η ζητούμενη τιμή είναι proxy της έννοιας «αξία». Η λειτουργική εγκυρότητα περιορίζεται επειδή δεν έχει μετρηθεί πραγματική ανθεκτικότητα σε μαζικό φορτίο ή πολλαπλές ταυτόχρονες διεργασίες.

Υπάρχει επίσης κίνδυνος συμπεριφορικής ανατροφοδότησης. Αν χρήστες προσαρμόζουν τις ζητούμενες τιμές τους στο μοντέλο και οι νέες αγγελίες επιστρέφουν στην εκπαίδευση, το σύστημα μπορεί να ενισχύει τις δικές του προηγούμενες εκτιμήσεις. Η πιθανότητα αυτή δεν έχει μετρηθεί, αλλά αποτελεί σημαντικό σχεδιαστικό ερώτημα για έναν μελλοντικό στόχο «διόρθωσης της αγοράς». Η ενημέρωση χρήστη δεν πρέπει να μετατρέπεται σε ανεξέλεγκτη κανονιστική επιβολή ενός αλγοριθμικού αριθμού.

Δεν υλοποιείται ανίχνευση απάτης ή αξιολόγηση ηθικής συμπεριφοράς πωλητών. Μια ακριβή αγγελία μπορεί να αντανακλά μη διαθέσιμη πληροφορία ή απλώς διαφορετική προσδοκία. Η υπεύθυνη παρουσίαση χρησιμοποιεί όρους όπως «απόκλιση από το μοντέλο» και «ανάγκη περαιτέρω ελέγχου». Η στόχευση του έργου είναι καλύτερη ενημέρωση και περιορισμός της πιθανότητας δυσμενούς συμφωνίας, όχι δημόσια καταγγελία προσώπων.

11. Εκκίνηση, λειτουργία και συντήρηση

11.1 Προϋποθέσεις περιβάλλοντος

Η εκτέλεση προϋποθέτει διαθέσιμη PostgreSQL με τα απαιτούμενα δεδομένα και extensions, Go σύμφωνα με το go.mod, Node συμβατό με το package.json και Python με τις εξαρτήσεις του requirements.txt. Το frontend δηλώνει Node τουλάχιστον 22.20.0. Το ONNX Runtime χρησιμοποιεί την τοπική onnxengine.dll και συμβατή αρχιτεκτονική διεργασίας. Η ύπαρξη του αρχείου δεν αρκεί αν η native βιβλιοθήκη δεν μπορεί να φορτωθεί στο συγκεκριμένο λειτουργικό σύστημα.

Πριν εκκίνηση πρέπει να επιβεβαιωθούν τα δύο Phase2 ONNX, UBJ, manifests και mappings. Σε υπάρχουσα βάση απαιτείται η migration 002 για εξηγήσεις· η migration εικόνων αφορά το προαιρετικό property_images. Η τεκμηρίωση δεν δημοσιεύει πραγματικό connection string, password ή cookie. Η σημερινή σύνδεση βάσης είναι ενσωματωμένη στον κώδικα και δεν υποστηρίζει αυτομάτως μια νέα υποθετική DATABASE_URL. Η μεταφορά σε ασφαλή διαμόρφωση είναι απαραίτητη αλλαγή πριν deployment, όχι ολοκληρωμένη δυνατότητα.

11.2 Πραγματική σειρά εκκίνησης

Η βάση πρέπει να είναι ήδη διαθέσιμη. Κατόπιν ξεκινά η Python υπηρεσία από τον φάκελό της, με ρητή διαδρομή μοντέλων για να αποφεύγεται η επιλογή λάθος αντιγράφου. Το παρακάτω παράδειγμα αντιστοιχεί στη διάταξη του εξεταζόμενου workspace και όχι σε γενικό εγκαταστάτη άλλου υπολογιστή.

```
Set-Location S:\ptixiaki-bundle\Ptixiaki-ml-microservice
$env:PTIXIAKI_MODEL_ROOT = (
    Resolve-Path '..\Ptixiaki\Worker\Models\TreeOutputs'
).Path
python main.py
```

Σε δεύτερο terminal εκκινεί η Go από τον δικό της φάκελο. Τα σχετικά paths του main.go αναζητούνται από εκεί. Η εντολή go run . συμπεριλαμβάνει το package main συνολικά. Το terminal παραμένει ενεργό για να εμφανίζονται σφάλματα βάσης, μοντέλων και αιτημάτων. Δεν καλείται /train ως health check, επειδή είναι δαπανηρή και μεταβάλλει δεδομένα.

```
Set-Location S:\ptixiaki-bundle\Ptixiaki
go run .
```

Σε τρίτο terminal ξεκινά ο frontend dev server. Η πρόσβαση γίνεται στο localhost:5173 και τα API requests δρομολογούνται μέσω proxy. Η χρήση npm run build δημιουργεί static assets αλλά δεν εκκινεί τη Go, την Python ή τη βάση. Αν χρησιμοποιηθεί άλλος server για production hosting, απαιτείται αντίστοιχη ρύθμιση reverse proxy, HTTPS και cookies.

```
Set-Location S:\ptixiaki-bundle\Ptixiaki-frontend\Ptixiaki-Frontend
npm run dev -- --host 127.0.0.1
```

Η Python που βρίσκει η Go στο PATH πρέπει να διαθέτει και τις Worker εξαρτήσεις, όπως onnxmltools και skl2onnx. Δεν αρκεί να λειτουργεί το FastAPI σε άλλο venv. Το περιβάλλον υπηρεσίας και το περιβάλλον child process είναι διαφορετικές επιλογές interpreter. Η εντολή Get-Command python και ο έλεγχος import στο ίδιο terminal της Go είναι πιο χρήσιμοι από την απλή υπόθεση ότι «η Python είναι εγκατεστημένη».

11.3 Έλεγχος βασικών διαδρομών χωρίς εκπαίδευση

Μετά τη σύνδεση από τον browser ελέγχονται πρώτα Browse και Portfolio, που είναι αναγνωστικές διαδρομές. Η επιλογή περιοχής επιβεβαιώνει τη λειτουργία admin_boundaries. Μια λεπτομερής προβολή ελέγχει τις joins και τα αποθηκευμένα παράγωγα δεδομένα χωρίς νέα εξήγηση. Μια Analysis είναι ακριβότερη και εκπαιδεύει προσωρινά μοντέλα, αλλά δεν αντικαθιστά τους καθολικούς πίνακες SHAP.

Για terminal requests χρησιμοποιείται cookie jar που έχει παραχθεί από κανονική σύνδεση. Το όνομα cookie.txt στα παραδείγματα είναι τοπικό αρχείο συνεδρίας και δεν πρέπει να κοινοποιείται. Η URL σε PowerShell πρέπει να είναι απλή συμβολοσειρά, όχι Markdown link. Για πολύπλοκο JSON είναι ασφαλέστερο ένα ελεγχόμενο αρχείο request και --data-binary @file από εύθραυστο συνδυασμό εισαγωγικών και backticks.

```
curl.exe -i -b .\cookie.txt `
    "http://localhost:8080/User/BrowseProperties?transaction=sell&index=1"
curl.exe -i -b .\cookie.txt "http://localhost:8080/User/Portfolio"
```

Η δοκιμή Add απαιτεί πραγματική επιλεγμένη διεύθυνση από το address endpoint. Δεν ενσωματώνεται αυθαίρετο OSM ID ως δήθεν επαληθευμένο παράδειγμα. Ο ελεγκτής αναζητεί διεύθυνση, επιλέγει την canonical απάντηση και χρησιμοποιεί τα πραγματικά osm_type/osm_id. Μετά από επιτυχία διαβάζει το UID της απόκρισης. Μετά από αμφίβολο timeout ελέγχει πρώτα το Portfolio ώστε να μην παράγει διπλή καταχώριση.

11.4 Διάγνωση ανά σύμπτωμα

Πίνακας 11.1. Λειτουργική διάγνωση χωρίς αυθαίρετη επανεκπαίδευση

Σύμπτωμα	Πρώτος έλεγχος	Ερμηνεία ή ενέργεια
Δεν ανοίγει η διεπαφή	Vite terminal και θύρα 5173	Το frontend είναι ξεχωριστή διεργασία
401 στα δεδομένα	Κανονική σύνδεση και cookies	Έλεγχος συνεδρίας, όχι διαγραφή δεδομένων
Κενές προτάσεις περιοχής	admin_boundaries και όνομα	Η κάλυψη είναι του τοπικού καταλόγου
422 σε Add	Επιλεγμένο OSM object	Νέα επιλογή, όχι χειροκίνητη αλλαγή συντεταγμένων
Explanation null	Python logs, registry, hash	Η εισαγωγή μπορεί να έχει επιτύχει χωρίς SHAP
Reload mismatch	PTIXIAKI_MODEL_ROOT και manifest	Πιθανή διαφορετική δημοσίευση στον server
Υψηλή CPU	Τελευταίο progress stage	SHAP interactions ή area training μπορεί να είναι ακριβό
Μερικό Analysis	warnings ανά transaction	Δεν υπάρχει κατ’ ανάγκη αποτυχία των clusters
Σφάλμα schema preflight	Migration 002	Δεν πρέπει να ξεκινήσει επανειλημμένα training

Το progress ETA είναι αναγωγή του μέχρι στιγμής ρυθμού, όχι χρονική εγγύηση. Διαφορετικά batches μπορεί να έχουν διαφορετικό κόστος. Η ακύρωση ενός trainer πριν από file activation αφήνει συνήθως τα προηγούμενα τελικά artifacts, αλλά διακοπή σε μεταγενέστερο σημείο απαιτεί έλεγχο αρχείων, βάσης και υπηρεσιών χωριστά. Η ύπαρξη ενός τελικού log δεν πρέπει να είναι το μόνο αποθηκευμένο αποδεικτικό της κατάστασης.

11.5 Ασφάλεια και προτεραιότητες πριν από δημόσια διάθεση

Πρώτη προτεραιότητα είναι η αφαίρεση και περιστροφή εκτεθειμένων μυστικών. Η αλλαγή μόνο του αρχείου τοπικά δεν ακυρώνει ένα μυστικό που έχει ήδη αποθηκευτεί σε ιστορικό ή διανεμημένο bundle. Ακολουθούν least-privilege ρόλοι βάσης, TLS, περιορισμός πρόσβασης της Python στο εσωτερικό περιβάλλον και προστασία /train και /test με πραγματικό administrative authorization. Το admin flag δεν πρέπει να επιλέγεται ελεύθερα κατά δημόσια εγγραφή.

Δεύτερη προτεραιότητα είναι η σαφής πολιτική δεδομένων. Η ιδιωτική προσθήκη δεν πρέπει να εμφανίζεται στο browse χωρίς ενημερωμένη επιλογή δημοσίευσης. Η αφαίρεση σχέσης portfolio δεν είναι πλήρης διαγραφή καταχώρισης. Τα logs μπορεί να περιέχουν UID, διευθύνσεις ή SQL και χρειάζονται έλεγχο ελαχιστοποίησης. Το schema έχει ευρείες άδειες και δεν εφαρμόζει γενικευμένο row-level security. Αυτά καταγράφονται ως περιορισμοί και όχι ως λυμένα επειδή υπάρχει JWT.

Τρίτη προτεραιότητα είναι η διαχείριση βλαβών. Η εισαγωγή θα ωφελούνταν από idempotency key και σαφή κατάσταση παράγωγης επεξεργασίας. Η εκπαίδευση θα ωφελούνταν από durable generation record και κοινή ενεργοποίηση αρχείων/στατιστικών. Η εξυπηρέτηση θα απαιτούσε ρυθμισμένα HTTP server timeouts, graceful shutdown και παρακολούθηση υγείας. Η σημερινή εφαρμογή είναι ερευνητικό πρωτότυπο με σημαντική λειτουργική κάλυψη, όχι έτοιμη παραγωγική υπηρεσία οικονομικής συμβουλής.

12. Συμπεράσματα και μελλοντική εργασία

12.1 Αποτίμηση των στόχων

Το Property Co-Pilot υλοποιεί μια συνεκτική ροή από καταχώριση και χωρική επιλογή μέχρι πρόβλεψη, κατάταξη και εξήγηση. Η ανάγκη που αντιμετωπίζει είναι η δυσκολία συγκριτικής αξιολόγησης ετερογενών αγγελιών. Ο επιδιωκόμενος σκοπός είναι μεγαλύτερη διαφάνεια, εντοπισμός πιθανών υπερτιμολογήσεων και υποστήριξη του χρήστη απέναντι σε δυσμενείς οικονομικές επιλογές. Η εργασία δεν αποδεικνύει ότι έχει ήδη διορθώσει τιμές αγοράς ή μειώσει πραγματικές ζημίες χρηστών.

Ως προς το ΕΕ1, το συμβόλαιο τριάντα χαρακτηριστικών, τα mappings, οι έλεγχοι native/ONNX και τα tests υποστηρίζουν την τεχνική συμβατότητα. Ως προς το ΕΕ2, το GeoPremium παρέχει συγκεκριμένη μέθοδο χωρικής εμπλουτισμένης εισόδου, αλλά η offline/online επιλογή γειτόνων και η διαρροή στην αξιολόγηση περιορίζουν την εγκυρότητα. Ως προς το ΕΕ3, η διάκριση baseline, contribution, importance, κατάλοιπου και Elo επιτρέπει πιο σαφή παρουσίαση. Ως προς το ΕΕ4, οι δοκιμές αποδεικνύουν συγκεκριμένες συμπεριφορές, όχι τελική χρησιμότητα ή γενίκευση.

Η σημαντικότερη τεχνική συμβολή είναι η σύνδεση ετερογενών περιβαλλόντων χωρίς να χάνεται η ταυτότητα του μοντέλου και η σημασία των εισόδων. Το UBJ υποστηρίζει native εξηγήσεις, το ONNX τοπικό inference και το manifest περιγράφει το συμβόλαιο. Τα hashes βοηθούν στην ταυτοποίηση, οι κλειδαριές στη μνήμη και οι συναλλαγές σε επιμέρους εγγυήσεις βάσης. Η ανάλυση έδειξε ταυτόχρονα ότι καμία από αυτές τις τεχνικές δεν αρκεί για καθολική ατομικότητα.

12.2 Περιορισμοί που καθορίζουν την επόμενη έκδοση

Η χαμηλή validation επίδοση μισθώσεων αποτελεί πρώτο επιστημονικό εμπόδιο. Προέχει η διερεύνηση δεδομένων και η καθαρή αξιολόγηση αντί της απλής αύξησης αριθμού δέντρων. Χρειάζονται χρονικά και χωρικά ανεξάρτητοι διαχωρισμοί, baselines, MAE/RMSE και ανάλυση ανά υποομάδα. Η αξιολόγηση με πραγματικές τελικές συμφωνίες θα ήταν αναγκαία για ισχυρισμούς περί πραγματικής υπερτίμησης.

Στο λογισμικό προέχουν πολιτική δημοσίευσης, ενοποίηση ταυτότητας, ασφάλεια μυστικών και διαχείριση μερικών αποτυχιών. Η διατήρηση latest-only μπορεί να παραμείνει επιλογή, αλλά χρειάζεται μηχανισμός σαφούς ενεργής γενιάς. Οι online εξηγήσεις ακινήτων χωρίς asking πρέπει να διατηρούνται ή να επανυπολογίζονται με προβλέψιμη πολιτική. Η χρονική αναφορά των features πρέπει να γίνει μεταδεδομένο και όχι διάσπαρτη σταθερά.

12.3 Τελική συμβολή

Η εργασία προσφέρει λειτουργικό πρωτότυπο και τεκμηριωμένη μελέτη των ορίων του. Η εξηγησιμότητα δεν χρησιμοποιείται ως διακοσμητικό γράφημα αλλά ως ελεγχόμενο αριθμητικό συμβόλαιο. Η κατάταξη δεν παρουσιάζεται ως βεβαιότητα κέρδους αλλά ως ευρετική που χρειάζεται αξιολόγηση. Η εφαρμογή μπορεί να αποτελέσει βάση για καλύτερη συγκριτική πληροφόρηση, αρκεί η συνέχεια του έργου να δώσει ίσο βάρος στην ποιότητα δεδομένων, στην ανεξάρτητη επαλήθευση και στην υπεύθυνη επικοινωνία προς τον χρήστη.

Πίνακας ορολογίας

Πίνακας Ο.1. Βασικοί ελληνικοί και αγγλικοί όροι

Ελληνικός όρος	Αγγλικός όρος	Χρήση στο έργο
Ζητούμενη τιμή	Asking price	Τιμή αγγελίας, όχι κατ' ανάγκη ολοκληρωμένης συναλλαγής
Εκτίμηση	Prediction / estimate	Έξοδος μοντέλου σε €/m²
Χαρακτηριστικό	Feature	Μία από τις εισόδους του μοντέλου
Ετικέτα / στόχος	Label / target	AskingPricePerSqm κατά την εκπαίδευση
Κατάλοιπο	Residual	Απόκλιση asking από prediction
Τιμή αναφοράς	Baseline	Σταθερός όρος της αποσύνθεσης SHAP
Συνεισφορά	Contribution	Προσημασμένη επίδραση χαρακτηριστικού στη μοντελοποιημένη τιμή
Σημασία χαρακτηριστικού	Feature importance	Συγκεντρωτικό ή τοπικό μέτρο συμμετοχής
Αλληλεπίδραση	Interaction	Συνδυαστική συμμετοχή δύο χαρακτηριστικών
Χωρική προσαύξηση	GeoPremium	Εσωτερική ονομασία μέσου Attr καταλοίπου γειτόνων
Τυποποιημένη απόκλιση	Z-score	Απόσταση από μέσο σε μονάδες τυπικής απόκλισης
Ενίσχυση κλίσης	Gradient boosting	Διαδοχική προσθήκη δέντρων για μείωση απώλειας
Ομαδοποίηση	Clustering	Περιγραφική δημιουργία ομάδων ομοιότητας
Επικύρωση	Validation	Έλεγχος σε διαχωρισμένο σύνολο, με τους δηλωμένους περιορισμούς
Διαρροή πληροφορίας	Data leakage	Χρήση μη επιτρεπτής πληροφορίας κατά αξιολόγηση
Παραγόμενο αρχείο	Artifact	ONNX, UBJ, manifest, mapping ή prediction output
Αποτύπωμα	Hash digest	Ταυτότητα bytes UBJ με SHA-256
Δημοσίευση	Publication / activation	Αντικατάσταση ενεργών artifacts ή πινάκων
Επαναφόρτωση	Reload	Φόρτωση αποθηκευμένου μοντέλου στη μνήμη υπηρεσίας

Ελληνικός όρος	Αγγλικός όρος	Χρήση στο έργο
Προσωρινή αποθήκευση	Staging	Ελεγχόμενο ενδιάμεσο πριν από δημοσίευση
Αντίθλιψη ροής	Backpressure	Αναμονή παραγωγού όταν ο καταναλωτής δεν προλαβαίνει
Αμοιβαίος αποκλεισμός	Mutual exclusion	Προστασία κοινών μεταβλητών ή buffers
Δεξαμενή συνδέσεων	Connection pool	Επαναχρησιμοποίηση συνδέσεων PostgreSQL
Στιγμιότυπο ανάγνωσης	Read snapshot	Συνεπής όψη δεδομένων μιας συναλλαγής
Γεωκωδικοποίηση	Geocoding	Αντιστοίχιση επιλογής διεύθυνσης σε θέση
Ορθογώνιο ορίων	Bounding box	Min/max latitude και longitude
Σελιδοποίηση	Pagination	Διαίρεση αποτελεσμάτων σε σελίδες
Αυθεντικοποίηση	Authentication	Έλεγχος ταυτότητας χρήστη
Εξουσιοδότηση	Authorization	Έλεγχος δικαιώματος συγκεκριμένης ενέργειας
Ιδιοδυναμία επανάληψης	Idempotency	Ασφαλής επανάληψη ενέργειας χωρίς νέο αποτέλεσμα

Πίνακας συντμήσεων και ακρωνυμίων

Πίνακας Σ.1. Ανάπτυξη ακρωνυμίων που χρησιμοποιούνται στο κείμενο

Ακρωνύμιο	Ανάπτυξη	Σημασία
API	Application Programming Interface	Συμβόλαιο επικοινωνίας εφαρμογών
AVM	Automated Valuation Model	Αυτοματοποιημένο μοντέλο εκτίμησης
CSV	Comma-Separated Values	Πινακοειδές αρχείο εκπαίδευσης και mappings
DDL	Data Definition Language	Ορισμοί σχήματος βάσης
EOF	End Of File	Τέλος ροής bytes
GIS	Geographic Information System	Γεωγραφικό πληροφοριακό σύστημα
HTTP	Hypertext Transfer Protocol	Πρωτόκολλο αιτημάτων υπηρεσιών
JSON	JavaScript Object Notation	Μορφή ανταλλαγής δομημένων δεδομένων
JWT	JSON Web Token	Υπογεγραμμένο token συνεδρίας
MAE	Mean Absolute Error	Μέσο απόλυτο σφάλμα
ML	Machine Learning	Μηχανική μάθηση
ONNX	Open Neural Network Exchange	Φορητή αναπαράσταση μοντέλου
OOF	Out Of Fold	Πρόβλεψη από fit που δεν περιλαμβάνει τη συγκεκριμένη γραμμή
OSM	OpenStreetMap	Γεωγραφικός πάροχος και αναγνωριστικά αντικειμένων
R^2	Coefficient of determination	Συντελεστής προσδιορισμού
RMSE	Root Mean Squared Error	Ρίζα μέσου τετραγωνικού σφάλματος
SHA-256	Secure Hash Algorithm, 256 bits	Συνάρτηση αποτυπώματος αρχείου
SHAP	SHapley Additive exPlanations	Προσθετικές εξηγήσεις χαρακτηριστικών
SQL	Structured Query Language	Γλώσσα σχεσιακών ερωτημάτων
SRID	Spatial Reference System Identifier	Αναγνωριστικό συστήματος αναφοράς
SVG	Scalable Vector Graphics	Διανυσματική γραφική αναπαράσταση
TLS	Transport Layer Security	Ασφάλεια μεταφοράς δικτύου
UBJ	Universal Binary JSON	Native δυαδική μορφή μοντέλου XGBoost
UID	Unique Identifier	Αναγνωριστικό ακινήτου ή χρήστη
ULID	Universally Unique Lexicographically Sortable Identifier	Αναγνωριστικό νέων εγγραφών

Βιβλιογραφία

Οι παραπομπές ακολουθούν σύστημα συγγραφέα και έτους. Οι τεχνικές περιγραφές της εφαρμογής προκύπτουν πρωτίστως από το εξεταζόμενο bundle, ενώ οι παρακάτω πηγές θεμελιώνουν έννοιες, συμβάσεις και επιλογές. Οι διαδικτυακές πηγές ελέγχθηκαν κατά τη σύνταξη στις 19/10/2026. Τα παραδείγματα ΔΙΩΝΗ χρησιμοποιήθηκαν για μελέτη οργάνωσης ακαδημαϊκού κειμένου και όχι για μεταφορά αποτελεσμάτων άλλου πεδίου.

Antipov, E. A. και Pokryshevskaya, E. B. (2012). Mass appraisal of residential apartments: An application of Random forest for valuation and a CART-based approach for model diagnostics. Expert Systems with Applications. Εγγραφή του πρωτότυπου άρθρου.

Chen, T. και Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016. DOI: 10.1145/2939672.2939785. Πρωτότυπη εργασία.

Go Authors (χ.χ.-α). io package: Reader και ReadCloser. Επίσημη τεκμηρίωση. Τεκμηρίωση io.

Go Authors (χ.χ.-β). os/exec package: Cmd, StdoutPipe, Start και Wait. Επίσημη τεκμηρίωση. Τεκμηρίωση os/exec.

Go Authors (χ.χ.-γ). bufio package: Scanner και Buffer. Επίσημη τεκμηρίωση. Τεκμηρίωση bufio.

Go Authors (χ.χ.-δ). sync package: Mutex και RWMutex. Επίσημη τεκμηρίωση. Τεκμηρίωση sync.

Lundberg, S. M. και Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems. Πρωτότυπη εργασία SHAP.

Lundberg, S. M. και συν. (2020). From local explanations to global understanding with explainable AI for trees. Nature Machine Intelligence. DOI: 10.1038/s42256-019-0138-9. Δημοσιευμένη εργασία TreeSHAP.

OpenStreetMap Foundation (χ.χ.). Nominatim Usage Policy. Πολιτική της δημόσιας υπηρεσίας.

PostGIS Project (χ.χ.). ST_DWithin. Επίσημη τεκμηρίωση λειτουργίας γεωμετρίας και γεωγραφίας. Τεκμηρίωση χωρικής απόστασης.

PostgreSQL Global Development Group (χ.χ.). PostgreSQL 17 Documentation: Transaction Isolation. Επίπεδα απομόνωσης συναλλαγών.

Roberts, D. R. και συν. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. Ecography. DOI: 10.1111/ecog.02881. Πλήρες κείμενο από ερευνητικό φορέα.

Rosen, S. (1974). Hedonic Prices and Implicit Markets: Product Differentiation in Pure Competition. Journal of Political Economy, 82(1), 34-55. DOI: 10.1086/260169. Πρωτότυπο άρθρο.

scikit-learn Developers (χ.χ.). r2_score. Επίσημη τεκμηρίωση μετρικής παλινδρόμησης. Ορισμός και ιδιότητες R^2.

XGBoost Developers (χ.χ.). Introduction to Model IO. Επίσημη τεκμηρίωση αποθήκευσης JSON/UBJSON και διάκρισης μοντέλων από memory snapshots. Αποθήκευση μοντέλων.

Πανεπιστήμιο Πειραιώς, Τμήμα Πληροφορικής (χ.χ.). Κανονισμός εκπόνησης πτυχιακής εργασίας και ενσωματωμένο πρότυπο συγγραφής. Παρεχόμενο PDF 46 σελίδων· οδηγίες συγγραφής στις σελίδες 12-18 και πρότυπο από τη σελίδα 20. Ιστοσελίδα κανονισμού Τμήματος. Η παρεχόμενη τοπική έκδοση αποτελεί το υλικό μορφοποίησης της παρούσας εργασίας.

Πάκια, Λ. (2025). Μηχανική μάθηση στην ανίχνευση κυβερνοεπιθέσεων: βιβλιογραφική ανασκόπηση των μεθόδων και των τεχνολογιών. Πτυχιακή εργασία, Τμήμα Πληροφορικής, Πανεπιστήμιο Πειραιώς. Αποθετήριο ΔΙΩΝΗ.

Φρονίστας, Κ. (2025). Ανάπτυξη διαδραστικού εκπαιδευτικού λογισμικού για την εισαγωγή στον προγραμματισμό με Python. Πτυχιακή εργασία, Τμήμα Πληροφορικής, Πανεπιστήμιο

Πειραιώς. Αποθετήριο ΔΙΩΝΗ.

Πηγαίο υλικό έργου (2026). Property Co-Pilot: Ptixiaki, Ptixiaki-ml-microservice, ml_shared, Ptixiaki-frontend/Ptixiaki-Frontend και schema_dump.sql. Τοπικό bundle που παραδόθηκε για μελέτη· έλεγχος 19/10/2026. Δεν υπάρχει στην παρούσα έκδοση δημόσιο archival DOI ή πλήρες ανωνυμοποιημένο dataset. Οι ακριβείς σχετικές διαδρομές και γραμμές των κρίσιμων αποσπασμάτων αναφέρονται στο σώμα και στο παράρτημα.

Παράρτημα Α. Συμβόλαια HTTP και διαδρομές

Α.1 Routes που χρησιμοποιεί η διεπαφή

Πίνακας Α.1. Διαδρομές και αναγνωστικά ή μεταβλητικά συμβόλαια

Route	Μέθοδος	Είσοδος / πρόσβαση	Βασική έξοδος
/register/SignIn	POST	email, password	Cookie και αποτέλεσμα σύνδεσης
/register/SignUp	POST	Στοιχεία εγγραφής	Cookie και αποτέλεσμα εγγραφής
/User/BrowseProperties	GET	JWT, filters, index	index, page_size, has_next, properties
/User/BrowseProperty	GET	JWT, uid	Κοινό detail αγοράς
/User/Portfolio	GET	JWT	property_count, total_estimated_value, properties
/User/ViewProperty	GET	JWT, uid, ownership	Owner detail
/User/AddProperty	POST	JWT, transaction, JSON	201, message, uid, Location
/User/DeleteProperty	DELETE	JWT, property_uid, ownership	Μήνυμα αφαίρεσης σχέσης
/analysis/autocomplete	GET	JWT, q, optional kind=address	Προτάσεις περιοχής ή διεύθυνσης
/analysis/CreateAnalysisForArea	POST	JWT, αριθμητικό bbox	Points, Clusters, SalesReport, RentReport, warnings

Ο πίνακας περιγράφει τις μεθόδους χρήσης από τη διεπαφή. Δεν δηλώνει ότι όλοι οι παλαιότεροι auth handlers επιβάλλουν τις ίδιες method/body validation δικλίδες με τους νεότερους. Η owner detail διατηρεί επίσης legacy POST μορφή με PropertyUid. Τα /train, /test και /predict/sell υπάρχουν στην κύρια υπηρεσία αλλά δεν αποτελούν τις κανονικές προστατευμένες ενέργειες διεπαφής. Η δημόσια έκθεσή τους στην τρέχουσα μορφή δεν συνιστάται.

A.2 Εσωτερικό συμβόλαιο Python

Πίνακας A.2. Αιτήματα προς τη μικροϋπηρεσία

Endpoint	Είσοδος	Μοντέλο	Αποτέλεσμα
/ReloadExplainModels	transactions, expected_model_sha256	Φόρτωση από κοινό filesystem	Summaries ενεργών native μοντέλων
/ExplainProperty	transaction, features	Ήδη φορτωμένο UBJ	Baseline, contributions, prediction, hash
/TrainAndExplain	data, target, categories, dropped columns, params	Νέο προσωρινό XGBoost	R^2, importance, interactions, 10 examples

Το batch SHAP της καθολικής εκπαίδευσης δεν περνά από /ExplainProperty. Παράγεται μέσα στο Tree.py και μεταφέρεται μέσω stdout προς τη Go. Η διάκριση αποφεύγει χιλιάδες HTTP κλήσεις κατά την εκπαίδευση. Το /ExplainProperty προορίζεται για νέα μεμονωμένη αριθμητική είσοδο στην ήδη φορτωμένη γενιά.

A.3 Παράδειγμα μορφής επιτυχίας προσθήκης

Το ακόλουθο JSON δείχνει τη μορφή και όχι πραγματική νέα καταχώριση. Το UID πρέπει να ληφθεί από την πραγματική απόκριση. Δεν είναι προέκταση του request και δεν παράγεται από τον browser.

```
{
  "message": "Property added successfully",
  "uid": "property_<generated-ulid>"
}
```

Παράρτημα Β. Ολοκληρωμένος κώδικας κρίσιμης ροής

Β.1 Εκτέλεση trainer και δημοσίευση SHAP

Παρατίθεται ολόκληρη η RunTrainingAndPersist, ώστε τα επιμέρους αποσπάσματα του κεφαλαίου 7 να διαβάζονται και ως ενιαία ροή. Τα αμιγώς σχολιαστικά lines παραλείπονται για οικονομία χώρου, αλλά διατηρούνται οι αριθμοί γραμμών του αρχείου. Το απόσπασμα απαιτεί το υπόλοιπο package και δεν είναι αυτόνομο πρόγραμμα. Το context ελέγχει τη διάρκεια, ο parser την εγκυρότητα και η συναλλαγή τη δημοσίευση των συγκεκριμένων πινάκων.

Πηγή: Ptixiaki/Worker/explainability/training.go, γραμμές 63-182. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
 63  func RunTrainingAndPersist(
 64      ctx context.Context,
 65      db *pgxpool.Pool,
 66      scriptPath string,
 67      args []string,
 68      transaction string,
 69      beforePublish func(context.Context) error,
 70  ) error {
 71      tables, err := TablesForTransaction(transaction)
 72      if err != nil {
 73          return err
 74      }
 75      runCtx, cancel := context.WithTimeout(ctx, TrainingProcessTimeout)
 76      defer cancel()
 77      started := time.Now()
 78      log.Printf("[TRAIN %s] Checking explanation schema and preparing database
    staging", transaction)

 80      tx, err := db.Begin(runCtx)
 81      if err != nil {
 82          return fmt.Errorf("begin explanation publication transaction: %w", err)
 83      }
 84      defer func() { _ = tx.Rollback(context.Background()) }()

 86      if err := preflightPersistenceSchema(runCtx, tx, tables); err != nil {
 87          return err
 88      }
 89      if err := createStagingTables(runCtx, tx); err != nil {
 90          return err
 91      }

 93      stager := &propertyStager{ctx: runCtx, tx: tx, rows: make([][]any, 0,
    PropertyStageBatchSize)}
 94      parser, err := NewStreamParser(transaction, stager.add)
 95      if err != nil {
 96          return err
 97      }
 98      progress := &progressLogger{transaction: transaction, printf: log.Printf}
 99      parser.SetProgressHandler(progress.report)

101      cmdArgs := append([]string{scriptPath}, args...)
102      cmd := exec.CommandContext(runCtx, "python", cmdArgs...)
103      stdout, err := cmd.StdoutPipe()
104      if err != nil {
105          return fmt.Errorf("open Python stdout: %w", err)
106      }
107      cmd.Stderr = os.Stderr

109      log.Printf("[TRAIN %s] Launching Python training with explanations: %s",
    transaction, scriptPath)
110      if err := cmd.Start(); err != nil {
111          return fmt.Errorf("start Python training: %w", err)
112      }

114      scanner := bufio.NewScanner(stdout)
115      scanner.Buffer(make([]byte, 64*1024), ScannerMaxTokenBytes)
```

```
116     var protocolErr error
117     metaLogged := false
118     for scanner.Scan() {
119         line := scanner.Text()
120         recognized, parseErr := parser.ParseLine(line)
121         if parseErr != nil {
122             protocolErr = parseErr
123             break
124         }
125         if !recognized && strings.TrimSpace(line) != "" {
126             protocolErr = fmt.Errorf("unexpected non-protocol Python stdout: %q",
 line)
127             break
128         }
129         if parser.meta != nil && !metaLogged {
130             log.Printf("[TRAIN %s] Explaining all %d properties; global
 interactions use %d properties (seed %d)",
131                 transaction, parser.meta.PropertyCount,
 parser.meta.InteractionSampleCount, parser.meta.InteractionSampleSeed)
132             metaLogged = true
133         }
134     }
135     if protocolErr == nil {
136         protocolErr = scanner.Err()
137     }
138     if protocolErr != nil && cmd.Process != nil {
139         _ = cmd.Process.Kill()
140     }
141     waitErr := cmd.Wait()
142     if protocolErr != nil {
143         return fmt.Errorf("invalid Python explanation stream: %w", protocolErr)
144     }
145     if waitErr != nil {
146         if errors.Is(runCtx.Err(), context.DeadlineExceeded) {
147             return fmt.Errorf("Python explanation training timed out after %s",
 TrainingProcessTimeout)
148         }
149         return fmt.Errorf("Python training process failed: %w", waitErr)
150     }
```

```
152     records, err := parser.Finalize()
153     if err != nil {
154         return fmt.Errorf("validate Python explanation stream: %w", err)
155     }
156     log.Printf("[TRAIN %s] Python finished; validated %d property, %d importance
 and %d interaction records. Finishing database staging", transaction,
 records.PropertyCount, len(records.Importance), len(records.Interactions))
157     if err := stager.flush(); err != nil {
158         return fmt.Errorf("flush property explanation staging: %w", err)
159     }
160     if stager.count != records.PropertyCount {
161         return fmt.Errorf("staged property count mismatch: got %d, expected %d",
 stager.count, records.PropertyCount)
162     }
163     if err := stageAggregateRecords(runCtx, tx, records); err != nil {
164         return err
165     }
166     if beforePublish != nil {
167         log.Printf("[TRAIN %s] Persisting predictions, global statistics and
 Elo", transaction)
168         if err := beforePublish(runCtx); err != nil {
169             return fmt.Errorf("complete dependent training persistence before
 explanation publication: %w", err)
170         }
171     }
172     log.Printf("[TRAIN %s] Publishing explanation tables and model statistics",
 transaction)
```

```
173      if err := publishStagedRecords(runCtx, tx, transaction, records); err != nil
     {
174          return err
175      }
176      if err := tx.Commit(runCtx); err != nil {
177          return fmt.Errorf("commit explanation publication: %w", err)
178      }
```

```
180      log.Printf("[TRAIN %s] Explanation publication committed successfully
     (elapsed %s)", transaction, time.Since(started).Round(time.Second))
181      return nil
182  }
```

B.2 Πλήρης μαθηματικός ορισμός του score στον κοινό helper

Οι helpers χρησιμοποιούνται ως κοινός ορισμός της τυποποίησης και της βαθμολογίας. Η απλότητα αυτών των συναρτήσεων διευκολύνει τη δοκιμή κατεύθυνσης των προσήμων. Δεν αρκεί όμως για ισότητα δύο πλήρων pipelines όταν διαφέρουν τα σύνολα γειτόνων ή τα αποθηκευμένα στατιστικά τους.

Πηγή: Ptixiaki/Worker/Pipelines/CalculateEloForProps.go, γραμμές 198-207. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
198  func CalculateZScore(value float64, mean float64, std float64) float64 {
199      if std == 0 {
200          return 0
201      }
202      return (value - mean) / std
203  }
```

```
205  func CalculateEloScore(globalResidualZ float64, localResidualZ float64,
     geoPremiumZ float64) float64 {
206      return 10000 - 3000*globalResidualZ - 2000*localResidualZ + 3000*geoPremiumZ
207  }
```

B.3 Αναπαραγώγιμη επιλογή δείγματος αλληλεπιδράσεων

Η παρακάτω συνάρτηση δείχνει όλο τον μηχανισμό επιλογής. Η ταξινόμηση πριν από την ψευδοτυχαία δειγματοληψία προστατεύει από αλλαγή σειράς δεδομένων. Το seed και το πλήθος αποθηκεύονται ως μεταδεδομένα, ώστε η διαδικασία να είναι ελέγξιμη.

Πηγή: Ptixiaki/Worker/Models/TreeCode/Tree.py, γραμμές 95-108. Διατηρείται η αρίθμηση πηγής· παραλείπονται οι γραμμές σχολίων και αναδιπλώνονται οπτικά οι μεγάλες γραμμές.

```
 95  def interaction_sample_indices(uids: list[str], sample_size: int, seed: int) ->
     np.ndarray:
 96      if not uids or sample_size <= 0 or seed < 0:
 97          raise ValueError(
 98              "interaction sampling requires rows, a positive sample size and a
     nonnegative seed"
 99          )
100      if len(uids) <= sample_size:
101          return np.arange(len(uids))
104      ordered = np.asarray(sorted(range(len(uids)), key=uids.__getitem__))
105      selected = np.random.default_rng(seed).choice(
106          len(uids), size=sample_size, replace=False
107      )
108      return ordered[np.sort(selected)]
```

Παράρτημα Γ. Χάρτης ανάγνωσης και αναπαραγωγής

Γ.1 Από το σύμπτωμα στο αρχείο

Πίνακας Γ.1. Αφετηρίες ανάγνωσης πηγαίου κώδικα

Θέμα	Αφετηρία πηγαίου κώδικα
Τι ξεκινά πρώτο	Ptixiaki/main.go: main, WarmUpApplication, WarmUpTrees
Πώς εκπαιδεύονται δύο αγορές	main.go: trainResidenceTrees και Worker/Pipelines/TrainingPipeline.go
Ποιες στήλες μπαίνουν στο μοντέλο	Worker/training, Worker/testing/CalculatePrice.go και explainability/contract.go
Πώς δημιουργείται manifest	Tree.py: stage_native_explainability_artifacts και ml_shared/explain_artifacts.py
Ποιος διαβάζει το stdout	Worker/explainability/training.go και records.go
Πώς τυπώνεται πρόοδος	Worker/explainability/progress.go
Πού βρίσκει UBJ η Python	Ptixiaki-ml-microservice/main.py: discover_model_roots
Πώς προστίθεται ακίνητο	Backend/Api/UserActions/AddPropertyHandler.go, UserActions.go, DataBaseActions.go
Πώς διαβάζεται πλήρες detail	Backend/Api/UserActions/PropertyDetails.go
Πώς γίνεται pagination	Backend/Api/UserActions/BrowseProperties.go
Πώς επιλέγεται περιοχή	MarketAnalysis/AreaSearch.go και frontend AreaSearch.tsx
Πώς δημιουργούνται clusters	MarketAnalysis/Kmeans.go και AnalysisActions.go
Πώς αλλάζει σελίδα το React	src/components/Workspace.tsx

Γ.2 Εντολές επαλήθευσης

Οι εντολές δεν περιλαμβάνουν ενεργοποίηση προαιρετικών live database tests και δεν καλούν /train. Η .gocache είναι writable build cache στο συγκεκριμένο workspace. Σε διαφορετικό υπολογιστή επιλέγεται αντίστοιχη διαδρομή. Η Python εντολή πρέπει να εκτελείται με interpreter που διαθέτει και Worker εξαρτήσεις, όχι με ελλιπές service-only περιβάλλον.

```
Set-Location S:\ptixiaki-bundle\Ptixiaki
$env:GOCACHE = 'S:\ptixiaki-bundle\.gocache'
go test ./...
```

```
Set-Location S:\ptixiaki-bundle\Ptixiaki-frontend\Ptixiaki-Frontend
npm run lint
npm run build
npm test
```

```
Set-Location S:\ptixiaki-bundle\Ptixiaki-ml-microservice
python -m unittest -v test_explainability.py
```

Γ.3 Τι απαιτείται για πλήρη επιστημονική αναπαραγωγή

Η αναπαραγωγή μοντέλων απαιτεί περισσότερα από την επιτυχή εκτέλεση των εντολών δοκιμών. Χρειάζεται συγκεκριμένο ανωνυμοποιημένο snapshot δεδομένων, τεκμηρίωση δικαιωμάτων χρήσης, καθαρό πρωτόκολλο split, εκδόσεις interpreters και native runtime, πλήρη manifests και ελεγχόμενη διαμόρφωση βάσης. Κανένα μυστικό ή session αρχείο δεν πρέπει να περιλαμβάνεται στο συνοδευτικό υλικό. Το source bundle πρέπει να καθαριστεί από τέτοια στοιχεία πριν κατατεθεί ή δημοσιευθεί.